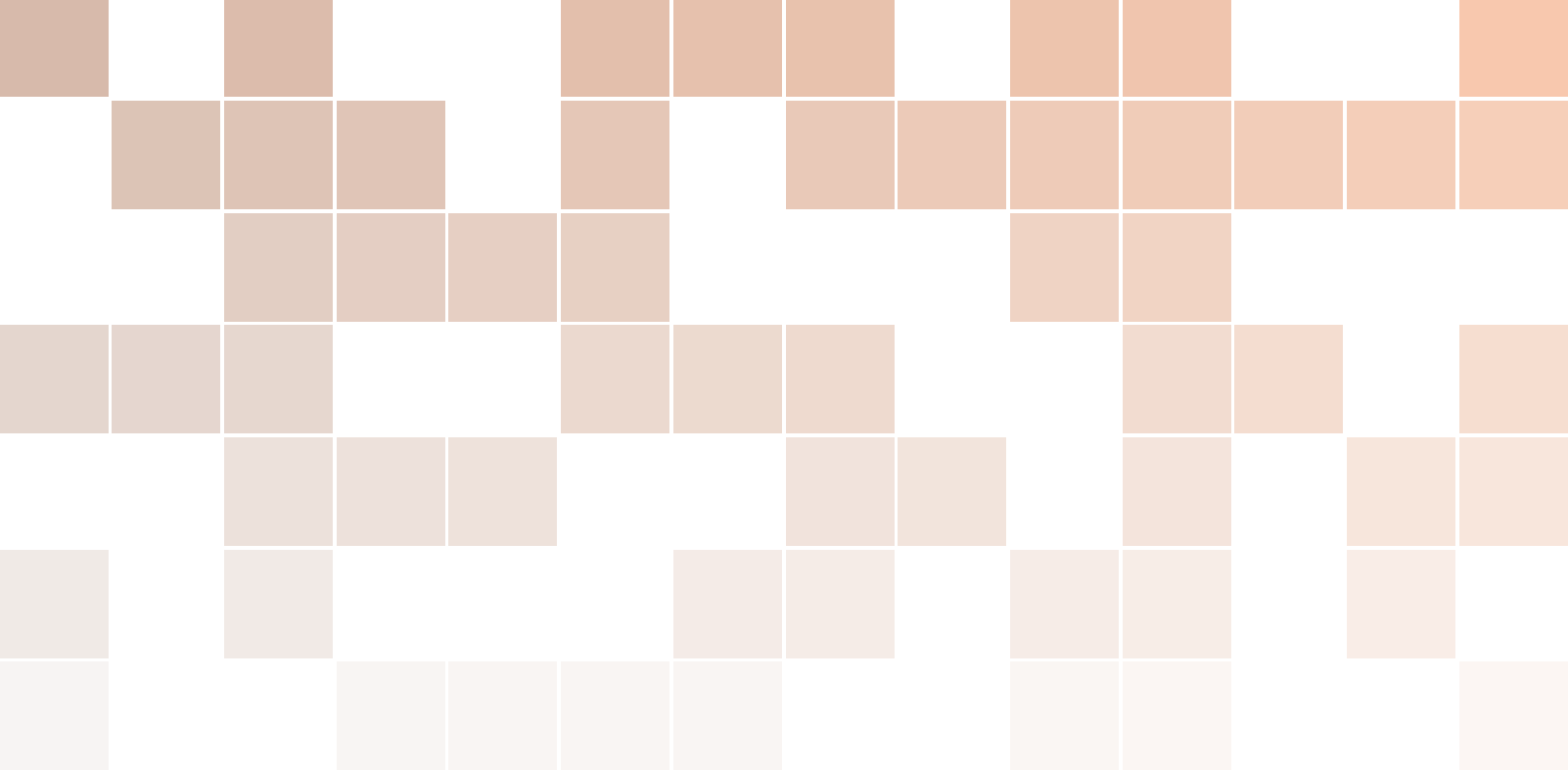
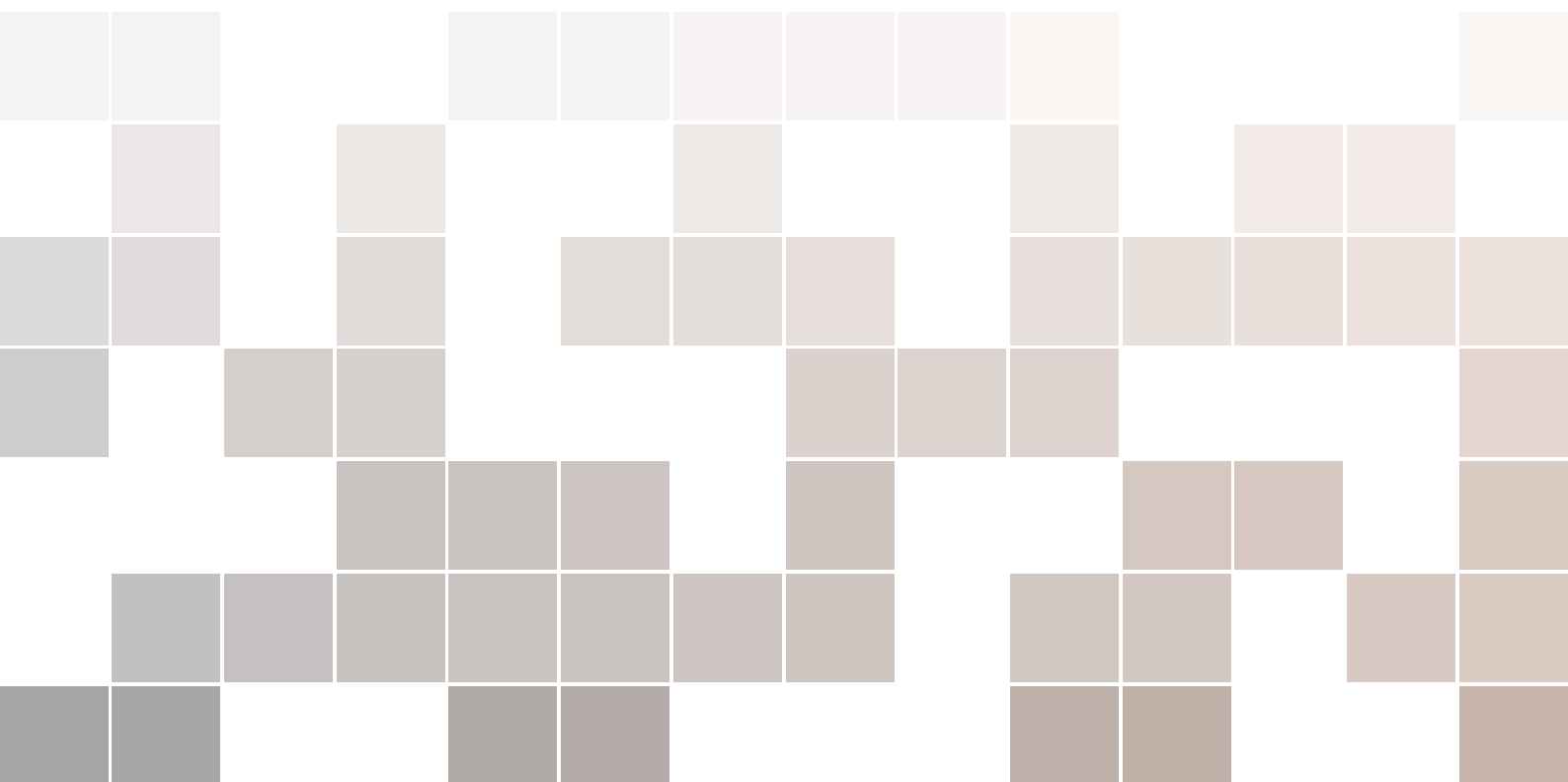




Zaawansowane algorytmy wizyjne

Skrypt do ćwiczeń laboratoryjnych

Tomasz Kryjak, Mateusz Wąsala, Hubert Szolc, Piotr Wzorek, Michał Daniłowicz



Copyright © 2026
Tomasz Kryjak
Mateusz Wąsala
Hubert Szolc
Piotr Wzorek
Krzysztof Błachut
Michał Daniłowicz

ZESPÓŁ WBUDOWANYCH SYSTEMÓW WIZYJNYCH
LABORATORIUM SYSTEMÓW WIZYJNYCH
WEAiIB, AGH W KRAKOWIE

*Wydanie czwarte, zmienione i poprawione
Kraków, luty 2026*

Spis treści

I	Blok 1	
1	Laboratorium 1	
1	Algorytmy wizyjne w Python 3.X – wstęp	11
1.1	Wykorzystywane oprogramowanie	11
1.2	Moduły Pythona wykorzystywane w przetwarzaniu obrazów	11
1.3	Operacje wejścia/wyjścia	12
1.4	Konwersje przestrzeni barw	13
1.4.1	OpenCV	13
1.4.2	Matplotlib	13
1.5	Skalowanie, zmiana rozdzielczości przy użyciu OpenCV	14
1.6	Operacje arytmetyczne: dodawanie, odejmowanie, mnożenie, moduł z różnicy	14
1.7	Wyliczenie histogramu	15
1.8	Wyrównywanie histogramu	15
1.9	Filtracja	16
2	Laboratorium 2	
2	Detekcja pierwszoplanowych obiektów ruchomych	19
2.1	Wczytywanie sekwencji obrazów	19

2.2	Odejmowanie ramek i binaryzacja	20
2.3	Operacje morfologiczne	20
2.4	Indeksacja i prosta analiza	21
2.5	Ewaluacja wyników detekcji obiektów pierwszoplanowych	22
2.6	Przykładowe rezultaty algorytmu do detekcji ruchomych obiektów pierwszoplanowych	23

3

Laboratorium 3

3	Segmentacja obiektów pierwszoplanowych	27
3.1	Cel zajęć	27
3.2	Segmentacja obiektów pierwszoplanowych	27
3.3	Metody oparte o bufor próbek	28
3.4	Aproksymacja średniej i mediany (tzw. sigma-delta)	30
3.5	Polityka aktualizacji	30
3.6	OpenCV – GMM/MOG	31
3.7	OpenCV – KNN	31
3.8	Przykładowe rezultaty algorytmu do segmentacji obiektów pierwszoplanowych	31
3.9	Wykorzystanie sieci neuronowej do segmentacji obiektów pierwszoplanowych	34

4

Extra laboratorium 1

4	Uogólniona transformata Hougha	37
4.1	Cel zajęć	37
4.1.1	R-table	37
4.2	Implementacja uogólnionej transformaty Hougha	37
4.3	Wyszukiwanie wzorców różniących się orientacją	40

5

Mini projekt 1

5	Projekt 1	45
5.1	Cel projektu	46
5.2	Przykładowa literatura	46

II

Blok 2

6	Laboratorium 6	
6	Punkty Charakterystyczne	53
6.1	Cel zajęć	53
6.2	Detekcja narożników metodą Harrisa – teoria	53
6.3	Implementacja metody Harrisa	54
6.4	Prosta deskrypcja punktów charakterystycznych	55
6.5	ORB (FAST+BRIEF)	56
6.6	Wykorzystanie punktów charakterystycznych do łączenia obrazów – panorama	59
7	Laboratorium 7	
7	Przepływ optyczny	63
7.1	Cel zajęć	63
7.2	Przepływ optyczny (ang. <i>optical flow</i>)	63
7.3	Implementacja metody blokowej	64
7.4	Implementacja wyliczania w wielu skalach	66
7.5	Inne sposoby wyznaczania przepływu optycznego	70
7.6	Detekcja kierunku ruchu i klasyfikacja obiektów z wykorzystaniem przepływu optycznego	71
8	Laboratorium 8	
8	Śledzenie obiektów	77
8.1	Filtr korelacyjny	77
8.2	Filtr korelacyjny - zadanie	79
8.3	Syjamska sieć neuronowa	79
8.4	Sieć syjamska - zadanie	80
9	Extra laboratorium 2	
9	Detekcja sylwetek ludzkich	83
9.1	Cel zajęć:	83
9.2	Histogram of Oriented Gradients – opis algorytmu	83
9.3	Support Vector Machines – opis algorytmu	86
9.4	Deskryptor HOG	89
9.5	Klasyfikator SVM	91
9.6	Detekcja sylwetki ludzkiej	92

10	Mini projekt 2	
10	Projekt 2	97
10.1	Cel projektu	97
10.2	Przykładowa literatura	98
III	Blok 3	
11	Laboratorium 11	
11	Kalibracja kamery i stereowizja	103
11.1	Kalibracja pojedynczej kamery	105
11.2	Kalibracja układu kamer	108
11.3	Obliczenia korespondencji stereo	109
11.4	Wykorzystanie sieci neuronowej do realizacji zadania analizy głębi obrazu	111
12	Laboratorium 12	
12	Termowizja	115
12.1	Prosta analiza obrazu termowizyjnego	115
12.2	Wykorzystanie detektora bazującego na sieci neuronowej do analizy obrazu RGB + Termowizja (YOLOv3)	116
12.3	Detekcja obiektów z wykorzystaniem wzorca probabilistycznego	117
13	Extra laboratorium 3	
13	Odometria wizyjna	123
13.1	Odometria wizyjna	123
13.2	Śledzenie obiektów - zadanie	123

1

Laboratorium 1

1	Algorytmy wizyjne w Python 3.X – wstęp 11
1.1	Wykorzystywane oprogramowanie
1.2	Moduły Pythona wykorzystywane w przetwarzaniu obrazów
1.3	Operacje wejścia/wyjścia
1.4	Konwersje przestrzeni barw
1.5	Skalowanie, zmiana rozdzielczości przy użyciu OpenCV
1.6	Operacje arytmetyczne: dodawanie, odejmowanie, mnożenie, moduł z różnicą
1.7	Wyliczenie histogramu
1.8	Wyrównywanie histogramu
1.9	Filtracja

1. Algorytmy wizyjne w Python 3.X – wstęp

W ramach ćwiczenia zaprezentowane/przypomniane zostaną podstawowe informacje związane z wykorzystaniem języka Python do realizacji operacji przetwarzania i analizy obrazów, a także sekwencji wideo.

1.1 Wykorzystywane oprogramowanie

Przed rozpoczęciem ćwiczeń **proszę utworzyć** własny katalog roboczy w miejscu podanym przez Prowadzącego. Nazwa katalogu powinna być związana z Państwa nazwiskiem – zalecana forma to `NazwiskoImie` bez polskich znaków.

Do przeprowadzania ćwiczeń można wykorzystać jedno z trzech środowisk programowania w Pythonie – Spyder5, PyCharm lub Visual Studio Code (VSCode). Ich zaletą jest możliwość łatwego podglądu tablic dwuwymiarowych (czyli obrazów). Spyder jest prostszy, ale także ma mniejsze możliwości i więcej niedociągnięć. PyCharm jest oprogramowaniem komercyjnym udostępnianym także w wersji darmowej (wygląd zbliżony do CLion dla C/C++). W PyCharmie pracę należy zacząć od utworzenia projektu, Spyder pozwala na pracę na pojedynczych plikach (bez konieczności tworzenia projektu).

Visual Studio Code jest darmowym, lekkim, intuicyjnym i łatwym w obsłudze edytorem kodu źródłowego. Jest to uniwersalne oprogramowanie do dowolnego języka programistycznego. Konfiguracja programu do ćwiczeń sprowadza się jedynie do wybrania odpowiedniego interpretera, a dokładnie odpowiedniego środowiska wirtualnego w języku Python.

1.2 Moduły Pythona wykorzystywane w przetwarzaniu obrazów

Python udostępnia wbudowane kontenery (np. listy), ale nie oferuje natywnego typu tablicowego zaprojektowanego do wydajnych operacji wektorowych/element-po-element (np. dodawania, mnożenia czy progowania całych bloków danych). W praktyce standardem jest użycie modułu *NumPy*, który dostarcza typ `ndarray` oraz mechanizmy takie jak wektoryzacja i *broadcasting*. Obraz cyfrowy można wtedy traktować jako tablicę $H \times W$ (skala

szarości) lub $H \times W \times C$ (kolor), zwykle o typie `uint8` (zakres 0–255) albo `float` (np. 0–1). *NumPy* stanowi fundament dla większości bibliotek używanych dalej do przetwarzania i wizualizacji obrazów.

Najczęściej spotkane pakiety wspierające przetwarzanie obrazów to m.in.:

- *SciPy* (moduł *ndimage*) – podstawowe operacje filtracji i przekształceń (również wielowymiarowych) oraz *Matplotlib* (*pyplot*) – wizualizacja obrazów i wykresów,
- *Pillow* (fork starszego, nierozwijanego modułu PIL) – wygodne wczytywanie/zapisywanie i proste operacje na obrazach,
- *OpenCV* (moduł *cv2*) – rozbudowany zestaw algorytmów widzenia komputerowego (filtry, geometria, cechy, dopasowanie, kalibracja, itp.),
- *scikit-image* (*skimage*) – biblioteka z algorytmami przetwarzania obrazów zintegrowana z ekosystemem naukowym Pythona,
- *Plotly* (*plotly*) – interaktywne wykresy i wizualizacje (np. mapy cieplne, wykresy 3D, dashboardy w Jupyter/HTML),
- *Rerun* (*rerun*, *rerun.io*) – logowanie i interaktywna wizualizacja danych (np. obrazy, adnotacje, 3D/robotyka) w viewerze Rerun,
- *pandas* – narzędzia do pracy z danymi tabelarycznymi (np. wczytywanie adnotacji i metadanych z CSV/Excel, analiza wyników eksperymentów).

W tym kursie oprzemy się głównie na *OpenCV*. Równolegle będziemy używać *Matplotlib* (zwłaszcza do wyświetlania wyników; uwaga: *OpenCV* domyślnie przechowuje kolor w kolejności BGR, a *Matplotlib* oczekuje RGB) oraz sporadycznie *ndimage* – gdy dana operacja nie jest dostępna w *OpenCV* albo jest tam mniej wygodna w użyciu.

1.3 Operacje wejścia/wyjścia

Wczytywanie, wyświetlanie i zapisywanie obrazu z wykorzystaniem OpenCV oraz Matplotlib

Ćwiczenie 1.1 Wykonaj zadanie, w którym przećwiczysz obsługę plików z wykorzystaniem *OpenCV* oraz *Matplotlib*.

1. Ze strony kursu pobierz obraz *mandril.jpg* i umieść go we własnym katalogu roboczym.
2. Wczytaj, wyświetl oraz zapisz obraz z rozszerzeniem *.png* pod nową nazwą wykorzystując bibliotekę *OpenCV* oraz *Matplotlib*. Następnie umieść na obrazie **punkt** i **prostokąt** oraz wyświetl na ekranie.



Wykorzystaj:

- *OpenCV*: `cv2.imread()`, `cv2.imshow()`, `cv2.imwrite()`, `cv2.waitKey()`, `cv2.destroyAllWindows()`, `cv2.rectangle()`.
- *Matplotlib*: `plt.imread()`, `plt.imsave()`, `plt.figure()`, `plt.imshow()`, `plt.axis()`, `plt.show()` oraz `matplotlib.patches.Rectangle()`.



Funkcja `waitKey()` wyświetla obraz przez określony czas (argument 0 oznacza oczekiwanie na naciśnięcie klawisza). Niepotrzebne okno można zamknąć poleceniem `destroyWindow()` z odpowiednim parametrem, a wszystkie otwarte okna — poleceniem `destroyAllWindows()`. Dobrą praktyką jest zawsze stosowanie funkcji `cv2.destroyAllWindows()`.

Rozliczenie ćwiczenia

W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL.

**1.4 Konwersje przestrzeni barw****1.4.1 OpenCV**

Do konwersji przestrzeni barw służy funkcja `cvtColor`.

```
IG = cv2.cvtColor(I, cv2.COLOR_BGR2GRAY)
IHSV = cv2.cvtColor(I, cv2.COLOR_BGR2HSV)
```

Uwaga! Proszę zauważyć, że w OpenCV odczyt jest w kolejności BGR, a nie RGB. Może to być istotne w przypadku ręcznego manipulowania pikselami. Pełna lista dostępnych konwersji wraz ze stosownymi wzorami w [dokumentacji OpenCV](#)

Ćwiczenie 1.2 Wykonaj konwersję przestrzeni barw podanego obrazu.

1. Dokonać konwersji obrazu *mandril.jpg* do odcieni szarości i przestrzeni HSV. Wynik wyświetlić.
2. Wyświetlić składowe H, S, V obrazu po konwersji.

Rozliczenie ćwiczenia

W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL.



Przydatna składnia:

```
IH = IHSV[:, :, 0]
IS = IHSV[:, :, 1]
IV = IHSV[:, :, 2]
```

1.4.2 Matplotlib

Tu wybór dostępnych konwersji jest dość ograniczony.

1. RGB do odcieni szarości. Można wykorzystać rozbiecie na poszczególne kanały i wzór:

$$G = 0.299 \cdot R + 0.587 \cdot G + 0.144 \cdot B \quad (1.1)$$

Całość można opakować w funkcję:

```
def rgb2gray(I):
    return 0.299*I[:, :, 0] + 0.587*I[:, :, 1] + 0.114*I[:, :, 2]
```

Uwaga! Przy wyświetlaniu należy ustawić mapę kolorów. Inaczej obraz wyświetli się w domyślnej, która nie jest bynajmniej w odcieniach szarości: `plt.gray()`.

2. RGB do HSV.

```
import matplotlib # add at the top of the file
I_HSV = matplotlib.colors.rgb_to_hsv(I)
```

1.5 Skalowanie, zmiana rozdzielczości przy użyciu OpenCV

Ćwiczenie 1.3 Przeskaluj obraz *mandril*. Do skalowania służy funkcja `resize`.

Rozliczenie ćwiczenia

W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

Przykład użycia:

```
height, width = I.shape[:2] # retrieving elements 1 and 2, i.e. the corresponding
                             height and width
scale = 1.75 # scale factor
Ix2 = cv2.resize(I, (int(scale*height), int(scale*width)))
cv2.imshow("Big Mandril", Ix2)
```

1.6 Operacje arytmetyczne: dodawanie, odejmowanie, mnożenie, moduł z różnicy

Obrazy są macierzami, a zatem operacje arytmetyczne są dość proste – tak jak w pakiecie Matlab. Należy oczywiście pamiętać o konwersji na odpowiedni typ danych. Zwykle dobrym wyborem będzie `double`.

Ćwiczenie 1.4 Wykonaj operacje arytmetyczne na obrazie *lena*.

1. Pobierz ze strony kursu obraz *lena*, a następnie go wczytaj za pomocą funkcji z OpenCV – dodaj ten fragment kodu do pliku, który zawiera wczytywanie obrazu *mandril*. Wykonaj konwersję do odcieni szarości. Dodaj macierze zawierające mandryla i Leny w skali szarości. Wyświetl wynik.
2. Podobnie wykonaj odjęcie i mnożenie obrazów.
3. Zaimplementuj kombinację liniową obrazów.
4. Ważną operacją jest moduł z różnicy obrazów. Można ją wykonać „ręcznie” – konwersja na odpowiedni typ, odjęcie, moduł (`abs`), konwersja na `uint8`. Alternatywa to wykorzystanie funkcji `absdiff` z OpenCV. Proszę obliczyć moduł z różnicy „szarych” wersji mandryla i Leny.

Rozliczenie ćwiczenia

W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

Uwaga! Przy wyświetleniu obraz może nie być poprawny, ponieważ jest on typu *float64* (*double*). W praktyce należy:

1. ograniczyć zakres wartości (*clip*),
2. ewentualnie przeskalować do przedziału *[0,255]*,
3. skonwertować do *uint8*.

```
img8 = (255*np.clip(img,0,1)).astype(np.uint8) # when values are in [0,1] range
img8 = np.clip(img,0,255).astype(np.uint8) # when values are in [0,255] range
```

1.7 Wylczenie histogramu

Obliczanie histogramu można wykonać z wykorzystaniem funkcji *calcHist*. Zanim jednak do tego przejdziemy, przypomnijmy sobie podstawowe struktury sterowania w Pythonie: funkcje i podprogramy. Proszę samodzielnie dokończyć poniższą funkcję, która oblicza histogram obrazu w 256 odcieniach szarości:

```
def hist(img):
    h=np.zeros((256,1), np.float32) # creates and zeros single-column arrays
    height, width = img.shape[:2] # shape - we take the first 2 values
    for y in range(height):
        ...
    return h
```

Histogram można wyświetlić, korzystając z funkcji *plt.hist* lub *plt.plot* z biblioteki *Matplotlib*.

Funkcja *calcHist* umożliwia obliczenie histogramu dla kilku obrazów (lub ich składowych), dlatego jako parametry otrzymuje tablice (np. obrazów), a nie pojedynczy obraz. Najczęściej jednak wykorzystywana jest następująca postać:

```
hist = cv2.calcHist([IG],[0],None,[256],[0,256])
# [IG] -- input image
# [0] -- for greyscale images there is only one channel
# None -- mask (you can count the histogram of a selected part of the image)
# [256] -- number of histogram bins
# [0, 256] -- the range over which the histogram is calculated
```

Proszę sprawdzić czy histogramy uzyskane obiema metodami są takie same.

1.8 Wyrównywanie histogramu

Wyrównywanie histogramu to popularna i ważna operacja przetwarzania wstępnego.

1. Wyrównywanie "klasyczne" jest metodą globalną — wykonuje się je na całym obrazie. W bibliotece OpenCV znajduje się gotowa funkcja realizująca ten typ wyrównania:

```
IGE = cv2.equalizeHist(IG)
```

2. Wyrównywanie CLAHE (ang. *Contrast Limited Adaptive Histogram Equalization*) – metoda adaptacyjna, która poprawia warunki oświetleniowe na obrazie. W przeciwieństwie do klasycznego wyrównywania histogramu działa lokalnie: wyrównuje histogram w poszczególnych fragmentach obrazu, a nie dla całego obrazu. Metoda działa następująco:

- podział obrazu na rozłączne (kwadratowe) bloki,
- wyznaczenie histogramu w każdym bloku,

- wyrównanie histogramu z ograniczeniem maksymalnej „wysokości” (nadmiar jest redystrybuowany na sąsiednie przedziały),
- interpolacja wartości pikseli na podstawie histogramów dla sąsiednich bloków (zwykle z uwzględnieniem czterech najbliższych sąsiadów).

Szczegóły znajdują się na [Wiki](#) oraz w [tutorialu OpenCV](#).

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
# clipLimit - maximum height of the histogram bar - values above are distributed
# among neighbours
# tileGridSize - size of a single image block (local method, operates on separate
# image blocks)
I_CLAHE = clahe.apply(IG)
```

Ćwiczenie 1.5 Uruchom i porównaj obie metody wyrównywania.

Rozliczenie ćwiczenia

W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

1.9 Filtracja

Filtracja to bardzo ważna grupa operacji na obrazach. W ramach ćwiczenia proszę uruchomić:

- filtrację Gaussa (`GaussianBlur`)
- filtrację Sobela (`Sobel`)
- Laplasjan (`Laplacian`)
- medianę (`medianBlur`)

 Pomocna będzie [dokumentacja OpenCV](#).

Proszę zwrócić uwagę również na inne dostępne funkcje:

- filtrację bilateralną,
- filtry Gabora,
- operacje morfologiczne.

Ćwiczenie 1.6 Uruchom i porównaj wymienione metody.

Rozliczenie ćwiczenia

W celu rozliczenia ćwiczenia po wykonaniu zadania zgłoś prowadzącemu zajęcia swoją gotowość. Alternatywnie możesz zgłosić gotowość po wykonaniu wszystkich zadań dotyczących omawianego tematu zajęć. Po akceptacji z jego strony umieść skrypt z rozszerzeniem `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL. ■

2

Laboratorium 2

2	Detekcja pierwszoplanowych obiektów ruchomych	19
2.1	Wczytywanie sekwencji obrazów	
2.2	Odejmowanie ramek i binaryzacja	
2.3	Operacje morfologiczne	
2.4	Indeksacja i prosta analiza	
2.5	Ewaluacja wyników detekcji obiektów pierwszoplanowych	
2.6	Przykładowe rezultaty algorytmu do detekcji ruchomych obiektów pierwszoplanowych	

2. Detekcja pierwszoplanowych obiektów ruchomych

Co to są obiekty pierwszoplanowe?

To obiekty, które są dla nas (w kontekście rozważanej aplikacji) istotne. Najczęściej są to: ludzie, zwierzęta, samochody (lub inne pojazdy) oraz bagaże (potencjalne bomby). Definicja jest więc ściśle związana z docelową aplikacją.

Czy segmentacja obiektów pierwszoplanowych to segmentacja obiektów ruchomych?

Nie. Po pierwsze, obiekt, który się zatrzymał, nadal może być dla nas interesujący (np. człowiek stojący przed przejściem dla pieszych). Po drugie, istnieje wiele ruchomych elementów sceny, które nie są dla nas istotne (np. płynąca woda, fontanna, poruszające się drzewa czy krzaki). Warto też zauważyć, że w przetwarzaniu obrazu często analizuje się ruch. Dlatego detekcję obiektów pierwszoplanowych można (i warto) wspomagać detekcją obiektów ruchomych.

Najprostsza metoda detekcji obiektów ruchomych polega na odejmowaniu kolejnych (sąsiednich) ramek. W ramach ćwiczenia zrealizujemy proste odejmowanie dwóch ramek, połączone z binaryzacją, indeksacją i analizą otrzymanych obiektów. Na koniec spróbujemy potraktować wynik odejmowania jako rezultat segmentacji obiektów pierwszoplanowych i sprawdzimy jakość tej segmentacji.

2.1 Wczytywanie sekwencji obrazów

Wykorzystane sekwencje pochodzą ze zbioru danych dostępnego na stronie changedetection.net. W razie problemów z dostępem do serwisu zbiór jest udostępniony na platformie UPeL. Zbiór zawiera sekwencje z etykietami, tzn. każdej klatce obrazu przypisano referencyjną maskę obiektów (ang. *ground truth*), w której każdy piksel należy do jednej z pięciu kategorii: tło (0), cień (50), poza obszarem zainteresowania (85), nieznany ruch (170) oraz

obiekty pierwszoplanowe (255) – w nawiasach podano odpowiadające poziomy szarości.

W ramach ćwiczenia interesuje nas jedynie podział na obiekty pierwszoplanowe oraz pozostałe kategorie. Dodatkowo w folderze znajduje się maska obszaru zainteresowania (ROI) oraz plik tekstowy z przedziałem czasowym, dla którego należy analizować wyniki (`temporalROI.txt`) – szczegóły w dalszej części ćwiczenia.

Ćwiczenie 2.1 Wczytaj sekwencje. ■

2.2 Odejmowanie ramek i binaryzacja

W celu detekcji elementów ruchomych od ramki bieżącej odejmujemy ramkę poprzednią; należy zaznaczyć, że w praktyce obliczamy wartość bezwzględną tej różnicy. Następnie, aby dokonać binaryzacji, trzeba wyznaczyć próg i dobrać go tak, aby obiekty były względnie wyraźne. Proszę zwrócić uwagę na artefakty związane z kompresją.

R Aby uniknąć problemów z odejmowaniem liczb bez znaku (*uint8*), należy wykonać konwersję do typu *int* – `IG = IG.astype('int')`.

```
(T, thresh) = cv2.threshold(D, 10, 255, cv2.THRESH_BINARY)
# D -- input array
# 10 -- threshold value
# 255 -- maximum value to use with the THRESH_BINARY and THRESH_BINARY_INV
#       thresholding types
# cv2.THRESH_BINARY -- thresholding type
# T -- our threshold value
# thresh -- output image
```

R Pierwszy argument zwracanych wartości przez funkcję `cv2.threshold` to próg binaryzacji (jest on użyteczny w przypadku stosowania automatycznego wyznaczenia progu np. metodą Otsu lub trójkątów). Szczegóły w dokumentacji OpenCV.

Ćwiczenie 2.2 Wykonaj odpowiednie operacje, aby uzyskać zbinaryzowany obraz. ■

2.3 Operacje morfologiczne

Operacje morfologiczne to proste operacje przetwarzania obrazu binarnego (czasem także w skali szarości), które modyfikują kształt obiektów na podstawie tzw. elementu strukturalnego (np. 3×3).

- erozja (*erosion*) – redukuje obiekty: zmniejsza je, usuwa drobne jasne szумы, może rozrywać cienkie połączenia,
- dylatacja (*dilation*) – rozszerza obiekty: powiększa je, domyka małe przerwy/dziury, skleja bliskie fragmenty,
- otwarcie (*opening* = erozja → dylatacja) – usuwa drobne obiekty/szum, wygładza kontur,
- domknięcie (*closing* = dylatacja → erozja) – wypełnia małe dziury i przerwy, łączy szczeliny.

Uzyskany obraz jest dość zaszumiony. Celem jest uzyskanie jak najlepiej widocznej sylwetki przy możliwie małych zakłóceniach. Dla poprawy wyniku warto dodać etap filtracji medianowej (przed operacjami morfologicznymi) oraz w razie potrzeby skorygować próg binaryzacji.

Ćwiczenie 2.3 Wykonaj filtrację, wykorzystując erozję i dylatację (`erode` i `dilate` z OpenCV).

2.4 Indeksacja i prosta analiza

W kolejnym etapie należy przefiltrować uzyskany wynik. W tym celu zostanie wykorzystana indeksacja (nadanie etykiet grupom połączonych pikseli) oraz obliczenie parametrów tych grup. Służy do tego funkcja `connectedComponentsWithStats`. Wywołanie:

```
retval, labels, stats, centroids = cv2.connectedComponentsWithStats(B)
# retval -- total number of unique labels
# labels -- destination labelled image
# stats -- statistics output for each label, including the background label.
# centroids -- centroid output for each label, including the background label.
```

R Przy wyświetlaniu obrazu *labels* trzeba w odpowiedni sposób ustawić format oraz dodać skalowanie.

Do skalowania można wykorzystać informację o liczbie znalezionych obiektów.

```
cv2.imshow("Labels", np.uint8(labels / retval * 255))
```

Następnie wyświetl prostokąt otaczający (*bounding box*), pole i indeks dla największego obiektu. Poniżej znajduje się przykładowe rozwiązanie zadania. Proszę je uruchomić i ewentualnie spróbować je zoptymalizować.

```
I_VIS = I # copy of the input image

if (stats.shape[0] > 1):    # are there any objects

    tab = stats[1:,4] # 4 columns without first element
    pi = np.argmax( tab ) # finding the index of the largest item
    pi = pi + 1 # increment because we want the index in stats, not in tab
    # drawing a bbox
    cv2.rectangle(I_VIS, (stats[pi,0], stats[pi,1]), (stats[pi,0]+stats[pi,2], stats[pi,1]+stats[pi,3]), (255,0,0), 2)
    # print information about the field and the number of the largest element
    cv2.putText(I_VIS, "%f" % stats[pi,4], (stats[pi,0], stats[pi,1]), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255,0,0))
    cv2.putText(I_VIS, "%d" % pi, (np.int(centroids[pi,0]), np.int(centroids[pi,1])), cv2.FONT_HERSHEY_SIMPLEX, 1, (255,0,0))
```

Komentarze do przykładowego rozwiązania:

- `stats.shape[0]` to liczba obiektów. Ponieważ funkcja zlicza też obiekt o indeksie 0 (tj. tło), w sprawdzeniu czy są obiekty warunek jest `> 1`
- kolejne dwie linie to obliczenie indeksu maksimum z kolumny numer 4 (pole).
- uzyskany indeks należy inkrementować, ponieważ w analizie pominęliśmy element 0 (tło).
- do rysowania prostokąta otaczającego na obrazie wykorzystujemy funkcję `rectangle` z OpenCV. Składnia `"%f" % stats[pi,4]` pozwala wypisać wartość w odpowiednim formacie (f - float). Kolejne parametry to współrzędne dwóch przeciwległych wierzchołków prostokąta. Następnie kolor w formacie (B, G, R), a na końcu grubość linii. Szczegóły w dokumentacji funkcji.

- do wypisywania tekstu na obrazie służy funkcja `putText`. Do określenia pozycji pola tekstowego podaje się współrzędne lewego dolnego wierzchołka. Następnie czcionka (pełna lista w dokumentacji), rozmiar i kolor.

Ćwiczenie 2.4 Wykorzystaj funkcję `cv2.connectedComponentsWithStats` do indeksacji oraz oblicz parametry otrzymanych obiektów. Wyświetl prostokąt otaczający, pole oraz indeks dla największego obiektu. Na podstawie wskazówek i przykładów zamieszczonych w tekście powyżej spróbuj zoptymalizować rozwiązanie. ■

2.5 Ewaluacja wyników detekcji obiektów pierwszoplanowych

Aby ocenić algorytm detekcji obiektów pierwszoplanowych, należy porównać zwracaną przez niego maskę obiektów z maską referencyjną (ang. *ground truth*). Porównywanie odbywa się na poziomie poszczególnych pikseli. Jeśli wykluczy się cienie (tak jak założyliśmy na wstępie), możliwe są cztery sytuacje:

- *TP* – wynik prawdziwie dodatni (ang. *true positive*) – piksel należący do obiektu z pierwszego planu jest wykrywany jako piksel należący do obiektu z pierwszego planu,
- *TN* – wynik prawdziwie ujemny (ang. *true negative*) – piksel należący do tła jest wykrywany jako piksel należący do tła,
- *FP* – wynik fałszywie dodatni (ang. *false positive*) – piksel należący do tła jest wykrywany jako piksel należący do obiektu z pierwszego planu,
- *FN* – wynik fałszywie ujemny (ang. *false negative*) – piksel należący do obiektu jest wykrywany jako piksel należący do tła.

Na podstawie tych wartości można policzyć szereg miar. W dalszej części wykorzystamy trzy: precyzję (ang. *precision* – *P*), czułość (ang. *recall* – *R*) oraz tzw. miarę *F1*. Zdefiniowane są one następująco:

$$P = \frac{TP}{TP + FP} \quad (2.1)$$

$$R = \frac{TP}{TP + FN} \quad (2.2)$$

$$F1 = \frac{2PR}{P + R} \quad (2.3)$$

Miara *F1* jest z zakresu $[0;1]$, przy czym im jej wartość jest większa, tym lepiej.

Ćwiczenie 2.5 Zaimplementuj obliczanie miar *P*, *R* i *F1*.



Przy czym obliczenia wykonujemy tylko wtedy, gdy dostępna jest poprawna mapa referencyjna. W tym celu należy sprawdzić zależność licznika ramki i wartości z pliku `temporalROI.txt` – musi się on zawierać w zakresie tam opisanym. ■

Ćwiczenie 2.6 Podsumowując, należy wykonać detekcję obiektów ruchomych:

1. Wczytaj sekwencję `pedestrian`, a następnie `highway` i `office`.
2. Dokonaj detekcji zmian: odejmowanie klatek i binaryzacja.
3. Usuń szum (np. filtrem medianowym lub Gaussa).
4. Zastosuj operacje morfologiczne.
5. Wykonaj indeksację.
6. Przeprowadź ewaluację.

Rozliczenie ćwiczenia

W celu rozliczenia ćwiczenia, po wykonaniu zadania zgłoś swoją gotowość prowadzącemu zajęcia. Równocześnie możesz zgłosić swoją gotowość po wykonaniu wszystkich zadań dotyczących poruszanego tematu zajęć. Po akceptacji z jego strony, umieść skrypt o rozszerzeniu `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL.



Informacje dotyczące kolejnych zajęć.

1. W ramach dalszych ćwiczeń będziemy poznawać kolejne algorytmy i funkcjonalności języka Python. Jednak nie kładziemy szczególnej uwagi na poznawanie języka Python, ale na wykorzystaniu w praktyce kolejnych algorytmów pojawiających się na wykładach. Przedmiot Zaawansowane Algorytmy Wizyjne nie jest kursem Pythona!
2. Przedstawione rozwiązania należy zawsze traktować jako przykładowe. Na pewno problem można rozwiązać inaczej, a czasem lepiej.

2.6 Przykładowe rezultaty algorytmu do detekcji ruchomych obiektów pierwszoplanowych

W celu lepszej weryfikacji poszczególnych etapów zaproponowanej metody do detekcji ruchomych obiektów pierwszoplanowych przedstawiony zostanie przykładowy wynik dla obrazu o indeksie 350.



Rysunek 2.1: Przykładowy wynik poszczególnych etapów algorytmu. Od lewej obraz wejściowy z prostokątem otaczającym, obraz po binaryzacji, obraz po zastosowaniu filtracji medianowej oraz operacji morfologicznych (erode, dilate), obraz reprezentujący etykiety po indeksacji, obraz referencyjny.

3

Laboratorium 3

3	Segmentacja obiektów pierwszoplanowych	27
3.1	Cel zajęć	
3.2	Segmentacja obiektów pierwszoplanowych	
3.3	Metody oparte o bufor próbek	
3.4	Aproksymacja średniej i mediany (tzw. sigma-delta)	
3.5	Polityka aktualizacji	
3.6	OpenCV – GMM/MOG	
3.7	OpenCV – KNN	
3.8	Przykładowe rezultaty algorytmu do segmentacji obiektów pierwszoplanowych	
3.9	Wykorzystanie sieci neuronowej do segmentacji obiektów pierwszoplanowych	

3. Segmentacja obiektów pierwszoplanowych

3.1 Cel zajęć

- zapoznanie się z zagadnieniem segmentacji obiektów pierwszoplanowych oraz problemami z tym związanymi,
- implementacja prostych algorytmów modelowania tła opartych na buforze próbek – analiza ich wad i zalet,
- implementacja algorytmów średniej bieżącej oraz aproksymacji medianowej – analiza ich wad i zalet,
- poznanie metod dostępnych w *OpenCV* – modelu Gaussian Mixture Model (zwanego też Mixture of Gaussians) oraz metody opartej na algorytmie KNN (K-Nearest Neighbours, k-najbliższych sąsiadów),
- zapoznanie się z architekturą sieci neuronowej oraz przykładową aplikacją.

3.2 Segmentacja obiektów pierwszoplanowych

Segmentacja obiektów pierwszoplanowych (ang. *foreground object segmentation*)

Segmentacja polega na wydzieleniu z obrazu interesujących obiektów. Nie musi to oznaczać *klasyfikacji* (np. *samochód, pieszy*); często wystarcza informacja, *gdzie* na obrazie znajduje się obiekt. Pojęcie *obektu pierwszoplanowego* jest zależne od aplikacji (np. w monitoringu: ludzie i pojazdy).

Model tła

W najprostszym ujęciu jest to obraz „pustej sceny”, tj. bez obiektów pierwszoplanowych. W praktyce, w wielu metodach model tła jest niejawny (nie jest pojedynczym obrazem) i nie da się go wprost wyświetlić (np. GMM, ViBE, PBAS).

Inicjalizacja modelu tła

Model trzeba zainicjalizować przy starcie algorytmu. Najczęściej jako start przyjmuje się

pierwszą ramkę (lub pierwsze N ramek w metodach buforowych). Założenie: w fazie inicjalizacji nie ma obiektów pierwszoplanowych. Jeśli obiekty są obecne, model startuje z błędem, co w literaturze określa się jako *bootstrap*. Bardziej odporne podejścia analizują zmiany w czasie (i czasem także spójność przestrzenną), zakładając, że tło jest widoczne przez większość czasu.

Modelowanie (generacja) tła

Statyczny „obraz tła” działa tylko w ściśle kontrolowanych warunkach. W typowych scenach tło zmienia się (oświetlenie, pora dnia) albo przestawiane są elementy sceny (np. krzesło). Dlatego model tła powinien się adaptować; proces tej aktualizacji nazywa się modelowaniem/generacją tła. Brak adaptacji (albo zbyt wolna adaptacja) prowadzi do fałszywych detekcji, m.in. *ghost* (*duch*).

Pułapki przy modelowaniu tła

Nie istnieje jedna metoda dobra dla wszystkich scen. Typowe sytuacje problematyczne:

- szum i artefakty kompresji (MJPEG, H.264/265, MPEG),
- drżenie kamery,
- automatyka kamery (balans bieli, ekspozycja),
- zmiany oświetlenia: powolne (pora dnia) i nagle,
- obiekty obecne podczas inicjalizacji,
- ruchome elementy tła,
- kamuflaż (podobieństwo obiektu do tła),
- obiekty przestawione w tle.

Ghost (duchy)

Przykład: samochód zatrzymuje się na parkingu i po pewnym czasie zostaje włączony do tła. Gdy odjedzie, „puste miejsce” bywa wykrywane jako obiekt — to właśnie *ghost* (obiekt obecny w modelu, a nie w bieżącej ramce).

Polityka aktualizacji

Aktualizacja modelu może być:

- **konserwatywna** — uaktualniane są tylko piksele sklasyfikowane jako tło,
- **liberalna** — uaktualniane są wszystkie piksele.

Polityka konserwatywna może „zamrozić” błąd (piksele nigdy nie są poprawiane), a liberalna sprzyja wtapianiu obiektów w tło i powstawaniu *ghostów* lub smug.

Cienie

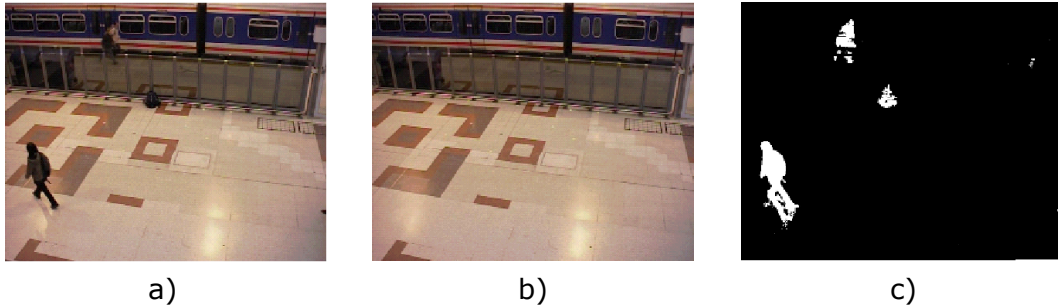
Cień rzucany przez obiekt zwykle spełnia kryteria „pierwszego planu”, ale z punktu widzenia dalszej analizy jest zakłóceniem (zmienia kształt, łączy obiekty). Dlatego cień często traktuje się jako osobną klasę: nie tło i nie obiekt pierwszoplanowy.

Przykład

Na rysunku 3.1 pokazano segmentację z wykorzystaniem modelu tła. Zwróć uwagę na cień pod nogami osoby po lewej oraz osobę za barierką (słabo widoczną w bieżącej ramce).

3.3 Metody oparte o bufor próbek

W ramach ćwiczenia zaprezentowane zostaną dwie metody: średnia z bufora oraz mediana z bufora. Rozmiar bufora przyjmiemy na $N = 60$ próbek. W każdej iteracji algorytmu należy usunąć ostatnią ramkę z bufora, dodać bieżącą (bufor działa na zasadzie kolejki FIFO) oraz obliczyć średnią lub medianę z bufora.



Rysunek 3.1: Przykład segmentacji obiektów pierwszoplanowych z wykorzystaniem modelowania tła. a) bieżąca ramka, b) model tła, c) maska obiektów pierwszoplanowych. Źródło: sekwencja [PETS 2006](#).

1. Na początku należy zadeklarować bufor o rozmiarze $N \times YY \times XX$ (polecenie `np.zeros`), gdzie YY i XX to odpowiednio wysokość i szerokość ramki z sekwencji *pedestrians*.

```
BUF = np.zeros((YY, XX, N), np.uint8)
```

Należy też zainicjalizować licznik bufora (`np.iN`) wartością 0. Parametr `iN` pełni rolę wskaźnika: wskazuje pozycję w buforze, w której należy usunąć najstarszą ramkę i zastąpić ją ramką bieżącą.

2. Obsługa bufora powinna wyglądać następująco: w pętli na pozycji wskazywanej przez `iN` należy zapisać bieżącą ramkę w skali szarości.

```
BUF[:, :, iN] = IG;
```

Następnie licznik `iN` należy inkrementować i sprawdzać, czy nie osiągnął rozmiaru bufora (N). Jeśli tak, to trzeba go ustawić na 0.

3. Obliczenie średniej lub mediany realizuje się za pomocą funkcji `mean` lub `median` z biblioteki *numpy*. Jako parametr podaje się wymiar (oś), dla którego ma być liczona wartość (pamiętać o indeksowaniu od zera). Aby wszystko działało poprawnie, należy dokonać konwersji wyniku na `uint8`.
4. Ostatecznie realizujemy odejmowanie tła, tzn. od bieżącej sceny odejmujemy model, a wynik binaryzujemy – analogicznie jak przy różnicy sąsiednich ramek. Dodatkowo wykorzystujemy filtrację medianową maski obiektów i/lub operacje morfologiczne.
5. Wykorzystując stworzony w ramach poprzedniego ćwiczenia kod, porównaj działanie metody ze średnią i medianą. Zanotuj wartości wskaźnika $F1$ dla obu przypadków.

R Mediana może być obliczana przez dłuższą chwilę.

Zastanów się, dlaczego mediana działa lepiej. Sprawdź działanie na innych sekwencjach – w szczególności dla sekwencji *office*. Zaobserwuj zjawisko smużenia oraz wtapiania się sylwetki w tło, a także powstawania *ghostów*.

Ćwiczenie 3.1 Zaimplementuj algorytm (średniej i mediany) na podstawie powyższego opisu. ■

3.4 Aproksymacja średniej i mediany (tzw. sigma-delta)

Użycie bufora próbek wymaga znacznych zasobów pamięci. Średnią można aktualizować w prosty sposób (warto zastanowić się, jak zaktualizować średnią w buforze bez ponownego przeliczania wszystkich elementów). W przypadku mediany sytuacja jest trudniejsza: nie istnieją równie szybkie algorytmy jej wyznaczania, choć nie trzeba sortować wszystkich elementów w każdej iteracji. Dlatego częściej stosuje się metody, które nie wymagają bufora. W przypadku aproksymacji średniej wykorzystuje się zależność:

$$BG_N = \alpha I_N + (1 - \alpha) BG_{N-1} \quad (3.1)$$

gdzie: I_N – bieżąca ramka, BG_N – model tła, α – parametr wagowy (zwykle 0,01–0,05, choć wartość zależy od konkretnej aplikacji).

Aproksymację mediany uzyskuje się, wykorzystując zależność:

$$BG_N = \begin{cases} BG_{N-1} + 1, & \text{gdy } BG_{N-1} < I_N, \\ BG_{N-1} - 1, & \text{gdy } BG_{N-1} > I_N, \\ BG_{N-1}, & \text{w przeciwnym razie.} \end{cases} \quad (3.2)$$

Ćwiczenie 3.2 Zaimplementuj obie metody.

1. Jako pierwszy model tła przyjmij pierwszą ramkę z sekwencji. Zanotuj wartość wskaźnika $F1$ dla tych dwóch metod (parametr α ustal na 0.01). Zaobserwuj czas działania.
2. Poeksperymentuj z wartością parametru α . Zobacz jaki ona ma wpływ na model tła.

R Dla metody średniej bieżącej kluczowe jest, aby **model tła był przechowywany w formacie zmiennoprzecinkowym** (*float64*).

We wzorze 3.1 uniknięcie stosowania pętli po całym obrazie jest oczywiste. Dla zależności 3.2 również jest to możliwe, ale wymaga chwili zastanowienia. Warto w tym celu sięgać do dokumentacji *numpy*. Proszę też zauważyć, że w Pythonie można wykonywać działania arytmetyczne na wartościach boolowskich.

3.5 Polityka aktualizacji

Jak dotąd stosowaliśmy liberalną politykę aktualizacji – aktualizowaliśmy wszystko. Spróbujemy teraz wykorzystać podejście konserwatywne.

Ćwiczenie 3.3 Polityka aktualizacji.

1. Dla wybranej metody zaimplementuj konserwatywne podejście do aktualizacji. Sprawdź jego działanie.

R Wystarczy zapamiętać poprzednią maskę obiektów i odpowiednio wykorzystać ją w procedurze aktualizacji.

2. Zwróć uwagę na wartość wskaźnika $F1$ oraz na model tła. Czy pojawiły się w nim jakieś błędy ?

3.6 OpenCV – GMM/MOG

W bibliotece OpenCV dostępna jest jedna z najpopularniejszych metod segmentacji obiektów pierwszoplanowych: Gaussian Mixture Models (GMM), nazywana też Mixture of Gaussians (MoG) – obie nazwy występują równolegle w literaturze. W największym skrócie scena jest modelowana za pomocą kilku rozkładów Gaussa (np. średniej jasności/koloru oraz odchylenia standardowego). Każdy rozkład opisany jest również wagą, która odzwierciedla, jak często był on obserwowany na scenie (tj. prawdopodobieństwo wystąpienia). Rozkłady o największych wagach stanowią tło. Segmentacja polega na obliczaniu odległości pomiędzy bieżącą obserwacją piksela a każdym z rozkładów. Jeśli piksel jest podobny do rozkładu uznanego za tło, klasyfikowany jest jako tło; w przeciwnym przypadku zostaje uznany za obiekt. Model tła jest również aktualizowany w sposób zbliżony do równania 3.1. Zainteresowane osoby odsyłamy do literatury, np. do pracy [Stauffera i Grimsona](#).

Ćwiczenie 3.4 Metoda ta jest przykładem algorytmu wielowariantowego – model tła ma kilka możliwych reprezentacji (wariantów).

1. Wykorzystaj klasę `BackgroundSubtractorMOG2`, tworząc obiekt za pomocą `createBackgroundSubtractorMOG2`. W pętli, dla każdego obrazu, wywołuj metodę `apply()`.
2. Poeksperymentuj z parametrami `history` i `varThreshold` oraz wyłącz detekcję cieni. W metodzie `apply()` można również ustalić współczynnik uczenia – `learningRate`.
3. Wyznacz miarę $F1$ (dla metody bez detekcji cieni). Zaobserwuj, jak działa metoda. Zwróć uwagę, że nie da się „wyświetlić” modelu tła.

R W pakiecie OpenCV dostępna jest wersja GMM nieco inna niż oryginalna – między innymi wyposażona w moduł detekcji cieni oraz dynamicznie zmienianą liczbę rozkładów Gaussa.

3.7 OpenCV – KNN

Druga z metod dostępnych w OpenCV to KNN (`BackgroundSubtractorKNN`).

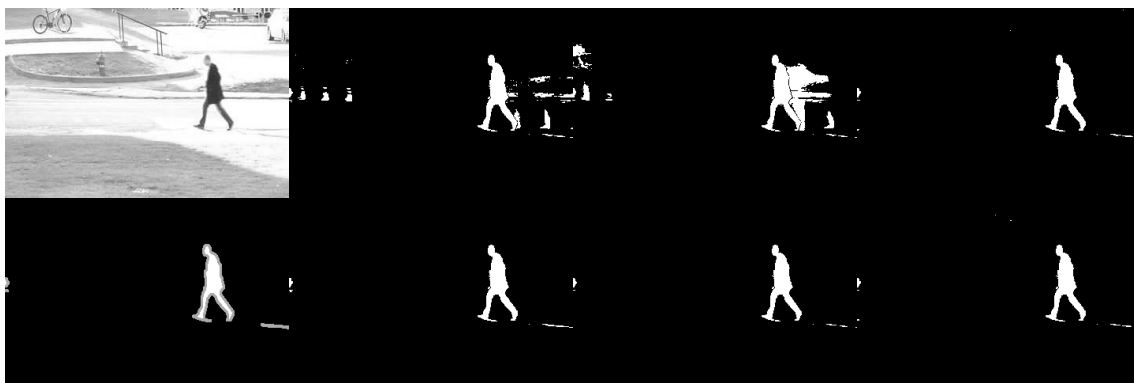
Ćwiczenie 3.5 Proszę uruchomić metodę KNN i dokonać jej ewaluacji.

3.8 Przykładowe rezultaty algorytmu do segmentacji obiektów pierwszoplanowych

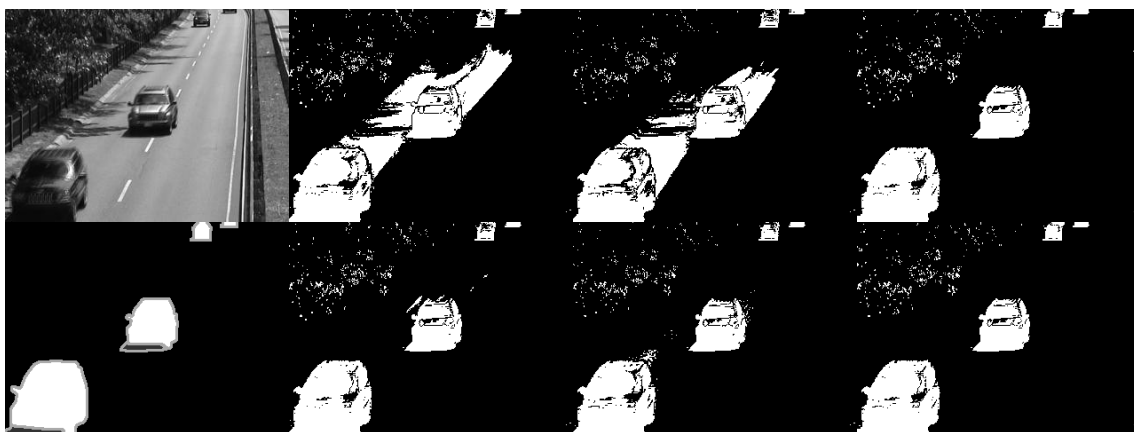
W celu lepszej weryfikacji poszczególnych etapów zaproponowanych metod do segmentacji obiektów pierwszoplanowych przedstawione zostaną przykładowe wyniki dla wybranych ramek obrazu.

Rozliczenie ćwiczenia

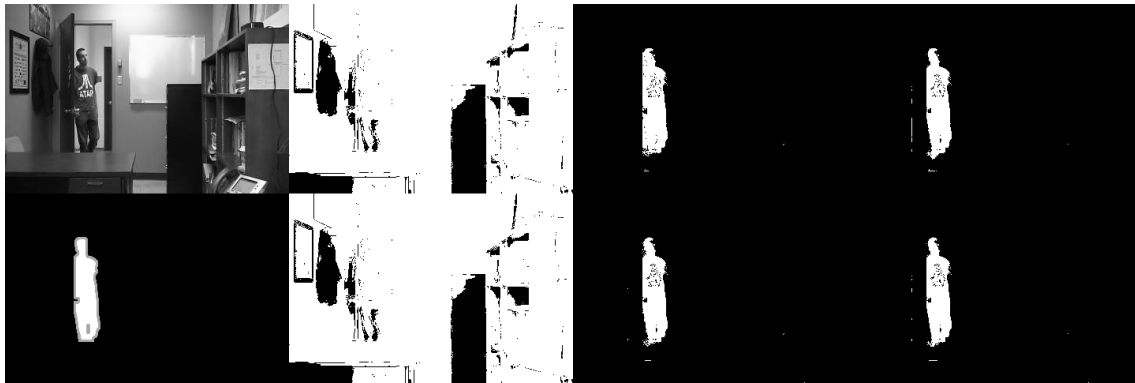
W celu rozliczenia ćwiczenia, po wykonaniu zadania zgłoś swoją gotowość prowadzącemu zajęcia. Równocześnie możesz zgłosić swoją gotowość po wykonaniu wszystkich zadań dotyczących poruszanego tematu zajęć. Po akceptacji z jego strony, umieść skrypt o rozszerzeniu `.py` lub `.ipynb` w odpowiednim miejscu w zasobach kursu na UPeL.



Rysunek 3.2: Przykładowy wynik poszczególnych wersji algorytmu do segmentacji obiektów pierwszoplanowych dla zbioru *pedestrian*. Indeks ramki to 600. Pierwsza od lewej kolumna to obraz wejściowy wraz z obrazem referencyjnym. Pierwszy wiersz dotyczy algorytmów wykorzystujących średnią, drugi wiersz wykorzystuje medianę. Kolejno od drugiej kolumny prezentowany jest wynik algorytmu: liberalna polityka aktualizacji, następnie aproksymacja funkcji z liberalną polityką aktualizacji oraz aproksymacja funkcji z konserwatywną polityką aktualizacji.



Rysunek 3.3: Przykładowy wynik poszczególnych wersji algorytmu do segmentacji obiektów pierwszoplanowych dla zbioru *highway*. Indeks ramki to 1200. Pierwsza od lewej kolumna to obraz wejściowy wraz z obrazem referencyjnym. Pierwszy wiersz dotyczy algorytmów wykorzystujących średnią, drugi wiersz wykorzystuje medianę. Kolejno od drugiej kolumny prezentowany jest wynik algorytmu: liberalna polityka aktualizacji, następnie aproksymacja funkcji z liberalną polityką aktualizacji oraz aproksymacja funkcji z konserwatywną polityką aktualizacji.



Rysunek 3.4: Przykładowy wynik poszczególnych wersji algorytmu do segmentacji obiektów pierwszoplanowych dla zbioru *office*. Indeks ramki to 600. Pierwsza od lewej kolumna to obraz wejściowy wraz z obrazem referencyjnym. Pierwszy wiersz dotyczy algorytmów wykorzystujących średnią, drugi wiersz wykorzystuje medianę. Kolejno od drugiej kolumny prezentowany jest wynik algorytmu: liberalna polityka aktualizacji, następnie aproksymacja funkcji z liberalną polityką aktualizacji oraz aproksymacja funkcji z konserwatywną polityką aktualizacji.

3.9 Wykorzystanie sieci neuronowej do segmentacji obiektów pierwszoplanowych

Do segmentacji obiektów pierwszoplanowych można wykorzystać również metody oparte na architekturze sieci neuronowych. Przykładem takiego rozwiązania jest praca¹. Zaproponowane podejście wykorzystuje konwolucyjną sieć neuronową BSUV-Net (ang. Background Subtraction of Unseen Videos Net). Architektura bazuje na strukturze U-Net z połączeniami resztkowymi (ang. residual connections) i składa się z enkodera oraz dekodera. Wejściem jest konkatencja kilku obrazów, tj. bieżącej ramki oraz dwóch ramek tła w różnych odstępach czasowych, wraz z odpowiadającymi im semantycznymi mapami segmentacji. Wyjściem jest maska zawierająca wyłącznie obiekty pierwszoplanowe.

Szczegóły dotyczące tej sieci neuronowej przedstawiono w artykule: [arXiv:1907.11371](#). Kod wraz z przykładowymi danymi oraz wytrenowanymi modelami udostępniono w serwisie GitHub w repozytorium [BSUV-Net-inference](#).

Ćwiczenie 3.6 Zadanie nieobowiązkowe. Uruchom konwolucyjną sieć neuronową BSUV-Net.

- Ściągnij z platformy UPEL wszystkie pliki do sieci neuronowej.
- Wykorzystana zostanie baza danych `pedestrians`, ale odpowiednio przeskalowana do rozmiaru wejścia sieci oraz przekonwertowana na sekwencje wideo,
- Otwórz plik `infer_config_manualBG.py` oraz zmodyfikuj ścieżki do:
 - w klasie `SemanticSegmentation` podaj ścieżkę absolutną do folderu `segmentation` – zmienna `root_path`,
 - w klasie `BSUVNet` podaj ścieżkę do modelu sieci `BSUV-Net-2.0.mdl` w folderze `trained_models` – zmienna `model_path`,
 - w klasie `BSUVNet` podaj ścieżkę do obrazu, który zawiera jedynie tło – pierwsze zdjęcie ze zbioru `pedestrians` – zmienna `empty_bg_path`
- W pliku `inference.py` podaj ścieżkę do wejścia sieci, czyli sekwencja wideo `pedestrians` – zmienna `inp_path` oraz ścieżkę do miejsca gdzie ma być zapisane wyjście sieci wraz z nazwą pliku – zmienna `out_path`.

¹Tezcan, Ozan and Ishwar, Prakash and Konrad, Janusz; BSUV-Net: A Fully-Convolutional Neural Network for Background Subtraction of Unseen Videos, [arXiv:1907.11371](#)

4

Extra laboratorium 1

4	Uogólniona transformata Hougha	37
4.1	Cel zajęć	
4.2	Implementacja uogólnionej transformaty Hougha	
4.3	Wyszukiwanie wzorców różniących się orientacją	

4. Uogólniona transformata Hougha

4.1 Cel zajęć

- implementacja uogólnionej transformaty Hougha,
- wyszukiwanie wzorców za pomocą uogólnionej transformaty Hougha.

4.1.1 R-table

4.2 Implementacja uogólnionej transformaty Hougha

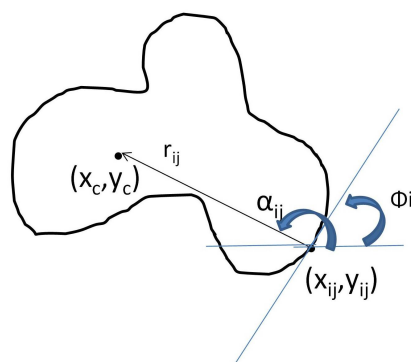
Ćwiczenie 4.1 Wyszukiwanie wzorców.

1. Ze strony kursu pobierz archiwum z danymi do ćwiczenia i rozpakuj je we własnym katalogu roboczym.
2. Utwórz nowy skrypt. Na podstawie obrazu ze wzorcem *trybik.jpg* stworzymy tabelicę *R-table*. W tym celu wyznacz **kontury** oraz **gradienty** na obrazie wzorca.
3. Aby **wyznaczyć kontur** należy najpierw przeprowadzić konwersję obrazu na odcień szarości oraz binaryzację z odpowiednim progiem. Następnie wykorzystaj funkcję `cv2.findContours` do uzyskania konturów występujących na obrazie.

R Zaneguj obraz – `cv2.bitwise_not` – przed przesłaniem go do funkcji, gdyż zawiera on czarny kontur na białym tle, a funkcja `findContours` oczekuje odwrotnego ustawienia

i	ϕ	R_{ϕ_i}
1	0	$(r_{11}, \alpha_{11})(r_{12}, \alpha_{12}) \dots (r_{1n}, \alpha_{1n})$
2	$\Delta\phi$	$(r_{21}, \alpha_{22})(r_{22}, \alpha_{22}) \dots (r_{2m}, \alpha_{2m})$
3	$2\Delta\phi$	$(r_{31}, \alpha_{31})(r_{32}, \alpha_{32}) \dots (r_{3k}, \alpha_{3k})$
...	...	...

Tabela 4.1: Budowa R-table.



Rysunek 4.1: Budowa R-table. Źródło: [Generalised Hough transform - Wikipedia](#).

Zwróć uwagę, aby uzyskać listy wszystkich punktów konturu – zastosuj parametr `CHAIN_APPROX_NONE`.

- R** W przypadku, gdy kontur zostanie podzielony można wykorzystać operacje morfologiczne, aby zapobiec podziałom lub ustawić parametr `RETR_TREE`, który pozwala ułożyć kontury w kolejności od najdłuższego do najkrótszego i wybrać najdłuższy kontur.

4. Wyświetl kontur za pomocą funkcji np. `cv2.drawContours`.
5. Wylicz gradienty, wykorzystując filtry **Sobela**.
 - Oblicz amplitudę gradientu,
 - Oblicz orientację gradientu.

- R** Wartość amplitudy gradientu warto znormalizować przez jej wartość maksymalną.

6. **Wybierz punkt referencyjny** – będzie to środek ciężkości wzorca wyznaczany ze zbinaryzowanego obrazu wzorca z wykorzystaniem momentów. Do wyznaczenia środka ciężkości można wykorzystać momenty centralne $m00$, $m10$ i $m01$ (funkcja `cv2.moments(bin, 1)`).
7. Do wypełnienia **R-table** będą potrzebne wektory łączące punkty konturu/konturów z punktem referencyjnym. Do **R-table** wpisujemy **długości tych wektorów** oraz **kąty** jakie tworzą z osią OX. Miejsce wpisania do tablicy **R-table** wyznacza orientacja gradientu w punkcie konturu (wyznaczona na podstawie filtracji maskami Sobela), **przy czym proszę przeliczyć radiany na stopnie** – **R-table** będzie miała 360 wierszy. Stosujemy dokładność wynoszącą 1 stopień. Wówczas np. `Rtable[30]` będzie listą współrzędnych biegunowych punktów konturu, których orientacja wyliczona na podstawie gradientów wynosi około 30° .
8. Na podstawie obrazu `trybiki2.jpg` oraz **R-table** z poprzedniego punktu wypełnij dwuwymiarową przestrzeń Hougha – wylicz ponownie gradient w każdym punkcie i dla punktów, których znormalizowana wartość gradientu przekracza 0.5, określ orientację w tym punkcie, a następnie zwiększ wartość o 1 w przestrzeni akumulacyjnej w punktach zapisanych w **R-table** według zależności:

```
x1 = -r*np.cos(fi) + x
y1 = -r*np.sin(fi) + y
```

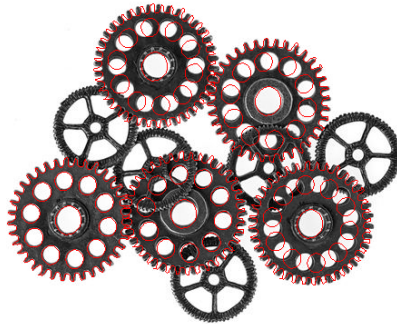
gdzie r, fi – wartości z odpowiedniego wiersza w $R-table$, x, y – współrzędne punktu, dla którego gradient przekracza 0.5.

Warto też sobie wyświetlić postać przestrzeni Hougha.

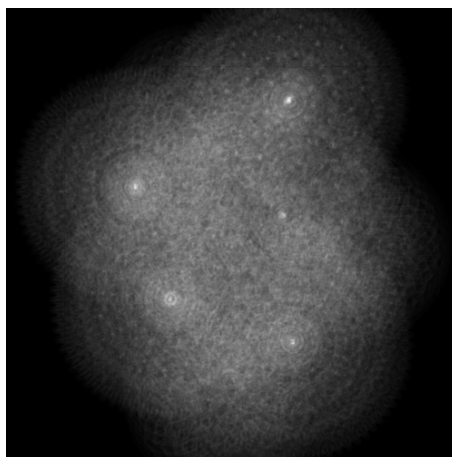
9. Wyszukaj maksimum w przestrzeni Hougha i zaznacz je na obrazie wejściowym – trybiki2.jpg.
10. Wynikiem działania powyższego algorytmu jest zaznaczenie znalezionego maksimum na obrazie oraz nałożenie konturu wzorca wokół tego punktu. Wyznacz wszystkie pięć maksimumów na obrazie.

R Aby bez problemu znaleźć kolejne maksima, warto wcześniej wyzerować pewien obszar wokół już znalezionego maksimum.

11. Przykładowe rozwiązanie zostało przedstawione na rys. 4.2, dodatkowo na rys. 4.3 zilustrowano przestrzeń Hougha.



Rysunek 4.2: Przykładowe rozwiązanie.



Rysunek 4.3: Przestrzeń Hougha.

4.3 Wyszukiwanie wzorców różniących się orientacją

Ćwiczenie 4.2 Poniższe zadanie prezentuje metodę wykorzystującą przestrzeń Hougha do wyszukiwania wzorców różniących się orientacją.

1. Bazując na algorytmie do wyszukiwania wzorca z zadania podstawowego, zostanie wprowadzona dodatkowa funkcjonalność, która wpłynie na interpretację wzorców różniących się orientacją.
2. W pierwszym kroku zwiększamy przestrzeń Hougha do 3 wymiarów, dodając obroty. Zakładamy początkową dyskretyzację kąta obrotu 10 stopni. Można do tego wykorzystać następującą składnię:

```
new_hough_shape = hough.shape + (36,)
new_hough = np.zeros(new_hough_shape)
```

gdzie: (36,) jest tu jednoelementową krotką – reprezentuje ona trzeci wymiar – krok kąta co 10 stopni.

3. Uwzględnianie obrotów odbywa się w dwóch krokach. W podstawowej wersji obliczaliśmy orientację w rozważanym punkcie i na tej podstawie odwoływaliśmy się do *R-table*. Teraz w danym punkcie będziemy rozważać 36 orientacji – bazową oraz przesuniętą co 10 stopni. Należy zatem przekształcić kod z punktu 4.2.8 tak, aby inkrementacja była realizowana dla co dziesiątego kąta z zakresu [0 – 360). W praktyce kąty odejmujemy od wartości zdyskretyzowanej orientacji w punkcie *alpha* i wyznaczamy moduł tj.

```
alpha_n = |alpha - d_alpha|
```

gdzie: *d_alpha* to kolejne wartości 0, 10, 20, 30, ..., 350).

4. W drugim kroku należy “skompensować” dokonaną zmianę orientacji. W tym celu odczytujemy odpowiednią wartość kąta *fi* dla wzorca z *R-table* i przy wyznaczaniu punktów akumulacji stosujemy zmodyfikowane wzory z punktu 4.2.8:

```
x1 = -r*np.cos(fi + df) + x
y1 = -r*np.sin(fi + df) + y
```

gdzie: *df* to kolejne kąty obrotu *d_alpha* (ale wyrażone w radianach).

5. W celu wykrycia wszystkich wzorców na obrazie należałoby wykryć odpowiednio duże maksima lokalne. Jednakże dla ułatwienia zastąpimy tę operację kilkukrotnym wykrywaniem maksimum globalnego i wyzerowaniem jego otoczenia. Do znalezienia maksimum w 3-wymiarowej przestrzeni Hougha można wykorzystać metodę tablicy *numpy* – *argmax()*. Jednakże zwraca ona pojedynczy indeks w tablicy “spłaszczonej” do jednego wymiaru. Do uzyskania użytecznych dla nas indeksów (po 3 wymiarach) należy wynik *argmax()* podać do funkcji *np.unravel_index* (wraz z *shape* tablicy). Po znalezieniu maksimum zaznacz je na obrazie wejściowym (potrzebne są tylko 2 współrzędne maksimum, kąt na razie nie jest istotny).
6. Jeżeli znalezione zostało to samo maksimum, wyzeruj otoczenie maksimum w przestrzeni Hougha – przykładowo:

```
hough[m[0]-delta:m[0]+delta, m[1]-delta:m[1]+delta, :] = 0
```

gdzie: *m* zawiera indeksy maksimum, a *delta* to rozmiar otoczenia (można przyjąć wartość 30). Następnie ponów operacje z poprzedniego punktu. Wyszukaj

i zaznacz w sumie 5 kolejnych maksimum. Sprawdź czy znalezione zostały odpowiednie obiekty. W przypadku problemów proszę ew. rozważyć zwiększenie rozdzielczości dyskretyzacji kątów np. z 36 do 72.

7. Dla lepszej wizualizacji można dodatkowo wyrysować wzorzec w znalezionych punktach. W tym celu należy wyrysować na obrazie wejściowym punkty odpowiadające wszystkim wektorom z tablicy *R-table* używając wzorów bardzo podobnych do tych z punktu 4.2.8, przy czym x, y to współrzędne maksimum znalezionego maksimum, a kąt df wynika z 3 współrzędnej maksimum.

5

Mini projekt 1

5	Projekt 1	45
5.1	Cel projektu	
5.2	Przykładowa literatura	

5. Projekt 1

Temat 1: Wykrywanie wad włosia szczoteczki

Wykrywanie wad włosia szczoteczki (ang. *toothbrush bristle defect detection*) realizuje się głównie za pomocą systemów wizyjnych i uczenia maszynowego. Dane do tego typu zadań obejmują obrazy wysokiej rozdzielczości oraz zdjęcia z mikroskopów elektronowych (SEM), co pozwala na analizę jakości końcówek włosia.

W systemach produkcyjnych (ang. *quality control*) najczęściej analizuje się:

- **Włosie rozszczone/rozłożone** (ang. *bristle splaying*) – deformacja włosia.
- **Ścieranie** (ang. *abrasion*) – zniszczenie końcówek włosia.
- **Błędy w stapianiu/mocowaniu** (ang. *bristle stapling defects*) – nieprawidłowe osadzenie pęczków.
- **Niewłaściwe zaokrąglenie końcówek** (ang. *irregular bristle end rounding*).
- **Braki we włosiu** (ang. *hair loss/missing bristles*).

Temat 2: Wykrywanie wad w kablu elektrycznym

Wykrywanie wad w kablu elektrycznym (ang. *electric cable defect detection*) polega na identyfikacji defektów izolacji oraz nieprawidłowości budowy wewnętrznej kabla na podstawie obrazów. W przedstawionym wariancie dane mogą stanowić zdjęcia **przekrojów poprzecznych** (kabel przecięty i sfotografowany), co pozwala ocenić położenie żył względem siebie, poprawność ułożenia oraz stan przewodników i izolacji.

W systemach produkcyjnych (ang. *quality control*) oraz w analizie przekrojów najczęściej wykrywa się:

- **Brak jednej żyły/przewodu** (ang. *missing conductor/core*) – niekompletna wiązka w kablu wielożyłowym.
- **Zagięty/nagięty przewód** (ang. *bent conductor*) – nienaturalne wygięcie żyły widoczne w przekroju.
- **Przemieszczenie żyły, mimośrodowość** (ang. *conductor displacement, eccentricity*) – żyła nie jest osiowo ułożona w izolacji.
- **Uszkodzenia żyły** (ang. *strand breakage / conductor damage*) – przerwane druciki, deformacje przekroju.

- **Zwarcie/mostek materiałowy między żyłami** (ang. *short / bridging*) – kontakt przewodników lub brak separacji.
- **Pęcherze, puste przestrzenie** (ang. *voids and bubbles*) – ubytki w izolacji lub wypełnieniu.
- **Nierównomierna grubość izolacji** (ang. *insulation thickness variation*) – lokalne osłabienie dielektryczne.
- **Nacięcia i pęknięcia izolacji** (ang. *cuts and cracks*) – rozdarcia powłoki widoczne na przekroju.

Temat 3: Wykrywanie wad tranzystora

Wykrywanie wad tranzystora (ang. *transistor defect detection*) dotyczy kontroli jakości elementów półprzewodnikowych na podstawie obrazów (np. zdjęć obudowy, wyprowadzeń oraz – w zależności od zadania – mikrofotografii struktury krzemowej). Celem jest wykrycie defektów mechanicznych i montażowych, które mogą prowadzić do awarii lub pogorszenia parametrów.

W systemach produkcyjnych (ang. *quality control*) najczęściej analizuje się:

- **Zagięte/skręcone wyprowadzenia** (ang. *bent/twisted leads*) – utrudniony montaż i ryzyko zwarc.
- **Brakujące lub uszkodzone wyprowadzenie** (ang. *missing/broken lead*).
- **Utlenienie/korozja** (ang. *oxidation/corrosion*) na wyprowadzeniach.
- **Pęknięcia i wyszczerbienia obudowy** (ang. *package cracks/chips*) – ryzyko nieuszczelnności i uszkodzeń mechanicznych.
- **Wady znakowania** (ang. *marking defects*) – nieczytelny nadruk, przesunięcie lub brak oznaczeń.
- **Zanieczyszczenia i pozostałości procesu** (ang. *contamination/residue*) – pył, przebarwienia, pozostałości topnika.
- **Odchyłki wymiarowe/geometrii** (ang. *dimensional/geometric defects*) – np. nieprawidłowy rozstaw pinów.

5.1 Cel projektu

Celem projektu jest opracowanie systemu wizyjnego umożliwiającego automatyczne wykrywanie wad wybranych obiektów. W ramach projektu należy:

- przygotować dane (obrazy) oraz zdefiniować klasy wad,
- zaproponować i zaimplementować etap wstępnego przetwarzania (np. normalizacja, odsumianie, poprawa kontrastu),
- wyznaczyć obszary istotne do analizy (np. segmentacja końcówek włosia) lub cechy opisujące defekty,
- dobrać i wytrenować model klasyfikacji/wykrywania (lub zbudować reguły detektor bazujący na cechach),
- ocenić jakość rozwiązania (np. accuracy, precision/recall, macierz pomyłek) oraz omówić ograniczenia.

Zbiór danych dostępny jest pod adresem: <https://drive.google.com/drive/folders/1nXdqY60uZIEWWwLzuxtZrkXygI2xNbS6?usp=sharing>.

5.2 Przykładowa literatura

- Bao, Nengsheng, et al. *A deep learning-based process monitoring system for tooth-brush manufacturing defect characterization. Procedia CIRP* **118** (2023): 1072–1077.

-
- Bergmann, Paul, et al. *MVTec AD–A comprehensive real-world dataset for unsupervised anomaly detection*. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2019).

6

Laboratorium 6

6	Punkty Charakterystyczne	53
6.1	Cel zajęć	
6.2	Detekcja narożników metodą Harrisa – teoria	
6.3	Implementacja metody Harrisa	
6.4	Prosta deskrypcja punktów charakterystycznych	
6.5	ORB (FAST+BRIEF)	
6.6	Wykorzystanie punktów charakterystycznych do łączenia obrazów – panorama	

6. Punkty Charakterystyczne

6.1 Cel zajęć

- zapoznanie z zagadnieniem wykrywania punktów charakterystycznych,
- implementacja prostego algorytmu detekcji – metoda Harrisa,
- zapoznanie z zagadnieniem opisywania punktów charakterystycznych,
- otoczenie punktu jako deskryptor,
- porównanie działania metody Harrisa, SIFT i ORB (FAST+BRIFT).

6.2 Detekcja narożników metodą Harrisa – teoria

Detekcja narożników metodą Harrisa polega na wyszukiwaniu pikseli dla których moduł gradientu pionowego i poziomego ma znaczną wartość.

Metoda opisana jest za pomocą następujących zależności:

$$H(x, y) = \det(M(x, y)) - k \operatorname{trace}^2(M(x, y)) \quad (6.1)$$

gdzie:

- $\det(\cdot)$ – wyznacznik macierzy,
- $\operatorname{trace}(\cdot)$ – ślad macierzy,
- k – stała empiryczna (zwykle w zakresie ok. 0.04–0.06),
- (x, y) – współrzędne piksela w obrazie.

Macierz autokorelacji M to:

$$M(x, y) = \begin{bmatrix} \langle I_x^2(x, y) \rangle & \langle I_{xy}(x, y) \rangle \\ \langle I_{xy}(x, y) \rangle & \langle I_y^2(x, y) \rangle \end{bmatrix} \quad (6.2)$$

gdzie poszczególne symbole oznaczają:

$$\langle I_x^2(x, y) \rangle = I_x^2(x, y) \otimes h(x, y), \quad (6.3)$$

$$\langle I_y^2(x, y) \rangle = I_y^2(x, y) \otimes h(x, y), \quad (6.4)$$

$$\langle I_{xy}(x, y) \rangle = (I_x I_y)(x, y) \otimes h(x, y) \quad (6.5)$$

gdzie:

- $I_x(x, y)$ oraz $I_y(x, y)$ są pochodnymi cząstkowymi obrazu w kierunku osi x i y (np. wyznaczonymi operatorem Sobela),
- $I_{xy}(x, y) = I_x(x, y) I_y(x, y)$,
- $\otimes$ oznacza spłot (konwolucję),
- $h(x, y)$ jest jądrem (maską) filtru Gaussa, używanym do lokalnego uśredniania w otoczeniu piksela.

$$h(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (6.6)$$

gdzie:

- σ – odchylenie standardowe.

Filtracja Gaussa redukuje wpływ szumu, który szczególnie silnie zaburza obliczanie gradientów.

Wyznacznik i ślad macierzy 2×2 (dla przypomnienia):

$$\det(M(x, y)) = \langle I_x^2(x, y) \rangle \langle I_y^2(x, y) \rangle - \langle I_{xy}(x, y) \rangle^2, \quad (6.7)$$

$$\text{trace}(M(x, y)) = \langle I_x^2(x, y) \rangle + \langle I_y^2(x, y) \rangle \quad (6.8)$$

Ostatecznie dany piksel jest klasyfikowany jako narożnik jeśli wyliczona wartość $H(x, y)$ jest większa niż określony próg.

Jako końcową filtrację warto wykorzystać wyszukiwanie lokalnych maksimów. Za kandydata na narożnik uznajemy tylko te piksele, które są lokalnymi maksimami w swoim otoczeniu (np. 7×7).

6.3 Implementacja metody Harrisa

Ćwiczenie 6.1 Na podstawie powyższej teorii proszę zaimplementować metodę Harrisa.

1. Ze strony kursu pobierz archiwum z danymi do ćwiczenia i rozpakuj je we własnym katalogu roboczym.
2. **Wczytaj obrazy** *fontanna1.jpg* oraz *fontanna2.jpg*.
3. **Zaimplementuj funkcję** wyliczającą wartość H z macierzy autokorelacji. Funkcja jako parametry powinna otrzymywać obraz w odcieniach szarości oraz rozmiar masek filtrów Sobela i Gaussa.
 - Wylicz odpowiednie iloczyny pochodnych kierunkowych i rozmyj je za pomocą filtru Gaussa - zgodnie ze wzorami podanymi wyżej.
 - Przyjmij, że rozmiary masek Sobela i Gaussa mogą być takie same.
 - Przyjmij wartość współczynnika $K = 0.05$.
 - Znormalizuj obraz wynikowy H do zakresu 0-1.
4. **Wykorzystaj** poniższą funkcję znajdującą maksima lokalne w otrzymanej jako parametr tablicy:

```
import scipy.ndimage.filters as filters
```

```
def find_max(image, size, threshold) : # size - maximum filter mask size
    data_max = filters.maximum_filter(image, size)
    maxima = (image == data_max)
    diff = image > threshold
    maxima[diff == 0] = 0
    return np.nonzero(maxima)
```

5. Dla obu wczytanych obrazów zastosuj powyższe funkcje.
 - Rozmiary masek w obu funkcjach możesz przyjąć 7.
 - Wyniki z drugiej funkcji wyświetl jako znaki (np. *) naniesione na obrazy początkowe.

 Warto stworzyć osobną funkcję rysującą.

6. **Wyświetl i sprawdź** czy wykrywane punkty na obu obrazach znajdują się (mniej więcej) w tych samych położeniach (czyli czy detektor jest powtarzalny).
7. **Powtórz** operacje z powyższych punktów dla obrazów *budynek1.jpg* i *budynek2.jpg*.



6.4 Prosta deskrypcja punktów charakterystycznych

Ćwiczenie 6.2 Niniejsze ćwiczenie jest kontynuacją poprzedniego zadania. Po detekcji punktów charakterystycznych wymagane jest jeszcze ich opisanie (deskrypcję).

1. Do funkcji z poprzedniego zadania **dodaj funkcję** tworzącą opisy punktów charakterystycznych. Opis to wycinek obrazu (patch) z otoczenia punktu o określonym rozmiarze.
 - argumenty wejściowe: obraz, lista współrzędnych punktów (wynik poprzedniej funkcji), rozmiar otoczenia,
 - usuń punkty, dla których otoczenie nie mieści się w obrazie,
 - użyj `filter` z funkcją `lambda` (X,Y – rozmiar obrazu, *size* – rozmiar otoczenia/wycinka).
 - wyjście: lista opisów punktów wraz z ich współrzędnymi.

```
pts = list(filter(lambda pt: pt[0] >= size and pt[0] < Y-size and pt[1] >= size and pt[1] < X-size, zip(pts[0],pts[1])))
```

2. **Dodaj funkcję** dopasowującą opisy punktów charakterystycznych z dwóch obrazów:
 - **Wejście:** dwie listy deskryptorów (z 1) oraz liczba N.
 - **Metoda:** brute force — dla każdego deskryptora z pierwszej listy znajdź najbardziej podobny z drugiej.
 - **Miara podobieństwa:** odległość wektorów, suma wartości bezwzględnych ich różnic lub iloczyn skalarny.
 - **Wynik:** lista (współrzędne pary punktów + wartość miary); zwróć N najlepszych dopasowań.

 Pamiętaj o prawidłowym posortowaniu przed wybraniem najlepszych dopasowań.

3. **Wczytaj obrazy** *fontanna1.jpg* i *fontanna2.jpg*. Znajdź dla nich punkty charakterystyczne metodą Harris'a za pomocą funkcji z poprzedniego zadania. Dla zna-

lezionej punktów utwórz listy opisów za pomocą funkcji z punktu 1. Rozmiar otoczenia ustaw 15 (możesz poeksperymentować z innymi ustawieniami). Wyszukaj najlepsze dopasowania za pomocą funkcji z punktu 2. Parametr n można ustawić na 20.

4. **Wyświetl rezultaty.** Wśród danych do ćwiczenia znajduje się plik *pm.py*. Można zaimportować go do swojego pliku (*import pm*) i wykorzystać znajdującą się w nim funkcję *pm.plot_matches*. Jednakże może być konieczne dostosowanie tej funkcji do listy zwróconej przez funkcję z punktu 2 (dostarczona implementacja zakłada, że każda para jest zapisana w postaci $([y1, x1], [y2, x2])$, obrazy powinny być w odcieniach szarości). Można również zaimplementować własną funkcję do wizualizacji dopasowań.
5. **Powtórz operacje** z poprzednich punktów dla obrazów *budynek1.jpg* i *budynek2.jpg*. Zauważ, że na tych obrazach budynki są położone pod nieco innym kątem. Czy wpłynęło to na jakość dopasowania?
6. **Powtórz operacje** z poprzednich punktów dla obrazów *fontanna1.jpg* i *fontanna_pow.jpg*. Obrazy znacznie różnią się skalą. Czy Harris poradził sobie z taką różnicą?
7. **Powtórz operacje** z poprzednich punktów dla obrazów *eiffel1.jpg* i *eiffel2.jpg*. Obrazy różnią się jasnością i nieco skalą. Czy Harris poradził sobie z taką różnicą?
8. **Zmodyfikuj funkcję** wyznaczającą opisy punktów (z punktu 1), tak aby uwzględnić afiniczne zmiany jasności:

$$w_{aff} = \frac{w - \bar{w}}{|w - \bar{w}|} \quad (6.9)$$

Przy czym średnią $\bar{w}$ można wyznaczyć jako `np.mean(w)`, a $|w - \bar{w}|$ jako odchylenie standardowe za pomocą `np.std(w)`.

Spróbuj teraz powtórzyć operacje dla wymienionych powyżej obrazów. Czy nastąpiła jakaś poprawa?

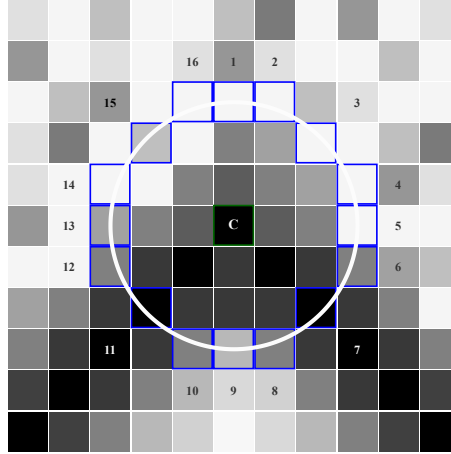


6.5 ORB (FAST+BRIEF)

Obecnie do detekcji i deskrypcji punktów charakterystycznych najczęściej wykorzystuje się trzy algorytmy oraz ich modyfikacje, tj. SIFT, SURF i ORB. Metoda SIFT (ang. *Scale-Invariant Feature Transform*) charakteryzuje się wysoką skutecznością i dokładnością, ale wymaga dużych nakładów obliczeniowych. Metoda SURF (ang. *Speeded Up Robust Features*) cechuje się mniejszą złożonością obliczeniową, jednak w wybranych przypadkach jakość uzyskiwanych rezultatów jest zauważalnie gorsza niż dla SIFT. Algorytm ORB (ang. *Oriented FAST and Rotated BRIEF*) stanowi wydajną alternatywę dla SIFT i SURF.

W celu wykrycia punktów charakterystycznych wykorzystywany jest detektor FAST (ang. *Features from Accelerated Segment Test*). Został on zaprojektowany w celu znajdowania narożników, podobnie jak detektory Moraveca i Harrisa. Realizacja tego zadania odbywa się poprzez porównywanie jasności I_c danego punktu obrazu z 16 pikselami położonymi na otaczającym go okręgu. Zostało to przedstawione na rysunku 6.1.

Punkt centralny jest w tym przypadku uznawany za narożnik, jeżeli n kolejnych pikseli na okręgu ma jasność równocześnie niższą niż $I_c - t$ bądź równocześnie wyższą od $I_c + t$, gdzie t określa pewien próg — parametr algorytmu. Najlepsze jakościowo rezultaty można uzyskać poprzez przyjęcie $n = 9$. Pewien kłopot może natomiast stanowić ryzyko niepożądanego wykrycia krawędzi, nieodłącznie związane z detektorami narożników. Wo-



Rysunek 6.1: Położenie pikseli, których jasność porównywana jest z punktem centralnym c w detektorze FAST.

Wobec tego należy uszeregować wszystkie otrzymane punkty względem miary Harrisa (6.1), a następnie wybrać N najlepszych rezultatów (n dotyczy testu FAST, natomiast N liczby pozostawionych punktów). Wszystkie wskazane w ten sposób narożniki nie są niestety niezmiennie względem obrotu obrazu oraz jego skali. W celu rozwiązania problemu obrotu wykorzystano ideę *centroidu jasności* (ang. *intensity centroid*). Bazuje ona na wyznaczeniu geometrycznych momentów wycinka obrazu wokół danego punktu charakterystycznego, zdefiniowanych poprzez równanie:

$$m_{pq} = \sum_{(x,y) \in \Omega} x^p y^q I(x,y) \quad (6.10)$$

gdzie: x, y – lokalne współrzędne względem wykrytego punktu charakterystycznego, $I(x, y)$ – jasność pikselu w punkcie (x, y) , Ω – zbiór pikseli wycinka.

Sumowanie przebiega przy tym po wszystkich pikselach znajdujących się w obrębie koła o promieniu r i środku w danym punkcie charakterystycznym. Na tej podstawie można określić współrzędne centroidu C :

$$C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right) \quad (6.11)$$

Orientację danego punktu charakterystycznego wyznacza się za pomocą:

$$\theta = \text{atan2}(m_{01}, m_{10}) \quad (6.12)$$

W celu uodpornienia algorytmu ORB od skali należy zastosować piramidę obrazów.

Po uzyskaniu zbioru punktów charakterystycznych następnym etapem jest ich lokalna deskrypcja. Do tego celu wykorzystywany jest algorytm BRIEF (ang. *Binary Robust Independent Elementary Features*). Polega on na konstruowaniu n -elementowych łańcuchów bitowych, zgodnie z zależnością:

$$f_n(p) := \sum_{i=1}^n 2^{i-1} \tau(\mathbf{p}; u_i, v_i) \quad (6.13)$$

przy czym i -ty binarny test τ jest definiowany poprzez równanie:

$$\tau(\mathbf{p}; u_i, v_i) := \begin{cases} 1 & \text{gdy } I(u_i) < I(v_i), \\ 0 & \text{w przeciwnym razie.} \end{cases}$$

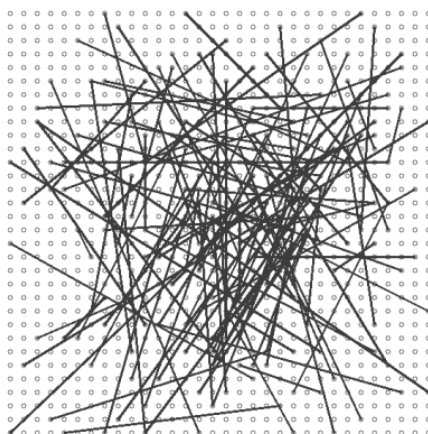
gdzie:

p – wycinek obrazu wokół danego punktu charakterystycznego,

u_i, v_i – pewne dwa piksele wewnątrz $\mathbf{p}$ ($u_i \neq v_i$),

$I(w)$ – jasność obrazu w punkcie $w = (x, y)$.

Wycinek $\mathbf{p}$ powinien zostać przy tym rozmyty w celu niwelacji wpływu zakłóceń. Rozmycie wykonuje się poprzez obrazy całkowite, w których dany punkt charakterystyczny reprezentuje się poprzez sumaryczną jasność otaczających go pikseli (przyjęto do tego kwadratowe sąsiedztwo 5×5 przy rozmiarze wycinka 31×31). Fundamentalne znaczenie dla działania całego algorytmu ma również odpowiedni wybór par pikseli do poszczególnych testów τ . Lokalizacja testowych par jest zdefiniowana losowo (rys. 6.2) lub deterministycznie na podstawie specjalnie opracowanego algorytmu uczącego zaproponowanego przez autorów algorytmu ORB¹. Wyjściem deskryptora jest 256-elementowy wektor binarny. Do wyznaczenia podobieństwa pomiędzy dwoma wektorami binarnymi stosuje się metrykę Hamminga.



Rysunek 6.2: Wygenerowane losowo pary punktów dla deskryptora BRIEF.

Ćwiczenie 6.3 Ćwiczenie ma na celu poznanie oraz implementację elementów algorytmu ORB w uproszczonej wersji. Pomocna będzie opisana teoria powyżej. Wyniki należy porównać z rezultatami z poprzedniego zadania.

1. w zadaniu zastosuj parę obrazków używanych w poprzednim zadaniu, np. *fontanna1.jpg* oraz *fontanna2.jpg*.
2. **Zaimplementuj detektor FAST** w celu wykrycia punktów charakterystycznych oraz wyznacz miarę Harrisa dla każdego punktu. Detektor FAST wyznacza punkt charakterystyczny na podstawie wycinka obrazu o rozmiarze 7×7 px.
3. Następnie za pomocą metody *non-maximum suppression* wykonujemy pierwszą filtrację. Polega ona na usunięciu punktów charakterystycznych, które znajdują się blisko siebie. Sprawdzamy czy w otoczeniu 3×3 px znajduje się wiele wykrytych

¹Rublee, Ethan, et al. "ORB: An efficient alternative to SIFT or SURF." 2011 International conference on computer vision. IEEE, 2011.

punktów charakterystycznych, jeśli tak to zostaw ten z największą miarą.

4. Kolejnym krokiem to usunięcie punktów charakterystycznych, które nie mają pełnego otoczenia 31×31 wykorzystywanego przez deskryptor.
5. Posortuj otrzymane punkty według miary Harrisa. Wybierz N najlepszych punktów.
6. Wyznacz współrzędne centroidu C oraz orientację punktu charakterystycznego na podstawie idei *centroidu jasności* dla każdego punktu.
7. **Zaimplementuj deskryptor BRIEF.** Wycinek o rozmiarze 31×31 należy najpierw rozmyć gaussem 5×5 . Pary punktów można wyznaczyć losowo z odpowiedniego przedziału lub wczytać gotowe pary punktów z pliku `orb_descriptor_positions.txt`. Jednak są to punkty zdefiniowane dla orientacji 0° . w zależności od kąta α uzyskanego na podstawie *centroidu jasności* należy obrócić pary punktów, zgodnie z równaniem:

$$x' = \cos \alpha * x - \sin \alpha * y \quad (6.14)$$

$$y' = \sin \alpha * y + \cos \alpha * x \quad (6.15)$$

Wykonujemy binarny test dla wszystkich par punktów, otrzymując 256-bitowy wektor.

8. Aby porównać punkty charakterystyczne z dwóch obrazów, należy wykorzystać metrykę Hamminga. Może być tutaj zastosowana metoda brute force – wyznaczamy podobieństwo dla każdej pary punktów z dwóch obrazów.
9. Następnie sortujemy według najlepszego podobieństwa i wybieramy N najlepszych punktów.
10. **Wyświetl wyniki** oraz **porównaj** z rezultatami z poprzedniego zadania. Zauważ, że algorytm ORB w porównaniu z poprzednim zadaniem powinien radzić sobie z obrotem.

■

6.6 Wykorzystanie punktów charakterystycznych do łączenia obrazów – panorama

Ćwiczenie 6.4 W zadaniu zostaną wykorzystane gotowe funkcje do detekcji i deskrypcji punktów charakterystycznych z biblioteki OpenCV.

1. Wczytaj obrazy: `left_panorama.jpg` i `right_panorama.jpg`. Przekonwertuj obrazy do skali szarości.
2. Znajdź punkty charakterystyczne na obrazach. Do tego można użyć deskryptora: `cv2.SIFT_create()` lub `cv2.xfeatures2d.SURF_create()`.
3. Wyświetl znalezione punkty za pomocą funkcji: `cv2.drawKeypoints`.
4. Następnie dopasowujemy punkty charakterystyczne z obu obrazów. Do tego służy klasa: `cv2.BFMatcher(NORMA)`. Dla SIFT i SURF norma będzie `cv2.NORM_L2`.
5. Funkcji dopasowujących jest dwie: Brute Force (BF) lub KNN. Jeśli wybierzemy BF to wywołujemy metodę `.match()`, jeśli KNN to metodę `.knnMatch()`.
6. Jeśli BF to otrzymane połączenia sortujemy według dystansu (im mniejszy dystans pomiędzy wektorami deskrypcji tym lepsze podobieństwo) i wybieramy N najlepszych punktów. Jeśli KNN to używamy poniższego kodu:

```
best_matches = [ ]
for m,n in matches: # m i~n - best and second best matches
    if m.distance < 0.5*n.distance: # the best is twice as good as the
        second
```

```
best_matches.append([m])
```

albo to samo w wersji z list comprehension:

```
best_matches = [ [m] for m,n in matches if m.distance < 0.5*n.distance]
```

7. Dla sprawdzenia można wyświetlić otrzymane połączenia – `cv2.drawMatches`.
8. Następnym krokiem jest określenie rotacji i translacji obrazów (punktów) względem siebie, czyli wyznaczamy macierz homografii – funkcja `cv2.findHomography`. Przed podaniem do tej funkcji punktów charakterystycznych należy odpowiednio przekonwertować je. Konwertujemy zbiór punktów do tablicy typu numpy:

```
keypointsL = np.float32([kp.pt for kp in keypointsL])
keypointsR = np.float32([kp.pt for kp in keypointsR])
```

a następnie tworzymy zbiory odpowiednich par już ze sobą dopasowanych:

```
ptsA = np.float32([keypointsL[m.queryIdx] for m in matches])
ptsB = np.float32([keypointsR[m.trainIdx] for m in matches])
```

9. Ostatnim krokiem jest wywołanie funkcji:

```
result = cv2.warpPerspective(image_left, H, (width, height))
```

gdzie: (width, height) to suma wymiarów obu przetwarzanych obrazów oraz połączenie z drugim obrazem:

```
result[0:image_right.shape[0], 0:image_right.shape[1]] = image_right
```

10. Wyświetl wyniki.
11. Dodatkowo można wykryć rzeczywisty wymiar połączonych obrazów i usunąć nadmiar czarnego tła.



7

Laboratorium 7

7	Przepływ optyczny	63
7.1	Cel zajęć	
7.2	Przepływ optyczny (ang. <i>optical flow</i>)	
7.3	Implementacja metody blokowej	
7.4	Implementacja wyliczania w wielu skalach	
7.5	Inne sposoby wyznaczania przepływu optycznego	
7.6	Detekcja kierunku ruchu i klasyfikacja obiektów z wykorzystaniem przepływu optycznego	

7. Przepływ optyczny

7.1 Cel zajęć

- zapoznanie z zagadnieniem przepływu optycznego,
- poznanie zakresu stosowalności przepływu optycznego i jego ograniczeń,
- poznanie podejścia wieloskalowego w wyznaczaniu przepływu optycznego,
- implementacja metody blokowej,
- zapoznanie z metodami dostępnymi w bibliotece OpenCV,
- wykorzystanie przepływu optycznego do segmentacji obiektów ruchomych.

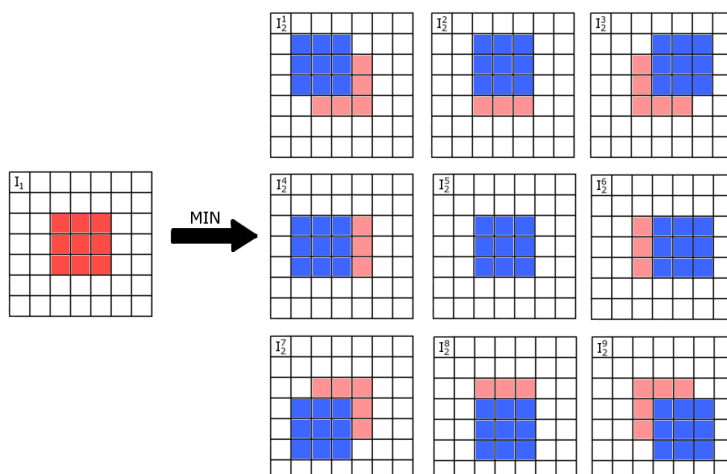
7.2 Przepływ optyczny (ang. *optical flow*)

Przepływ optyczny to pole wektorowe opisujące ruch pikseli (ew. tylko wybranych, tj. punkty charakterystyczne) pomiędzy dwoma kolejnymi ramkami z sekwencji wideo. Inaczej mówiąc, dla każdego piksela na ramce I_{n-1} wyznaczamy wektor przesunięcia, w rezultacie którego otrzymamy obraz I_n . Warto jednak zaznaczyć, że przepływ można (a niekiedy wręcz jest to pożądane) obliczać dla ramek o odstępach większym niż '1'.

Przepływ optyczny może być gęsty (ang. *dense*) lub rzadki (ang. *sparse*). W pierwszym przypadku przemieszczenie wyznaczane jest dla każdego piksela na obrazie, a w drugim tylko dla pewnego ich ograniczonego zbioru.

Podstawą działania przepływu optycznego jest „śledzenie” pikseli (lub pewnych lokalnych konfiguracji pikseli, tj. otoczeń) pomiędzy ramkami. Oznacza to, że na kolejnych ramkach poszukujemy tych samych pikseli (lub też ich niewielkich otoczeń). Zakłada się przy tym, że jasność piksela jest stała (w skali odcieni szarości, w ogólnym przypadku – stały kolor).

Niestety, jak pokaże ćwiczenie, podejście nie sprawdza się w kilku przypadkach, z których najważniejszy to duże, jednorodne powierzchnie. Dlaczego tak się dzieje? Wyobraźmy sobie dość duże kartonowe pudełko. Ma ono szare, jednorodne ściany bez tekstury oraz oświetlone jest równomiernie. Chcemy wyznaczyć przepływ optyczny dla wszystkich pikseli na obrazie, w tym dla piksela ze środka ścianki pudełka. Czy uda nam się znaleźć na



Rysunek 7.1: Przykład poszukiwania odpowiadającego bloku

kolejnej ramce dokładnie ten punkt? Czy jednoznaczne przypisanie jest możliwe?

Z tej przyczyny czasem korzystniej jest zastosować przepływ rzadki, obliczany w uprzednio określonych lokalizacjach. Tutaj mamy dwie możliwości. Albo arbitralnie wybieramy punkty np. na kwadratowej lub prostokątnej siatce (wtedy celem jest redukcja czasu obliczeń), albo wyszukujemy tzw. punkty charakterystyczne – mogą to być krawędzie, narożniki lub inne „wyrządzone” elementy obrazu. Istnieje szereg algorytmów ich detekcji, np. detektor narożników Harris’a, detektory punktów charakterystycznych SIFT lub SURF itp.

W literaturze opisano bardzo dużo metod – proszę zobaczyć na rankingi dostępne na stronach [Middlebury](#), [KITTI](#), [Sintel](#). Dwie najpopularniejsze („klasyczne”) to Horn-Schunck (HS) i Lucas-Kanade (LK). Druga z nich jest dostępna w bibliotece OpenCV.

7.3 Implementacja metody blokowej

Metoda blokowa, jak sama nazwa wskazuje, polega na dopasowywaniu pewnych bloków pikseli między kolejnymi ramkami. Z jednego obrazu wycinamy bloki o określonym rozmiarze (np. 3×3 , 5×5 itp.), stosując metodę okna przesuwającego po całym obrazie. Dla drugiego obrazu poszukujemy najbardziej podobnego bloku do tego wyciętego, przy czym poszukiwania ograniczamy do niewielkiego otoczenia współrzędnych wyciętego bloku. Takie postępowanie ma dwa uzasadnienia – zwykle przemieszczenie pikseli między kolejnymi ramkami jest niewielkie, a ponadto znacznie ograniczamy czas wykonywania algorytmu.

Ćwiczenie 7.1 Zaimplementuj metodę blokową do wyznaczania przepływu optycznego.

1. Utwórz nowy skrypt, wczytaj dwie kolejne ramki z sekwencji, które nazwiemy I oraz J (odpowiednio wcześniejsza i późniejsza), przykładowo ramki numer 150 oraz 151. Opcjonalnie możesz zmniejszyć oba obrazy na czas testów, np. czterokrotnie – obliczenia będą się wykonywały szybciej. Wykonaj konwersję obrazów do odcieni szarości – `cvtColor`. Wyświetl zadane obrazy oraz różnicę, wykorzystując `cv2.absdiff`.
2. Zaimplementuj metodę blokową wyznaczania przepływu optycznego. Przyjmij następujące założenia – porównujemy fragmenty obrazu o rozmiarze 7×7 . Wykorzystywane dalej oznaczenia oraz wartości dla okna 7×7 :

- $W2 = 3$ – wartość całkowita z połowy rozmiaru okna,
 - $dX = dY = 3$ – rozmiar przeszukiwania (maksymalnego przesuwania okna) w obu kierunkach.
3. Implementację należy zacząć od dwóch pętli `for` po obrazie. Wykorzystaj parametr $W2$ do warunku uwzględniającego przypadek brzegowy – zakładamy, że na brzegu nie wyliczamy przepływu optycznego.
 4. Wewnątrz pętli wycinamy fragment ramki I . Dla przypomnienia, niezbędna składnia to: `I0 = np.float32(I[j-W2:j+W2+1,i-W2:i+W2+1])`. W rozważaniach przyjmujemy, że j – indeks pętli zewnętrznej (po wierszach), i – indeks pętli wewnętrznej (po kolumnach). Proszę zwrócić uwagę na składnik „+1”. Konwersja na typ zmiennoprzecinkowy potrzebna jest do dalszych obliczeń.
 5. Następnie realizujemy kolejne dwie pętle `for` – przeszukiwanie otoczenia piksela $J(j,i)$. Mają one zakres od $-dX$ do dX i $-dY$ do dY . Wewnątrz pętli należy sprawdzić, czy współrzędne aktualnego kontekstu (tj. jego środek) mieszczą się w dopuszczalnym zakresie. Alternatywnie można też zmodyfikować zakres zewnętrznych pętli, aby wykluczyć dostęp do pikseli spoza dopuszczalnego zakresu.
 6. Wycinamy otoczenie $J0$, dokonujemy konwersji na `float32` a następnie obliczamy „odległość” między wycinkami $I0$ a $J0$. Można to wykonać instrukcją: `np.sqrt(np.sum((np.square(J0-I0))))`. Spośród wszystkich wycinków z $J0$ dla danego $I0$ należy znaleźć najmniejszą „odległość” – lokalizację „najbardziej podobnego” do $I0$ fragmentu na obrazie J .
 7. Uzyskane współrzędne znalezionych minimów należy zapisać w dwóch macierzach (przykładowo u i v) – należy je wcześniej utworzyć i zainicjować zerami, ich wymiary są takie jak dla obrazu I .
 8. Wyznaczone pole przepływu optycznego można zwizualizować na dwa sposoby – poprzez wektory (strzałki) – funkcja `plt.quiver` z biblioteki `matplotlib` lub kolory – zrealizujemy drugie z tych podejść. Pomysł opiera się na określeniu kąta i długości wektora wyznaczonego przez dwie składowe przepływu optycznego u i v dla każdego piksela. Otrzymamy wówczas reprezentację danych podobną jak w przestrzeni barw HSV. Po wyświetleniu kolor oznacza kierunek, w którym przemieszczają się piksele, natomiast jego nasycenie informuje o względnej szybkości ruchu pikseli – przeanalizuj koło kolorów na rys. 7.2.

Dokonaj konwersji wyznaczonego przepływu do układu współrzędnych biegunowych – `cartToPolar`. Utwórz zmienną na obraz w przestrzeni HSV o wymiarach wejściowego obrazu, 3 kanałach i typie `uint8`. Pierwszy kanał to składowa H , równa $angle \cdot 90 / np.pi$ (w Pythonie zakres od 0 do 180). Drugi kanał to składowa S – dokonaj normalizacji długości wektora do przedziału 0-255. Trzeci kanał to składowa V , ustaw ją na 255.

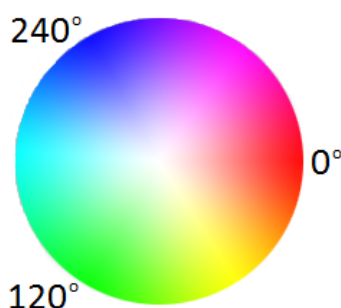
 Aby brak ruchu oznaczony był kolorem czarnym, a nie białym, należy zamienić kanały S i V .

Na koniec wykonaj konwersję obrazu do przestrzeni RGB i wyświetl go.

9. Proszę przeanalizować uzyskane wyniki oraz poeksperymentować z rozmiarem obrazu wejściowego oraz z parametrami $W2$ oraz dX oraz dY (np. obraz dwukrotnie zmniejszony, parametry równe 5). Czy dla obrazu wejściowego w oryginalnych rozmiarach takie parametry pozwalają poprawnie wyznaczyć przepływ optyczny? Jak duże okno jest potrzebne, aby otrzymać podobne wyniki?

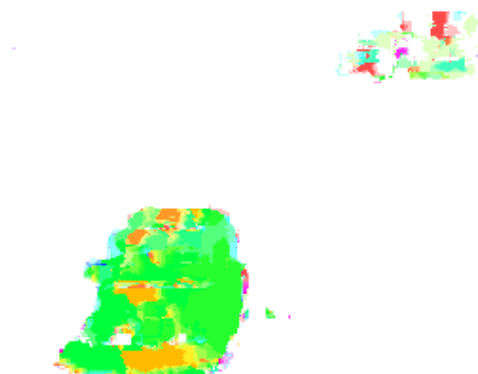
R Metoda blokowa z uwagi na swą prostotę jest dość niedokładna – wyliczone przesunięcie w poziomie lub w pionie jest całkowitoliczbowe. W innych metodach (np. HS czy LK) przepływ optyczny jest przeważnie niecałkowity.

10. Aby ograniczyć liczbę niepoprawnych dopasowań, można dodać pewną filtrację wyników na podstawie wyliczonej wcześniej różnicy jasności między ramkami. Zbinaryzuj ją, po czym wykonaj dylatację (przetestuj różne rozmiary masek i liczby iteracji). Na koniec “przepuszczamy” tylko te wyniki przepływu, które są wyliczone dla pikseli znajdujących się na obrazie różnicy po dylatacji.



Rysunek 7.2: Koło kolorów

Na rys. 7.3 zamieszczono przykładowy wynik dla analizowanej sekwencji testowej.

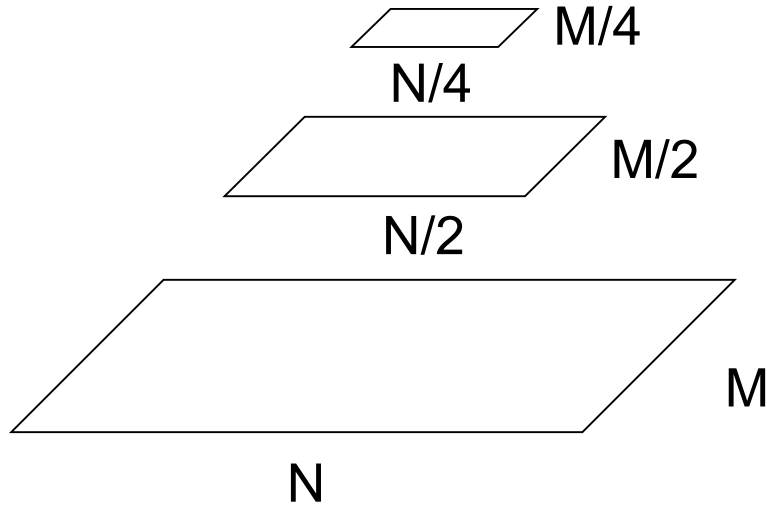


Rysunek 7.3: Przykładowy wynik przepływu optycznego wyznaczonego metodą blokową dla obrazów I oraz J (ramki numer 150 i 151 z sekwencji *highway* dla parametrów $W2 = dX = dY = 3$, rozmiaru okna 7×7 i oryginalnej rozdzielczości obrazu).

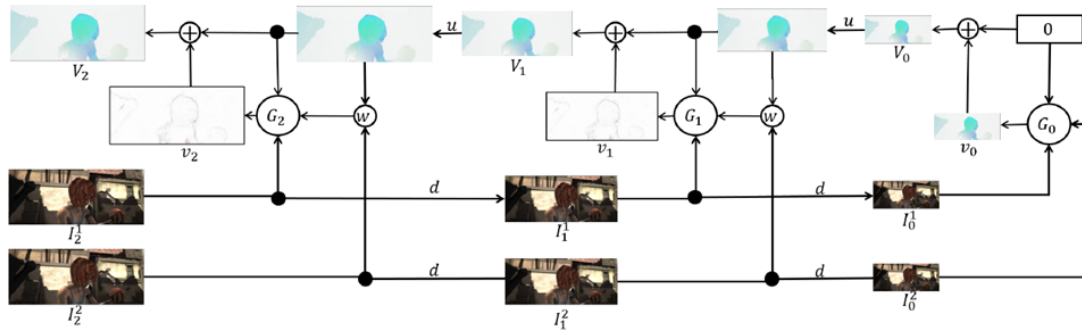
7.4 Implementacja wyliczania w wielu skalach

Wykrywanie dużych przemieszczeń zaimplementowaną metodą jest złożone obliczeniowo. Wynika to z konieczności użycia dużych wartości parametrów dX i dY , a zatem dla większości lokalizacji (oprócz brzegowych) konieczne będzie sprawdzenie podobieństwa większej liczby kontekstów (więcej operacji SAD). Takie podejście jest bardzo nieefektywne.

Lepszym, a także bardzo często stosowanym pomysłem, jest przetwarzanie obrazu w wielu skalach. Idea pokazana jest na rysunku 7.4.



Rysunek 7.4: Przykład konstrukcji piramidy obrazów (wielu skal)



Rysunek 7.5: Prowadzenie obliczeń w wielu skalach (w tym przypadku 3) na przykładzie sieci neuronowej SpyNet <https://spynet.is.tue.mpg.de/>. G to algorytm wyznaczania przepływu (dowolny), w to operacja warpingu, v to przepływ uzyskany w danej skali, V to przepływ całkowity, zsumowany z wielu skal, u to obraz (przepływ) po zwiększeniu wymiarów do wyższej skali.

Schemat działania metody:

- wykonujemy przeskalowanie obrazów (I oraz J) do skal: L_0 – rozdzielczość oryginalna, $L_1, \dots, L_m$ (najmniejszy obraz) – w literaturze określa się to mianem piramidy obrazów. W tego typu rozwiązaniach najczęściej wykorzystuje się skalowanie ze współczynnikiem 0.5.
- obliczamy przepływ optyczny dla pary obrazów I oraz J w skali L_m (np. metodą blokową).
- propagujemy wyniki obliczonego przepływu na skalę L_{m-1} :
 - w pierwszym kroku otrzymany przepływ zostaje przeskalowany do rozmiaru L_{m-1} , a same wartości przepływu również zostają odpowiednio zwiększone,
 - następnie druga ramka obrazu zostaje zmodyfikowana zgodnie z przepływem (ang. *warping*). Celem takiego zabiegu jest kompensacja ruchu, czyli przesunięcie pikseli w taki sposób, aby w większej skali L_{m-1} odpowiadające sobie na dwóch obrazach piksele były bliżej siebie.
- obliczamy dokładniejsze wartości przepływu na poziomie L_{m-1} pomiędzy obrazem I a zmodyfikowanym J (np. metodą blokową),

- osobno sumujemy przepływy otrzymane w różnych skalach,
- postępowanie realizujemy aż do poziomu L_0 .

Porównaj opis z rysunkiem 7.5.

Ćwiczenie 7.2 Zaimplementuj wieloskalową wersję metody blokowej do wyznaczania przepływu optycznego.

1. Na wstępie proszę stworzyć kopię dotychczas stworzonego algorytmu w ramach zadania 1. Następnie fragment algorytmu dotyczący wyliczania OF zamieniamy na **funkcję** dla wybranej skali.

Funkcja powinna mieć następującą postać:

```
def of(J_org, I, J, W2=3, dY=3, dX=3):
```

Poszczególne parametry to J_org i I – obrazy wejściowe, J – obraz po modyfikacji, W2, dX, dY – parametry metody (identyczne jak w 7.3). Tak naprawdę przesyłanie J_org do funkcji nie jest potrzebne – sugerowane rozwiązanie ma na celu obliczenie absdiff między obrazami i wyświetlenie wyników oraz obrazów J_org, I oraz J dla lepszego zrozumienia działania metody. W rzeczywistości obliczenia prowadzone są dla I i J. W funkcji można, tak jak wcześniej, wykonać dodatkową binaryzację i dylatację.

2. Przydatne będzie także zamienienie fragmentu algorytmu dotyczącego wizualizacji przepływu optycznego na **funkcję**. Funkcja ta powinna być postaci:

```
def vis_flow(u, v, YX, name):
```

Poszczególne parametry to u i v – składowe przepływu optycznego, YX – wymiary obrazu, name – wyświetlana nazwa okna, np. 'of scale 2'.

3. Po napisaniu funkcji dobrze jest przetestować jej działanie dla przypadku jednej skali (L_0) – wynik powinien być identyczny jak otrzymywany wcześniej.
4. Po utworzeniu dwóch funkcji na podstawie napisanego już algorytmu, przechodzimy do napisania funkcji do generowania piramidy obrazów. Można wykorzystać następującą funkcję:

```
def pyramid(im, max_scale):
    images=[im]
    for k in range(1, max_scale):
        images.append(cv2.resize(images[k-1], (0,0), fx=0.5, fy=0.5))
    return images
```

W tym przypadku zakładamy, że pomniejszenia rozdzielczości zawsze będą dwukrotne. Czyli obraz wejściowy pomniejszamy 2-krotnie, 4-krotnie, 8-krotnie itd. Na potrzeby eksperymentu zakładamy, że ograniczamy się do 2-3 skal. Piramidę należy wygenerować dla każdego z obrazów wejściowych.

5. Przetwarzanie zaczynamy od skali najmniejszej, zatem do zmiennej I przypisujemy obraz z piramidy w najmniejszej skali: $I = IP[-1]$, gdzie IP to wygenerowana piramida dla pierwszego obrazu. Proszę zwrócić uwagę na składnię $[-1]$ – dostęp do ostatniego elementu.
6. Kluczowy komponent algorytmu to pętla po skalach – proszę zastanowić się nad indeksem początkowym i końcowym. Zaczynamy od wyliczenia przepływu i dwukrotnego zwiększenia jego wymiarów i wartości. Zwiększenie: `cv2.resize((2**s)*u, (0,0), fx=2**s, fy=2**s, interpolation=cv2.INTER_LINEAR)`.
7. Robimy kopię drugiego obrazu J_new. Następny krok to modyfikacja tej kopii zgodnie z przepływem. Tę operację wykonujemy dla wszystkich skal oprócz naj-

większej (czyli obrazu o wejściowych wymiarach). Można to zrealizować przy wykorzystaniu dwóch pętli `for` z ew. zabezpieczeniem przed wyjściem poza zakres. Poszczególnym pikselom z `J_new` przypisujemy odpowiednie piksele z `J`, zgodnie z wyliczonym przepływem. Warto sprawdzić, czy obraz drugi po modyfikacji jest zbliżony do obrazu pierwszego (`I`) – jeśli tak jest, to operacja została zrealizowana poprawnie. Na koniec obraz `J_new` przypisujemy do `J`.

8. Ostatni element algorytmu to wyliczenie całkowitego przepływu i jego wizualizacja. W tym celu należy przygotować pustą tablicę 2D dla każdej ze składowych `u` i `v` o wymiarach wejściowego obrazu. Następnie w pętli po skalach dodajemy do siebie przepływy z różnych skal.

R Krótki przykład dla wyjaśnienia. Mamy piksel, którego przemieszczenie wynosi 9 (*ground truth*), jednak przeszukiwanie mamy ograniczone do 5 pikseli w każdym kierunku. Obraz zmniejszamy dwa razy, teraz przemieszczenie piksela powinno wynosić 4.5. Znajdujemy najbardziej pasujący piksel w mniejszej skali – otrzymujemy przesunięcie równe 5, zatem w większej skali przesunięcie byłoby równe 10. Zgodnie z tym wynikiem modyfikujemy obraz w większej skali (czyli przesuwamy piksel o 10 w odpowiednim kierunku). Liczymy ponownie przepływ, tym razem w większej skali, i otrzymujemy przesunięcie równe -1 (przepływ rezydualny). Następnie sumujemy wyniki z obu skal i otrzymujemy całkowite przesunięcie równe 9.

Wnioski. Aby otrzymać poprawny wynik końcowy, należy przepływy `u` i `v` z każdej skali zwiększyć na dwa sposoby – wymiary tych „obrazów” muszą być takie, jak dla wejściowego `I`, a wartości przepływów muszą być zwiększone 2, 4, 8 itd. razy, w zależności od skali. Przepływ zsumowany z różnych skal należy zwizualizować przy pomocy funkcji `vis_flow`.

9. Porównaj działanie metody ze skalami i bez. Przeanalizuj wyniki dla większych rozmiarów okien w jednej skali oraz dla mniejszych rozmiarów okien z wieloma skalami, zwróć uwagę na czas wykonania. Czy stosując metodę wieloskalową (np. 2 skale), udało się uzyskać poprawny przepływ dla niewielkiego okna (np. $dX = dY = 3$) dla obrazu o oryginalnym rozmiarze (czyli dla większego przemieszczenia pikseli)?

R Metoda wieloskalowa w teorii działa bardzo dobrze – w praktyce dopasowanie pikseli nie zawsze jest dokładne, a skalowanie w dół i w górę zawsze powoduje utratę pewnych informacji, przez co nawet drobne błędy propagowane są na kolejne skale i końcowy wynik może być częściowo nieprawidłowy bądź zaszumiony. Z tego powodu w praktyce używa się 2, 3, maksymalnie 4 skale (ale i tak wyniki różnią się nieco od „oczekiwanych”).

Na rys. 7.6 oraz 7.7 zamieszczone zostały odpowiednio obraz po warpingu oraz całkowity przepływ optyczny dla 2 skal.



Rysunek 7.6: Obraz po modyfikacji (warpingu)



Rysunek 7.7: Całkowity przepływ optyczny zsumowany z 2 skal

7.5 Inne sposoby wyznaczania przepływu optycznego

Wykorzystaj stworzony w ramach wcześniejszych ćwiczeń skrypt do odczytu sekwencji obrazów. Upewnij się, że zawiera on parametr `iStep`, który pozwala ustawić, co która ramka będzie przetwarzana.

R Dla uproszczenia analizowane będą sekwencje w odcieniach szarości.

W bibliotece OpenCV dostępnych jest kilka funkcji do obliczania przepływu optycznego:

- Lucas-Kanade (w wersji wieloskalowej, rzadki i gęsty),
- Farneback,
- Dual TV-L1,
- PCA Flow,
- DIS Flow,
- Simple Flow,
- Deep Flow.

R Szczegóły na temat konkretnych algorytmów są dostępne w [dokumentacji OpenCV](#).

Warto też zapoznać się z [tutorialem do przepływu optycznego](#) i go przeanalizować.

Ćwiczenie 7.3 Uruchom przykładowe rozwiązania.

1. Zadanie 1 – wyznacz przepływ optyczny metodami gęstymi (czyli wszystkimi poza rzadkim LK). Kod powinien się składać z kilku elementów:

- wczytywanie obrazów wejściowych i konwersja do odcieni szarości,
- utworzenie zmiennej `flow` o dwóch kanałach (dla `u` i `v`),
- utworzenie instancji danej metody wyznaczania przepływu optycznego i wywołanie na niej `calc`,
- wizualizacja wyników jak w rozdziale 7.3.

Zwróć uwagę na wyniki dla różnych metod (m.in. dokładność wyznaczenia krawędzi obiektów), a także na czas wykonywania obliczeń.

- R** Do wizualizacji gęstego LK przydatne może być ograniczenie modułu przepływu do przedziału 0-10, np. poprzez `mag[mag>10]=10`.

2. Zadanie 2 – uruchomić rzadki przepływ optyczny metodą Lucas-Kanade. W [tutoriale](#) pokazano śledzenie punktów charakterystycznych. My obliczymy przepływ optyczny dla równomiernie rozmieszczonych punktów. Zatem kod należy odpowiednio zaadaptować. Kilka uwag:

- algorytm ma szereg parametrów, które po analizie wykładu powinny być jasne (ew. proszę doczytać w dokumentacji) – rozmiar okna, liczba skal, kryteria zakończenia obliczeń (rozwiązanie układu równań),
- należy wygenerować siatkę równomiernie rozmieszczonych punktów (np. krok 10 pikseli) – tu warto podglądać postać `p0` z przykładu,
- druga trudność to inna wizualizacja. Przykładowe rozwiązanie:

```
img = I
for jj in range(0, y, 1):
    if (st[jj] == 1):
        cv2.line(img, (points[jj,0,0],points[jj,0,1]), (new_points
            [jj,0,0],new_points[jj,0,1]), (0,0,255))
```

- proszę nie zapomnieć o przypisaniu aktualnego obrazu do poprzedniego na końcu pętli.

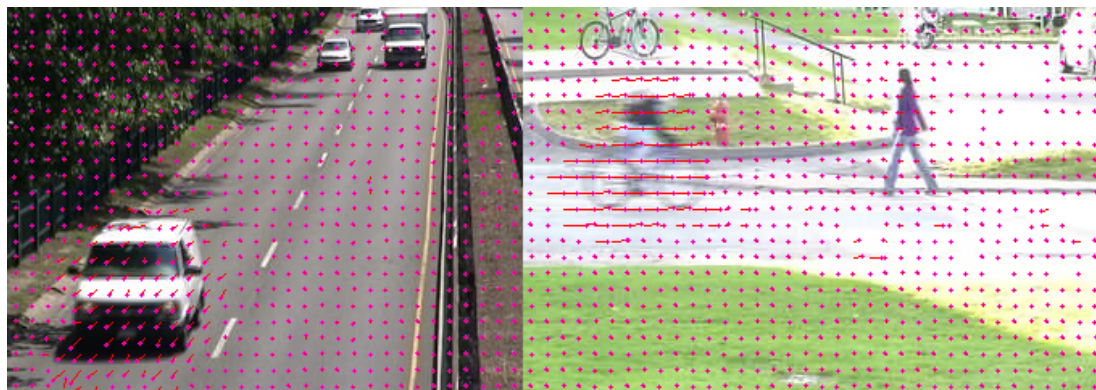
Proszę zaobserwować jak działa algorytm – czy wyznaczane przepływy są poprawne? Do wyświetlania można dodać wizualizację zwrotu – zaznaczyć początek lub koniec wektora kropką. Można też usunąć wektory o niewielkim module.

Na rys. 7.8 zamieszczone zostały przykładowe wyniki dla analizowanych sekwencji testowych.



7.6 Detekcja kierunku ruchu i klasyfikacja obiektów z wykorzystaniem przepływu optycznego

Pomysł jest następujący. Samochód, który jest bryłą sztywną, powinien charakteryzować się spójnym ruchem – każdy jego element powinien mieć podobne przemieszczenie, przynajmniej co do kierunku. Idący człowiek wprost przeciwnie, gdyż specyfika chodu powoduje, że kończyny przemieszczają się w różnych kierunkach (w zależności od fazy kroku).



Rysunek 7.8: Przykładowe wyniki rzadkiego przepływu optycznego wyznaczonego metodą LK: po lewej stronie dla obrazu o indeksie 158 z sekwencji *highway*; po prawej stronie dla obrazu o indeksie 471 z sekwencji *pedestrians*.

Spróbujemy zatem przeanalizować wektory ruchu dla każdego z obiektów (w zależności od sekwencji samochodu lub człowieka) i zobaczyć, czy na tej podstawie można coś powiedzieć i ew. dokonać klasyfikacji.

Ćwiczenie 7.4 Zadanie nieobowiązkowe.

Zaimplementuj algorytm, bazując na programie z przepływem optycznym z poprzedniego zadania (np. z algorytmem Farnebacka).

1. Na początku dodamy segmentację obiektów pierwszoplanowych. Wykorzystaj użyty na poprzednich laboratoriach algorytm MOG/GMM.
2. Bardzo istotna kwestia to filtracja maski. Tu również przydatne będzie doświadczenie zdobyte na poprzednich ćwiczeniach. Dobrym pomysłem wydaje się wyłączenie detekcji cieni – `detectShadows=False` w konstruktorze MOG2.
3. W kolejnym kroku dodajemy indeksację, której wynik wyświetlamy.
4. Następnie wykonujemy analizę przepływu optycznego. Sprowadza się ona do obliczenia średniej i odchylenia standardowego dla modułu i kąta wektorów odpowiadających danemu obiektowi. Stopień trudności zadania zależy od biegłości w Pythonie. Kilka podpowiedzi (można, ale nie trzeba skorzystać):
 - obliczenia prowadzimy tylko gdy są obiekty – `retval > 0`,
 - kluczem jest dogodny „kontener” na średnie i odchylenia. Można wykorzystać listy i instrukcję `append`. Wtedy uzyskamy taką funkcjonalność jak w Matlabie. Uwaga. Łatwiej użyć dwóch osobnych list, gdyż prostsze jest późniejsze obliczanie statystyk.
 - zasadnicze obliczenia realizujemy w pętli po obrazku. Jeśli etykieta jest różna od zera to:
 - sprawdzamy, czy amplituda przepływu optycznego w tym punkcie jest większa od zadanego progu (np. 1),
 - jeśli tak, to obliczamy kąt wektora (`atan2` z `math`) i wynik dodajemy do naszej listy pod „odpowiednim adresem”.
 - w kolejnym kroku obliczamy średnią i odchylenie standardowe dla modułu i kąta. Przydatne będą funkcje `mean` i `stdev` z pakietu `statistics`.



Przed obliczeniem trzeba sprawdzić, czy istnieją co najmniej dwa elementy na liście dla danego obiektu.

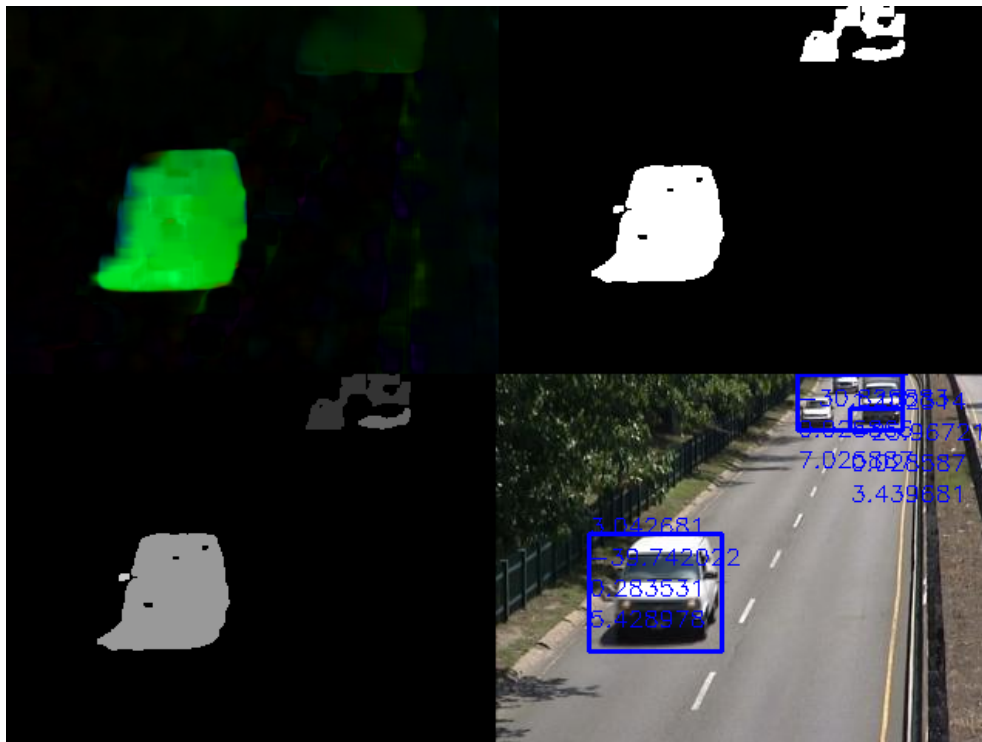
- ostatni etap to wizualizacja. Najprościej kod z ćwiczenia nt. segmentacji obiektów ruchomych przystosować do wyświetlania wyznaczonych parametrów – dwóch średnich i dwóch odchyłeń standardowych. Przy okazji można dodać filtrację małych obiektów.

5. Analiza wyników:

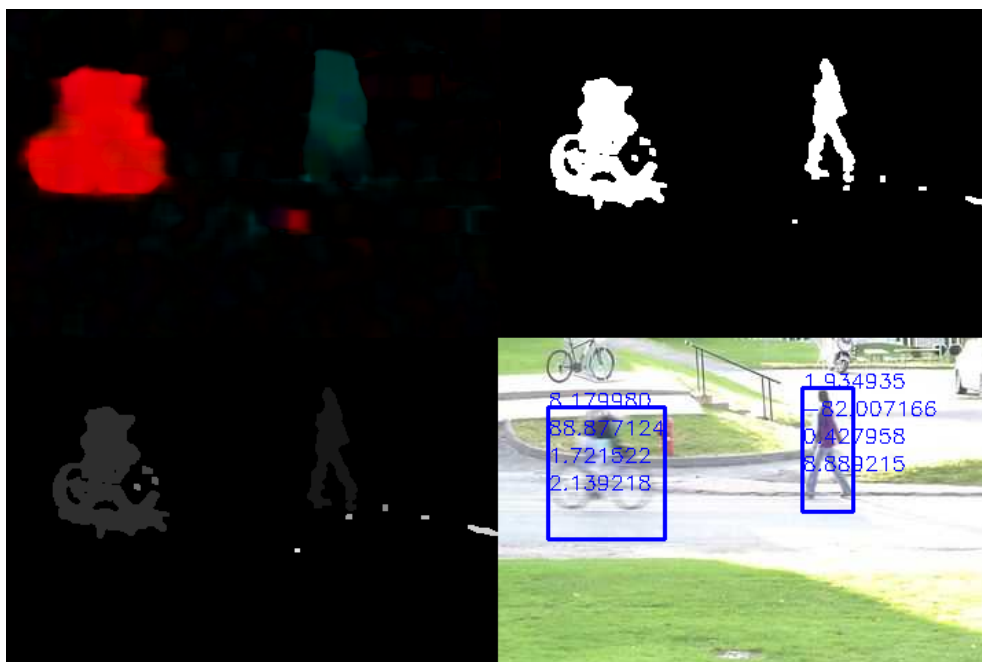
- Jak zachowuje się średnia i odchylenie standardowe dla obu sekwencji?
- Czy średni kierunek ruchu odpowiada „mniej więcej” rzeczywistości?
- Czy da się zauważyć jakąś różnicę pomiędzy obiema sekwencjami – odchylenie standardowe?
- Czy występują problemy z segmentacją? Jakie?

Ewentualnie można spróbować wyświetlić dla każdego obiektu średni wektor ruchu.

Na rys. 7.9 oraz 7.10 zamieszczone zostały przykładowe wyniki dla analizowanych sekwencji testowych.



Rysunek 7.9: Przykładowe wyniki dla obrazu o indeksie 146 z sekwencji *highway*: po lewej u góry – przepływ optyczny metodą Farnebacka, po prawej u góry – segmentacja obiektów pierwszoplanowych, po lewej na dole – indeksacja, po prawej na dole – wykryte obiekty wraz z wyliczonymi parametrami.



Rysunek 7.10: Przykładowe wyniki dla obrazu o indeksie 471 z sekwencji *pedestrians*: po lewej u góry – przepływ optyczny metodą Farnebacka, po prawej u góry – segmentacja obiektów pierwszoplanowych, po lewej na dole – indeksacja, po prawej na dole – wykryte obiekty wraz z wyliczonymi parametrami.

8

Laboratorium 8

8	Śledzenie obiektów	77
8.1	Filtr korelacyjny	
8.2	Filtr korelacyjny - zadanie	
8.3	Syjamska sieć neuronowa	
8.4	Sieć syjamska - zadanie	

8. Śledzenie obiektów

8.1 Filtr korelacyjny

Jednym z podejść do śledzenia jest wykorzystanie metod z rodziny filtrów korelacyjnych. W algorytmach tych wykorzystywany jest pewien filtr, za pomocą którego zamodelowany jest śledzony obiekt. W celu predykcji nowego położenia obiektu, dokonywana jest korelacja obszaru śledzenia z filtrem. Obliczenia wykonywane są w dziedzinie częstotliwości, ponieważ szybciej jest obliczyć transformatę Fouriera za pomocą algorytmu FFT (*Fast Fourier Transform*) i skorzystać z twierdzenia o konwolucji, niż obliczać korelację wprost. Algorytmy śledzenia z rodziny filtrów korelacyjnych cechuje niska złożoność obliczeniowa (czyli możliwe jest śledzenie w czasie rzeczywistym) przy zachowaniu konkurencyjnej jakości śledzenia.

Przedstawiona zostanie najprostsza wersja algorytmu opisana w publikacji: [Visual Object Tracking using Adaptive Correlation Filters](#).

W pierwszej klatce obrazu filtr H jest inicjalizowany na podstawie fragmentu obrazu w skali szarości f , który zawiera obiekt do śledzenia.



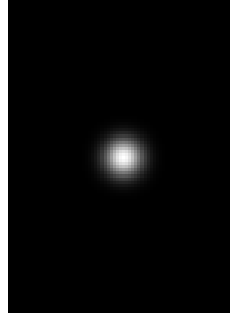
Rysunek 8.1: Wskazany obiekt do śledzenia.

$$H_t^* = \frac{A}{B} \quad (8.1)$$

$$A = \sum_{i=1}^N G_i \odot F_i^* \quad (8.2)$$

$$B = \sum_{i=1}^N F_i \odot F_i^* \quad (8.3)$$

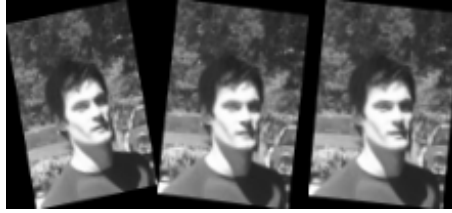
gdzie F_i jest dyskretną transformatą Fouriera fragmentu obrazu f_i , natomiast G jest transformatą Fouriera dwuwymiarowego rozkładu Gaussa g o rozmiarze śledzonego obszaru:



Rysunek 8.2: Przykład dyskretnego, dwuwymiarowego rozkładu Gaussa g o rozmiarze śledzonego obiektu.

Działanie $\odot$ oznacza mnożenie typu element przez element, a X^* oznacza sprzężenie zespolone (wynikiem transformacji Fouriera są liczby zespolone).

Suma po i oznacza, że z jednej próbki obiektu w pierwszej klatce śledzenia należy utworzyć zbiór N próbek uczących składających się z losowych rotacji próbki bazowej:



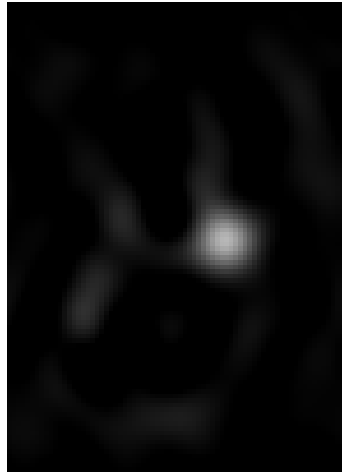
Rysunek 8.3: Próbkki uczące.

Następnie, w każdej klatce obrazu dokonywana jest predykcja położenia:

$$\mathcal{F}^{-1}\{F \odot H^*\} \quad (8.4)$$

gdzie $\mathcal{F}^{-1}$ oznacza odwrotną, dyskretną transformację Fouriera, natomiast F jest tutaj transformatą fragmentu obrazu scentrowanego na ostatnio znanej pozycji obiektu. Otrzymana w ten sposób mapa korelacji reprezentuje przewidziane przez filtr przesunięcie obiektu względem poprzedniej klatki obrazu:

Po każdej predykcji, filtr jest aktualizowany fragmentem aktualnej klatki obrazu wyciętym na nowej pozycji obiektu:



Rysunek 8.4: Mapa korelacji.

$$A_t = \eta G \odot F_t^* + (1 - \eta) A_{t-1} \quad (8.5)$$

$$B_t = \eta F_t \odot F_t^* + (1 - \eta) B_{t-1} \quad (8.6)$$

gdzie $\eta \in [0, 1]$ jest przyjętym współczynnikiem uczenia.

8.2 Filtr korelacyjny - zadanie

Ćwiczenie 8.1 Celem ćwiczenia jest zapoznanie się z metodą wykorzystującą filtry korelacyjne do śledzenia obiektów.

1. Pobierz materiały oraz szablon zadania z platformy UPeL.
2. Zadanie zostało przygotowane w notatniku Jupyter. Plik `.ipynb` można otworzyć i uruchomić za pomocą Google Colab (colab.research.google.com). Również możliwe jest uruchomienie pliku za pomocą edytora VS Code. Prawdopodobnie będzie wymagane zainstalowanie odpowiednich bibliotek. Jeśli wybieramy VS Code pomijamy wczytywanie biblioteki:

```
from google.colab import drive
drive.mount('/content/drive')
```

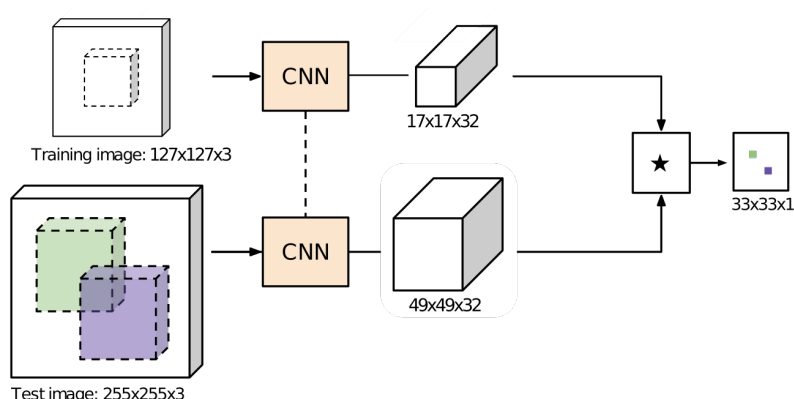
3. Wykonaj polecenia umieszczone w pliku.
4. Rozwiązaniem zadania jest uzupełniony oraz działający plik `.ipynb`, który należy wysłać na platformę UPeL.

8.3 Syjamska sieć neuronowa

Sieć syjamska to sieć w kształcie litery Y, w której dwie gałęzie są połączone w celu uzyskania pojedynczego wyjścia. Niezależnie od struktury gałęzi, sieć syjamską można traktować jako funkcję podobieństwa, która mierzy podobieństwo między dwoma wejściami. Podstawową architekturę sieci syjamskiej możemy zdefiniować za pomocą równania (8.7), gdzie ϕ oznacza ekstrakcję głębokich cech (np. poprzez gałęzie sieci), z reprezentuje wzorec (np. przykład, obiekt do śledzenia), x reprezentuje obszar, który jest porównywany z wzorcem (np. region wyszukiwania, ROI), a γ jest korelacją krzyżową.

$$y = \gamma(\phi(z), \phi(x)) \quad (8.7)$$

Jej schemat został przedstawiony na rysunku 8.5. Wejście z reprezentuje śledzony obiekt wybrany w pierwszej klatce i przeskalowany do rozmiaru 127×127 pikseli. W kolejnych klatkach wybierany jest obszar ROI wokół poprzedniej lokalizacji obiektu i przekazywany do sieci jako wejście x , skalowany do 255×255 pikseli. Oba obszary (przykład i ROI) dają w efekcie dwa bloki map cech, o rozmiarach odpowiednio $17 \times 17 \times 32$ (dla z) i $49 \times 49 \times 32$ (dla x). Na koniec są one krzyżowo skorelowane i uzyskuje się mapę ciepła o rozmiarze 33×33 . Jej najwyższy szczyt wskazuje lokalizację obiektu. Obszar ROI jest wybierany około 4 razy większy niż rozmiar obiektu, w pięciu skalach, aby uwzględnić zmiany rozmiaru obiektu.



Rysunek 8.5: Konwolucyjna sieć syjamska do śledzenia.

Oryginalny opis algorytmu można znaleźć pod adresem: [Fully-Convolutional Siamese Networks for Object Tracking](#).

8.4 Sieć syjamska - zadanie

Ćwiczenie 8.2 Celem ćwiczenia jest zapoznanie się z architekturą syjamskiej sieci neuronowej do śledzenia obiektów. Jest to przykład, w którym zostanie wykorzystana sieć neuronowa.

1. Pobierz materiały oraz szablon zadania z platformy UPeL.
2. Zadanie zostało przygotowane w notatniku Jupyter. Plik `.ipynb` można otworzyć i uruchomić za pomocą Google Colab (colab.research.google.com). Również możliwe jest uruchomienie pliku za pomocą edytora VS Code. Prawdopodobnie będzie wymagane zainstalowanie odpowiednich bibliotek. Jeśli wybieramy VS Code pomijamy wczytywanie biblioteki:

```
from google.colab import drive
drive.mount('/content/drive')
```

3. Wykonaj polecenia umieszczone w pliku.
4. Rozwiązaniem zadania jest uzupełniony oraz działający plik `.ipynb`, który należy wysłać na platformę UPeL.

9

Extra laboratorium 2

9	Detekcja sylwetek ludzkich	83
9.1	Cel zajęć:	
9.2	Histogram of Oriented Gradients – opis algorytmu	
9.3	Support Vector Machines – opis algorytmu	
9.4	Deskryptor HOG	
9.5	Klasyfikator SVM	
9.6	Detekcja sylwetki ludzkiej	

9. Detekcja sylwetek ludzkich

9.1 Cel zajęć:

- zapoznanie z zagadnieniem detekcji sylwetek ludzkich,
- zapoznanie z deskryptorem HOG (ang. *Histogram of Oriented Gradients*),
- zapoznanie z klasyfikatorem SVM (ang. *Support Vector Machine*),
- podstawy uczenia maszynowego (metodologia).

9.2 Histogram of Oriented Gradients – opis algorytmu

Histogram zorientowanych gradientów (HOG)¹ jest deskryptorem cech szeroko stosowanym w przetwarzaniu i rozpoznawaniu obrazów. Lokalny wygląd i kształt obiektu opisuje poprzez rozkład lokalnych gradientów intensywności oraz kierunków krawędzi. Algorytm HOG można podzielić na kilka etapów:

1. Obliczanie gradientu,
2. Wyznaczenie histogramu kierunków gradientu w komórkach (ang. *cells*),
3. Normalizacja histogramów w blokach komórek,
4. Wyznaczenie wektora cech.

Obliczanie gradientu

Dla każdego piksela obliczany jest gradient w pionie i w poziomie za pomocą dwóch jednowymiarowych masek: $\begin{pmatrix} -1 & 0 & 1 \end{pmatrix}$ i $\begin{pmatrix} -1 & 0 & 1 \end{pmatrix}^T$. W rezultacie uzyskiwane są dwie składowe gradientu, dzięki którym można wyznaczyć jego amplitudę oraz orientację.

Dla obrazu w skali szarości amplituda gradientu jest obliczana na podstawie wzoru (9.1):

$$m(x, y) = \sqrt{G^x(x, y)^2 + G^y(x, y)^2} \quad (9.1)$$

¹Dalal, Navneet, and Bill Triggs. "Histograms of oriented gradients for human detection." 2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05). Vol. 1. IEEE, 2005.

gdzie:

- (x, y) – współrzędne piksela,
- $G^x(x, y)$ – składowa pozioma gradientu,
- $G^y(x, y)$ – składowa pionowa gradientu,
- $m(x, y)$ – amplituda gradientu.

W przypadku przetwarzania obrazu kolorowego wyznaczana jest amplituda gradientu dla każdej składowej przestrzeni barw RGB, a następnie wybierana najwyższa wartość – wzór (9.2).

$$m(x, y) = \max(m_R(x, y), m_G(x, y), m_B(x, y)) \quad (9.2)$$

gdzie:

$m_R(x, y), m_G(x, y), m_B(x, y)$ – amplituda gradientu dla każdej składowej RGB.

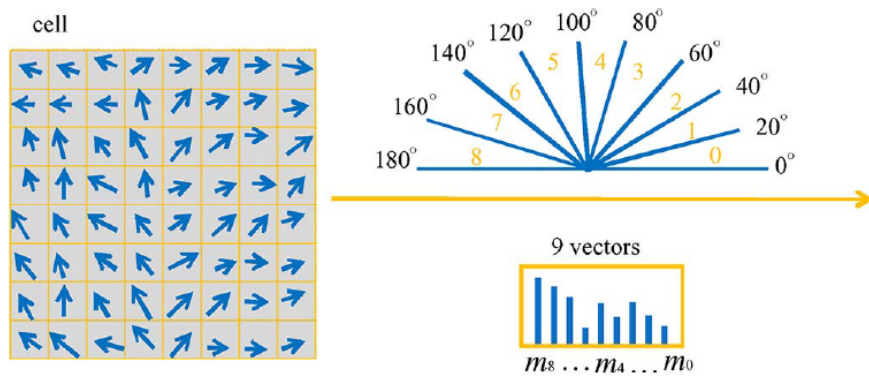
Orientacja gradientu obliczana jest na podstawie wzoru (9.3). W sytuacji, gdy okno detektora zawiera piksele w przestrzeni barw RGB, w każdej komórce kąt gradientu jest wyznaczany jedynie dla składowych gradientu, dla których amplituda jest największa.

$$\theta(x, y) = \arctan\left(\frac{G_{max}^y(x, y)}{G_{max}^x(x, y)}\right) \quad (9.3)$$

gdzie:

- $\theta(x, y)$ – orientacja gradientu,
- $G_{max}^y(x, y), G_{max}^x(x, y)$ – składowe gradientu dla największej amplitudy wśród gradientów poszczególnych kanałów RGB.

Wyznaczanie histogramu



Rysunek 9.1: Widok jednej komórki 8×8 pikseli w ramce detekcji. Rozłożenie przedziałów histogramu w zakresie 0° do 180° oraz przykładowy histogram.

Okno detektora jest podzielone na komórki (ang. *cell*) nienachodzące na siebie. Najchętniej stosowanym rozmiarem komórek jest 8×8 pikseli ze względu na łatwą implementację sprzętową. Dla każdej komórki wyznaczany jest histogram składający się z równomiernie rozłożonych przedziałów, reprezentujących orientacje gradientu w zakresie od 0°

do 180° lub 0° do 360° . Na rys. 9.1 podano przykład histogramu najczęściej stosowanego w detekcji pieszego. W histogramie wartości amplitudy gradientów akumulowane są na podstawie kierunku i czasem też zwrotu w poszczególnych przedziałach. Przynależność poszczególnego gradientu jest dokładnie wyznaczana za pomocą równania (9.3). Aby zniwelować aliasing, czyli różne przyporządkowanie wartości gradientu do dwóch sąsiednich przedziałów przy niewielkiej zmianie orientacji gradientu, zastosowano interpolację liniową. W konsekwencji amplituda gradientu jest liniowo przydzielana do sąsiednich przedziałów wg wzorów (9.4) i (9.5).

$$v_{UB}(x, y) = m(x, y) \left(\frac{\theta(x, y) - \theta_{LB}}{BW} \right) \quad (9.4)$$

$$v_{LB}(x, y) = m(x, y) \left(\frac{\theta_{UB} - \theta(x, y)}{BW} \right) \quad (9.5)$$

gdzie:

$v_{UB}(x, y), v_{LB}(x, y)$ – wartość amplitudy trafiająca do jednego z dwóch sąsiadujących, wcześniej ustalonych przedziałów histogramu,

BW – szerokość przedziału histogramu,

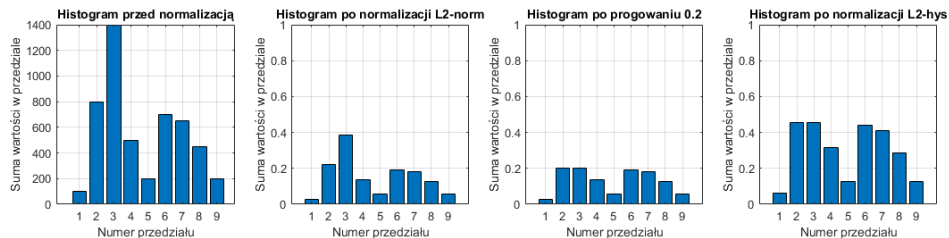
$m(x, y)$ – amplituda gradientu,

$\theta(x, y)$ – orientacja gradientu,

θ_{LB} – środek dolnego przedziału histogramu,

θ_{UB} – środek górnego przedziału histogramu.

Normalizacja histogramów i wyznaczanie wektora cech



Rysunek 9.2: Przykładowy przebieg normalizacji histogramu.

Norma gradientu w obrębie okna detekcji może znacząco się różnić. Z tego powodu wprowadza się lokalną normalizację histogramów. Każdy blok składa się z czterech sąsiadujących komórek w formie 2×2 . Obszary kolejnych bloków pokrywają się w 50% w obu kierunkach (pionowym i poziomym). Dzięki temu krawędzie znajdujące się blisko brzegów komórek zostają tak samo uwzględnione, jak te w centralnej części. Powstałe histogramy są grupowane w 36 elementowy wektor w obrębie jednego bloku. W celu redukcji wpływu nierównomiernego oświetlenia w ramce detekcji zastosowano podwójną normalizację: L2 (wzór (9.6)) i L2-hys (wzór (9.7)). Na rys. 9.2 przedstawiono przykładowy przebieg.

W pierwszym etapie wektor cech sprowadzany jest do wartości z przedziału $[0; 1]$. Dzięki temu zniwelowany jest różnorodny kontrast występujący na obrazie. Na obrazie mogą pojawić się dodatkowe zakłócenia w postaci zmiennego oświetlenia spowodowane odwzorowaniem intensywności kolorów przez kamerę oraz odbiciami od różnego typu powierzchni. W efekcie amplitudy niektórych fragmentów obrazu są znacząco większe niż

w innych częściach. Aby zredukować wpływ tych obszarów na kształt całego histogramu stosuje się górne progowanie. Wartości wektora powyżej 0.2 są obcinane i wyrównywane do tej liczby. Następnie wektor jest ponownie normalizowany do przedziału $[0; 1]$. W efekcie większy wpływ ma orientacja gradientu niż jego amplituda. Wartość progu została wyznaczona doświadczalnie w pracy D. Lowe².

- norma L2:

$$f_1 = \frac{f}{\sqrt{\|f\|_2^2 + \epsilon^2}} \quad (9.6)$$

- norma L2-hys:

$$f_2 = \frac{f_{th}}{\sqrt{\|f_{th}\|_2^2 + \epsilon^2}}, f_{th} = \min(f_1, 0.2) \quad (9.7)$$

gdzie:

- f – nieznormalizowany wektor cech,
- f_1 – znormalizowany wektor cech normą L2,
- f_2 – znormalizowany wektor cech normą L2-hys,
- f_{th} – wektor cech po przeprowadzeniu progowania,
- $\|f\|_k$ – k-ta norma wektora f ,
- ϵ – odpowiednia stała (uniknięcie dzielenia przez 0).

Finalnie wektory powstałe w każdym bloku są łączące w jeden jednowymiarowy wektor, który osiąga rozmiar $7 \times 15 \times 36 = 3780$ wartości. Następnie przekazywany jest on do klasyfikatora w celu przypisania obiektu do odpowiedniej klasy.

9.3 Support Vector Machines – opis algorytmu

Celem algorytmu SVM jest znalezienie (hiper)płaszczyzny w wielowymiarowej przestrzeni zdefiniowanej na podstawie cech, pochodzących z deskryptora, która jest maksymalnie oddalona od wszystkich punktów obu klas. Jest ona opisana poniższym wzorem (9.8) oraz przedstawiona na przykładzie na rys. 9.3.

$$f(x) = w^T x + b = 0 \quad (9.8)$$

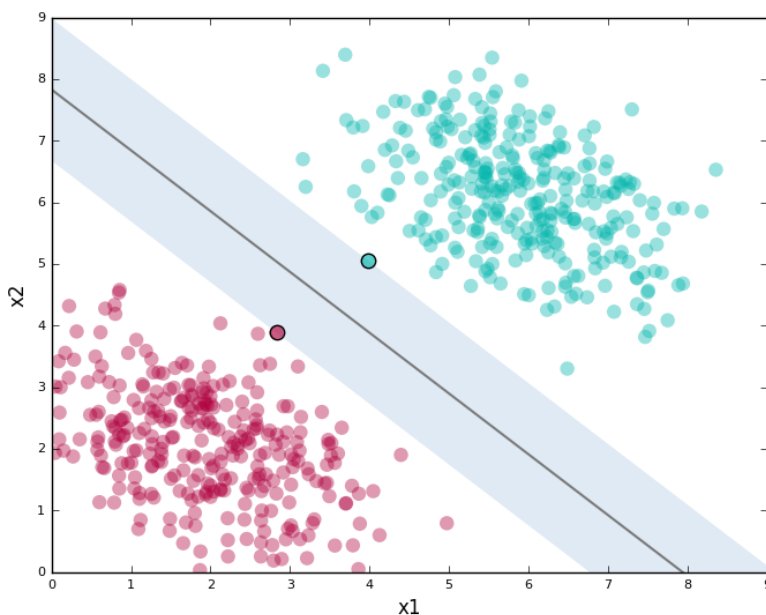
gdzie:

- x – wektor cech,
- w – wektor wag, parametry hiperpłaszczyzny,
- b – bias.

Wyróżnia się dwie wersje algorytmu SVM:

- *soft-margin* – klasyfikator z miękkim marginesem, który pozwala na błędną klasyfikację części danych,
- *hard-margin* – klasyfikator jednoznacznie rozdzielający dane pochodzące z dwóch klas.

²David G. Lowe. „Distinctive image features from scale-invariant keypoints”. W: International journal of computer vision 60.2 (2004), s. 91–110.



Rysunek 9.3: Przedstawienie hiperpłaszczyzny jednowymiarowej oddzielającej próbki dwóch klas dla klasyfikatora z twardym marginesem.

Ze względu na pojawiające się niedokładności obliczeń oraz deformacje na obrazie zbiory cech zwykle nie są jednak separowalne liniowo. Z tego powodu wprowadza się do zadania optymalizującego funkcję kary ze stałym współczynnikiem C , która pozwala na błędne zakwalifikowanie pewnej grupy punktów. Umożliwia to znalezienie płaszczyzny, która rozdzieli znacząco różniące się cechy obiektu, a pozostałe punkty, bliskie granicy podziału, nie będą mieć wpływu na jej położenie. Wprowadzenie osłabionego klasyfikatora lub innymi słowami klasyfikatora z miękkim marginesem posiada dodatkową zaletę. Algorytm ten nie jest wrażliwy na dane oddalone od punktu najgęstszej skupienia (ang. *outliers*), a z tego wynika, że lepiej generalizuje problem i jest odporny na nadmierne dopasowanie (ang. *overfitting*). Istotną kwestią jest znalezienie takiego współczynnika C , aby uzyskać jak najmniejszy odsetek błędnie zakwalifikowanych próbek testowych. Najczęściej do wyznaczenia najlepszej wartości stosuje się walidację krzyżową (ang. *cross-validation*). Im parametr jest większy, tym margines klasyfikatora jest węższy. Na rys. 9.4 przedstawiono zależność wartości współczynnika C od szerokości marginesu klasyfikatora.

Dzięki rozwiązaniu równania (9.9), otrzymywana jest hiperpłaszczyzna, która najlepiej uogólnia problem klasyfikacji pieszych. Zbiór okien detekcji, pochodzących z jednej ramek obrazu, jest wyrażony przez zbiór par: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, gdzie x_i jest wektorem cech opisującym okno detekcji, a $y_i \in \{-1, 1\}$ są etykietami klas (nie-pieszy, pieszy).

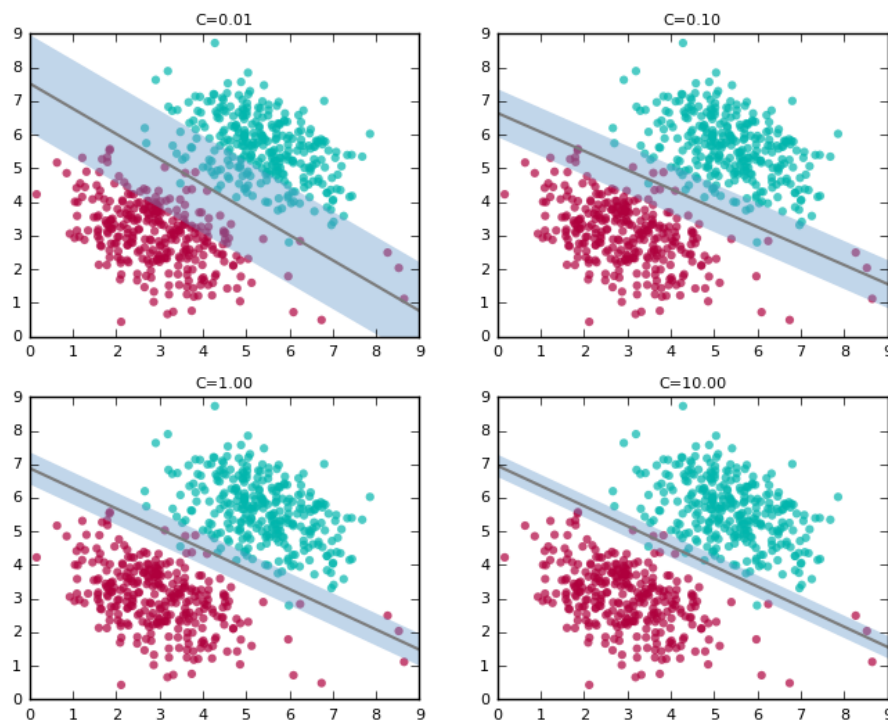
$$\min_{w, b, \xi} \left(\frac{1}{2} w^T w + C \sum_j \xi_j \right) \quad (9.9)$$

przy założeniu, że:

$$(w^T x_i + b) y_i \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad (i = 1, \dots, n) \quad (9.10)$$

gdzie:

w – wektor wag, parametry hiperpłaszczyzny,



Rysunek 9.4: Zależność parametru C w funkcji kary od szerokości marginesu.

C – stały współczynnik przy funkcji kary,
 ξ_i – zmienna swobodna (ang. *slack variable*), odległość punktu (x_i, y_i) od marginesu, położonego po odpowiedniej stronie hiperpłaszczyzny.

Parametr ξ_i zmniejsza margines separacji próbek. Wyróżnia się trzy przedziały, które określają pozycję danej próbki względem hiperpłaszczyzny. Jeśli spełniony jest warunek $0 \leq \xi_i < 1$ to punkt (x_i, y_i) leży po właściwej stronie hiperpłaszczyzny, klasyfikacja będzie prawidłowa. Natomiast w przypadku $1 < \xi_i$ to punkt (x_i, y_i) położony jest po niewłaściwej stronie, więc błędnie zostanie określona przynależność do klasy. Jeśli wartość ξ_i równa jest 1 to nie jest możliwe prawidłowe przyporządkowanie etykiety do próbki.



Warto jeszcze raz nadmienić, że powyższe operacje wykonywane są cyklicznie dla każdej ramki obrazu (64×128 px) z wykorzystaniem metody *sliding window*. Lista wszystkich parametrów algorytmu *HOG+SVM* przedstawia się następująco:

- bins* – liczba przedziałów histogramu (zwykle 9, co 20°),
- cell* – liczba pikseli w komórce (zwykle 8×8 px),
- block* – liczba komórek w bloku (zwykle 2×2 komórki),
- overlap* – liczba komórek wspólnych dla dwóch sąsiednich bloków lub procent nakładania się bloków (2 komórki, 50%),
- frame_width* – rozmiar ramki detekcji (zwykle 64×128 px),
- svm_weights* – liczba wag klasyfikatora SVM (3780 dla 64×128 px),
- bias_weight* – wartość biasu klasyfikatora SVM,
- stride* – liczba pikseli co ile przesuwana jest ramka detekcji (zwykle 8 lub wielokrotność dla 64×128 px),

9.4 Deskryptor HOG

W dużym uproszczeniu algorytm do detekcji cech można podzielić na trzy etapy:

- wyliczenie gradientów
- wyznaczenie histogramów gradientów
- normalizacja histogramów

Na wstępie proszę zwrócić uwagę na parametry algorytmu:

- przestrzeń barw – RGB,
- brak korekcji gamma (ani innego przetwarzania wstępnego),
- postać operatora gradientowego,
- liczbę przedziałów histogramu, analizowane orientacje,
- rozmiar komórki (ang. *cell*),
- rodzaj normy – zamiast *norm L2-hys*, będziemy używać po prostu *norm L2*,
- rozmiar bloku (ang. *block*), nachodzenie na komórki (ang. *stride*),
- rozmiar okna detekcji.

Ćwiczenie 9.1 Celem ćwiczenia jest implementacja deskryptora HOG.

1. Ze strony kursu pobierz zbiór wycinków obrazu z sylwetkami ludzi (próbki pozytywne) oraz bez ludzi (próbki negatywne).
2. Wczytaj dowolny obraz pozytywny – dla niego opracujemy implementację HOG. Można wspomniane wyżej trzy etapy zaimplementować w osobnych funkcjach.
3. Zaczynamy od obliczania gradientu. Należy zrealizować detekcję krawędzi pionowych i poziomych oraz policzyć amplitudę i orientację (w stopniach) w każdym kanale RGB osobno. Najlepiej przystosować funkcję zarówno do obrazu w skali szarości jak i w RGB. Do obliczenia gradientów z maską (z sekcji 9.2) można użyć funkcji:

```
dx = scipy.ndimage.filters.convolve1d(np.int32(im), np.array([-1, 0, 1]), 1)
dy = scipy.ndimage.filters.convolve1d(np.int32(im), np.array([-1, 0, 1]), 0)
```

Po obliczeniu gradientów dla trzech składowych należy wybrać z nich te, których wartość jest największa. Można to robić pętlą, ale będzie to nieefektywne. Lepiej wykorzystać możliwości Pythona, w której z tablicy wybierane są wartości spełniające warunek. Przykładowo wykorzystać funkcję `np.logical_and()`:

```
max_B = np.logical_and(magnitude[:, :, 1] < magnitude[:, :, 0], magnitude
[:, :, 2] < magnitude[:, :, 0])
```

Następnie analogicznie znajdujemy największe gradienty z pozostałych składowych. Wybierając największe gradienty z poszczególnych składowych otrzymamy tablicę zawierającą maksymalne wartości gradientów. Ten sam mechanizm należy wykorzystać w wyznaczaniu macierzy orientacji gradientów.

4. Drugi etap – wyznaczanie histogramów w blokach to najtrudniejszy element implementacji (technicznie, bo koncepcyjnie dość prosty). Na początku trzeba zdefiniować rozmiar komórki (8) oraz wyznaczyć liczbę komórek w osi X i Y (`np.YY_cell= np.int32(YY/cellSize)`). Następnie tworzymy kontener na histogramy – jeden histogram dla każdej z komórek. W pętli po komórkach realizujemy następujące operacje:
 - Pobieramy wartości dla odpowiedniej komórki z macierzy z amplitudą i orientacją.

- Po pierwsze trzeba obsłużyć przypadek ujemnych kątów (poprzez dodanie 180, jeśli rozpatrujemy wartości w stopniach).
 - Po drugie trzeba określić do jakiego przedziału histogramu należy rozważany kąt. W tym celu przedział $[0;180]$ dzielimy na 9 części i dla każdego z nich wyznaczamy element środkowy. Następnie należy wyznaczyć do którego przedziału należy każdy rozważany gradient oraz do której części przedziału, np. dla przedziału $0 - 20^\circ$, elementem środkowym jest 10. Więc gradient o orientacji 11° należy do przedziału $0 - 20^\circ$ oraz podprzedziału $10 - 20^\circ$. Jest to istotne ze względu na konieczne wykorzystanie interpolacji biliniowej pomiędzy dwoma sąsiadującymi przedziałami. **Proszę pamiętać** o specjalnym traktowaniu przejścia $180^\circ - 0^\circ$ (zawijanie).
 - Prawidłowym rezultatem na tym etapie jest otrzymanie odpowiedniego numeru przedziału (najlepiej przyjąć, że będzie to dolny przedział). Innym rozwiązaniem jest przechowywanie numerów przedziałów dla dolnego i górnego przedziału.
 - Następnie trzeba obliczyć odległość od rozważanego kąta do środka przedziałów według wzorów (9.5) i (9.4). Tu również należy pamiętać o specjalnym traktowaniu wartości zbliżonych do 180° .
 - Finalnie dodajemy wartości amplitudy z komórki do odpowiednich przedziałów histogramu.
5. Ostatni etap to normalizacja komórek w blokach. W pętli po współrzędnych komórek (rozmiar pomniejszony o 1) należy pobrać 4 histogramy należące do jednego bloku. Następnie należy je zestawić w jeden wektor (`np.concatenate`), obliczyć normę (funkcja `np.linalg.norm`) i dokonać normalizacji z zastosowaniem L2. Znormalizowany wektor stanowi element finalnego wektora cech.

```
# Normalisation in block
e = math.pow(0.00001,2)
F = []
for jj in range(0,YY_cell-1):
    for ii in range(0,XX_cell-1):
        H0 = hist[jj,ii,:]
        H1 = hist[jj,ii+1,:]
        H2 = hist[jj+1,ii,:]
        H3 = hist[jj+1,ii+1,:]
        H = np.concatenate((H0, H1, H2, H3))
        n = np.linalg.norm(H)
        Hn = H/np.sqrt(math.pow(n,2)+e)
        F = np.concatenate((F,Hn))
```

Na końcu powinniśmy otrzymać wektor 3780 liczb. Pierwsze 10 wartości dla obrazka *per00060.ppm*:

```
0.34384495
0.15958051
0.10338961
0.13061814
0.13671541
0.09076598
0.09694051
0.19526622
0.19388454
0.16014419
```

Mogą też Państwo wykorzystać poniższy kod do wyświetlenia gradientów histogramów (przed ich normalizacją).

```
def HOGpicture(w, bs): # w - histograms, bs - cell size (8)
    bim1 = np.zeros((bs, bs))
    bim1[np.round(bs//2):np.round(bs//2)+1,:]= 1;
    bim = np.zeros(bim1.shape+(9,));
    bim[:, :, 0] = bim1;
    for i in range(0,9):
        bim[:, :, i] = scipy.misc.imrotate(bim1, -i*20, 'nearest')/255

    Y,X,Z = w.shape
    w[w < 0] = 0;
    im = np.zeros((bs*Y, bs*X));
    for i in range(Y):
        iisl = (i)*bs
        iisu = (i+1)*bs
        for j in range(X):
            jjsl = j*bs
            jjsu=(j+1)*bs
            for k in range(9):
                im[iisl:iisu,jjsl:jjsu] += bim[:, :, k] * w[i,j,k];
    return im
```

Finalnie proszę utworzony kod (wygodny na etapie implementacji) przekształcić w funkcję, która będzie pobierać obrazek (macierz), a zwróci wyznaczony wektor cech (znormalizowane histogramy gradientów).



9.5 Klasyfikator SVM

Wyznaczone wektory cech za pomocą deskryptora HOG należy następnie odpowiednio sklasyfikować w celu oceny czy w analizowanej ramce obrazu znajduje się obiekt (pieszy) czy też nie.

Ćwiczenie 9.2 Celem tego ćwiczenia jest implementacja klasyfikatora SVM i jego uczenia.

1. Zaczynamy od przygotowania zbioru danych. Ze strony kursu ściągnij folder **pedestrians**. Zawiera on próbki pozytywne (z pieszymi) i negatywne (bez pieszych).
2. Dla obrazów z obu zbiorów wyliczmy deskryptor HOG (dla pierwszych 100).

```
HOG_data = np.zeros([2*100,3781],np.float32)

for i in range(0,100):
    IP = cv2.imread('pos/per%05d.ppm' % (i+1))
    IN = cv2.imread('neg/neg%05d.png' % (i+1))
    F = hog(IP)
    HOG_data[i,0] = 1;
    HOG_data[i,1:] = F;
    F = hog(IN)
    HOG_data[i+100,0] = 0;
    HOG_data[i+100,1:] = F;
```

Dane zawierają oprócz wektora indeks klasy (1 – pieszy, 0 – nie pieszy).

3. Przed przystąpieniem do uczenia rozdzielamy danej wejściowe na wektor etykiet (pierwsza kolumna) i dane.

```
labels = HOG_data[:,0];
```

```
data = HOG_data[:,1:]
```

Do pliku dołączamy moduł `from sklearn import svm`.

Tworzymy obiekt liniowy SVM: `clf = svm.SVC(kernel='linear', C = 1.0)`.

Przeprowadzamy uczenie: `clf.fit(data, labels)`.

Testujemy: `lp = clf.predict(data)`.

4. Analizujemy wyniki uczenia. Na zbiorze uczącym przeprowadzamy klasyfikację. Wyznaczamy macierz pomyłek (ang. confusion matrix), licząc współczynniki TP, TN, FP, FN. Wyniki dobrego uczenia powinny być zbliżone do 100 %.
5. Przeprowadzamy walidację rozwiązania. Zasadniczo powtarzamy powyższy eksperyment. To jest moment na wprowadzanie ewentualnych poprawek:
 - zmiana parametru C,
 - zmiana jądra itp.

Oczywiście po wprowadzaniu zmian każdorazowo należy sprawdzić ich efekt na zbiorach: testowym i walidacyjnym.

6. Ostatecznie, jak już jesteśmy pewni, że dana wersja klasyfikatora osiąga najlepsze wyniki, to przeprowadzamy test na zbiorze testowy. Jego wyniki uznajemy za “ostateczny” dla klasyfikatora. Tak przynajmniej jest poprawnie metodologicznie.

■

9.6 Detekcja sylwetki ludzkiej

W tym momencie mamy już wszystkie narzędzia do realizacji kompletnego systemu detekcji: deskryptor HOG i nauczony klasyfikator SVM. Możemy zatem zaimplementować detekcję na obrazie rzeczywistym.

Ćwiczenie 9.3 Detekcja sylwetki pieszego przy użyciu deskryptor HOG i nauczonego klasyfikatora SVM.

1. Na stronie kursu zamieszczone są cztery obrazy testowe (`testImage1-4.png`). Ale równie dobrze można użyć dowolnych obrazów.
2. Na początek warto sprawdzić, czy wysokość postaci na zdjęciu mniej więcej odpowiada rozmiarowi naszego okna detekcji. Można to zrobić ręcznie (zmierzyć wysokość w pikselach) lub automatycznie (nanieść okno detekcji na obraz). Druga metoda jest o tyle lepsza, że i tak będziemy potrzebować funkcji rysowania prostokąta na obrazie (do wizualizacji detekcji). Dla przypomnienia `cv2.rectangle`.
3. Następnie musimy dostosować rozmiar obrazka (funkcja `cv2.resize`) do wysokości postaci. Warto sobie przeglądać jak wyglądają sylwetki w zbiorze uczącym – pozwoli nam to zorientować się “jak mógł się nauczyć klasyfikator”.
4. Implementujemy pętlę po obrazie. W każdej iteracji pobieramy wycinek o rozmiarze 64×128 , obliczamy dla niego deskryptor HOG, a następnie dokonujemy klasyfikacji. Jeśli jest ona pozytywna to nanosimy prostokąt na obraz (oczywiście pomocniczy, żeby nie zakłócić detekcji w kolejnych lokalizacjach, kopię robimy za pomocą metody `copy`).

R Krok (czyli to o ile przesuwamy okno w pionie i poziomie) nie powinien wynosić 1, bo możemy nie doczekać się na wynik. Najlepiej dobrać go empirycznie, ale wartości 8 lub 16 wydają się odpowiednie.

5. Spróbuj, czy wykrywanie działa na załączonych obrazkach, ewentualnie na własnych.

6. Nieobowiązkowe. Przetwarzanie w wielu skalach.

Poprzednio dobieraliśmy odpowiednią skalę empirycznie/doświadczalnie (manualnie). Teraz przetwarzamy obraz w wielu skalach. Przykładowo zaczynamy od rozdzielczości 1 i każdorazowo zmniejszamy o 10% lub współczynnik skalowania ustawiamy na 1.2 albo 1.5. Sposób budowy piramidy należy dobrać empirycznie. W każdej skali realizujemy detekcję.

R Należy zdawać sobie sprawę, że można budować piramidę także “w górę” tj. zwiększać rozdzielczość obrazu wejściowego. Pozwala to na detekcję małych sylwetek (aczkolwiek skuteczność takiego podejścia nie jest bardzo duża).

R Do testów proszę dobrać obraz o względnie małych rozmiarach na którym występują dwie różniące się rozmiarami sylwetki. Przykładowo `testImage4.png`. Proszę jednak nie oczekiwać zupełnie bezbłędnego działania systemu. Przede wszystkim dlatego, że zbiór próbek negatywnych nie był zbyt obszerny.

7. Na koniec należy rozwiązać kwestię wizualizacji i ew. integracji detekcji. Najlepiej zapamiętać współrzędne okien, gdzie wykryto sylwetki, przeskalować je do rozdzielczości podstawowej i wyrysować.
8. Dodatkowo w celu likwidacji wielokrotnych detekcji tej samej sylwetki stosuje się metodę non-maximum suppression (NMS).



10

Mini projekt 2

10	Projekt 2	97
10.1	Cel projektu	
10.2	Przykładowa literatura	

10. Projekt 2

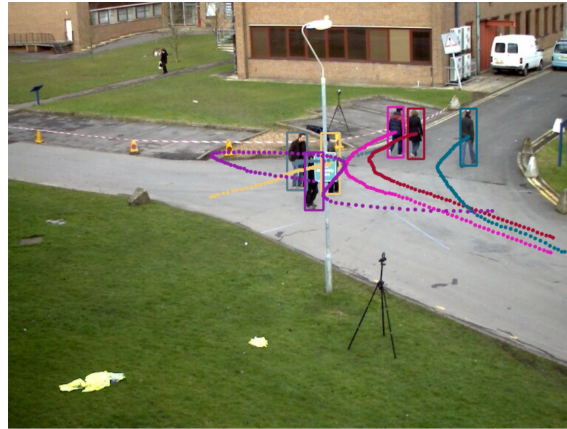
Śledzenie wielu obiektów (MOT - ang. Multiple Object Tracking) jest podstawowym zadaniem w systemach percepcji opartych na wizji komputerowej. Umożliwia pozyskanie informacji o trajektorii obiektów znajdujących się na scenie, co pozwala na unikanie kolizji lub namierzanie konkretnego celu – rys. 10.1. Wykorzystywane jest między innymi w pojazdach autonomicznych, zaawansowanym monitoringu wizyjnym i systemach celowania.

Ponieważ nie jest z góry znane, ile będzie śledzonych obiektów, ani kiedy one się pojawiają, zadanie śledzenia obiektów wymaga pewnego systemu detekcji. Zadanie sprowadza się wówczas do zdecydowania, którym z dotychczas zaobserwowanych obiektów jest każda z uzyskanych detekcji. System śledzenia powinien jednakże rozważyć szereg scenariuszy, które mogą wystąpić w tak zdefiniowanym zadaniu, są to między innymi:

- Detekcja może nie być żadnym ze znanych obiektów ponieważ nowy obiekt właśnie się pojawił na scenie.
- Detekcja może być błędem detektora (*false positive*).
- Obiekt może ciągle znajdować się na scenie i być widoczny, ale nie został w danej klatce wykryty przez detektor (*false negative*).
- Jeden obiekt może być przesłonięty przez drugi i jedna detekcja może pokrywać się z pozycją dwóch obiektów. System śledzenia może wówczas pomylić tożsamości dwóch obiektów (*identity switch*).

10.1 Cel projektu

Celem projektu jest opracowanie algorytmu śledzenia wielu obiektów w oparciu o predefiniowany zbiór detekcji. Dzięki temu skoncentrować się można na samym zadaniu śledzenia i uniezależnić wpływ detektora na jakość całego systemu w trakcie ewaluacji skuteczności. Jedną z najpopularniejszych metod ewaluacji jakości śledzenia jest metryka MOTA (ang. Multiple Object Tracking Accuracy).



Rysunek 10.1: Zadanie śledzenia wielu obiektów.

$$MOTA = 1 - \frac{\sum_t (FN_t + FP_t + IDSW_t)}{\sum_t g_t} \quad (10.1)$$

gdzie t - klatki sekwencji testowej, FN_t - liczba *false positivów*, FP_t - liczba *false negativów*, $IDSW_t$ - liczba *identity switchy*, g_t - rzeczywista liczba obiektów na klatce t .

W ramach projektu należy:

- zinterpretować i zwizualizować zbiór danych.
- zaproponować i zaimplementować algorytm śledzenia wielu obiektów. Należy zastanowić się nad podstawowymi zadaniami: inicjalizacją nowego obiektu, przypisaniem nowych detekcji do istniejących obiektów (na podstawie pozycji, wyglądu), usuwaniem obiektów.
- przeprowadzić ewaluację skuteczności śledzenia.

Zbiór danych dostępny jest pod adresem: [link](#). W celu ewaluacji warto skorzystać z oficjalnego narzędzia wykorzystywanego we wszelkich konkursach związanych ze śledzeniem wielu obiektów: [TrackEval - github](#). Udostępniony zbiór jest już w kompatybilnym formacie.

10.2 Przykładowa literatura

- Bewley, A., et al.: Simple online and realtime tracking.
- Zhang, Y., et al.: Bytetrack: Multi-object tracking by associating every detection box.
- Aharon, N., et al.: Bot-sort: Robust associations multi-pedestrian tracking.
- Bernardin, K., Stiefelhagen, R.: Evaluating multiple object tracking performance: The clear mot metrics.

11

Laboratorium 11

11	Kalibracja kamery i stereowizja	103
11.1	Kalibracja pojedynczej kamery	
11.2	Kalibracja układu kamer	
11.3	Obliczenia korespondencji stereo	
11.4	Wykorzystanie sieci neuronowej do realizacji zadania analizy głębi obrazu	

11. Kalibracja kamery i stereowizja

Kamera to urządzenie, które przekształca świat trójwymiarowy w obrazy dwuwymiarowe. Zależność tę można opisać następującym równaniem:

$$x = PX \quad (11.1)$$

gdzie: x oznacza obraz 2D, P oznacza macierz współczynników projekcji kamery, X oznacza punkty 3D.

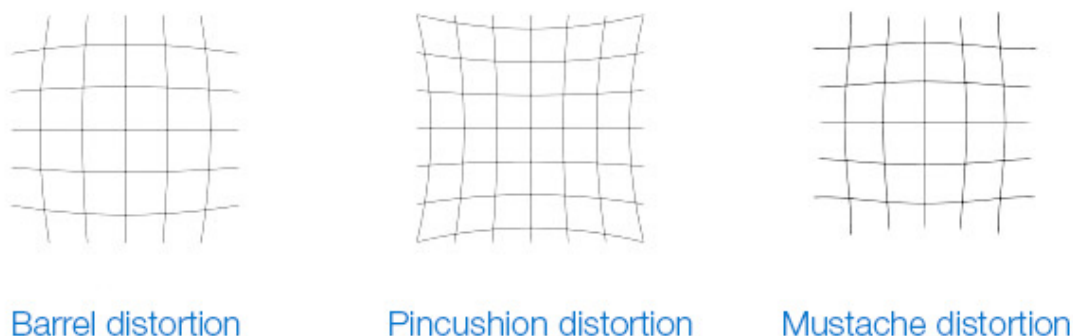
Algorytmy liniowe lub nieliniowe są stosowane do estymacji parametrów wewnętrznych i zewnętrznych z wykorzystaniem znanych punktów w czasie rzeczywistym i ich projekcji na płaszczyźnie obrazu.

Kalibracja kamery ma na celu eliminację zniekształceń (dystorsji) wprowadzanych przez układ optyczny. Zalicza się do nich zniekształcenia: radialne (beczkowe, poduszkowe, faliste – związane z obiektywem) oraz tangensoidalne (związane z nierównoległym montażem soczewki i matrycy – rys. 11.1 oraz rys. 11.2. Szczegółowe omówienie zagadnienia – [Distortion \(optics\)](#) i [Camera Calibration MathWorks](#).

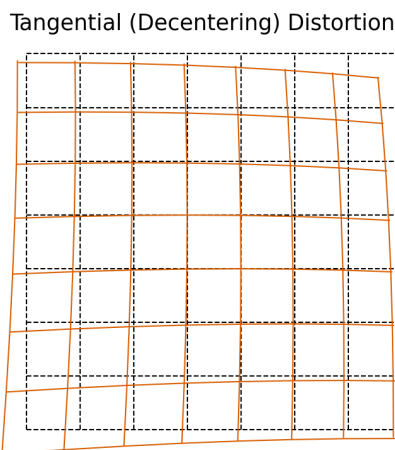
Korekcja zniekształceń wymaga wyliczenia pewnych współczynników. Jest to możliwe w procedurze kalibracji, gdzie rejestruje się obraz wzorca o znanych parametrach (wzajemne odległości między punktami i ich rozmiary “rzeczywiste”) w różnych położeniach względem kamery. Najczęściej stosuje się wzorzec w postaci szachownicy, na którym łatwo wykryć narożniki, nawet z dokładnością sub-pikselową (tak jak na analizowanym obrazie). Praktyka wskazuje, że obrazów powinno być co najmniej 10.

Parametry wyznaczane w procedurze kalibracji kamery dzieli się na dwie grupy:

- parametry wewnętrzne (ang. *Intrinsic*, *Internal*) – umożliwiają odwzorowanie współrzędnych piksela i współrzędnych kamery w ramce obrazu, np. środek optyczny, ogniskowa i współczynniki zniekształceń radialnych obiektywu.
- parametry zewnętrzne (ang. *Extrinsic*, *External*) – opisują orientację i położenie kamery. Odnoszą się do obrotu i przesunięcia kamery w stosunku do pewnego zewnętrznego (względem kamery) układu współrzędnych.



Rysunek 11.1: Rodzaje zniekształceń: (a) beczkowe, (b) poduszkowe (c) faliste.



Rysunek 11.2: Zniekształcenie tangensoidalne. Źródło: <https://www.tangramvision.com/blog/camera-modeling-exploring-distortion-and-distortion-models-part-i>.

Na rysunku 11.3 przedstawiono równanie modelu kamery, która zawiera współrzędne homogeniczne (jednorodne) oraz macierz projekcji (ang. *camera projection matrix*). Macierz kalibracji to iloczyn macierzy zawierających wewnętrzne oraz zewnętrzne parametry kamery. Do estymacji parametrów wewnętrznych i zewnętrznych wykorzystuje się algorytm liniowy lub nieliniowy. Wykorzystuje się znajomość parametrów rzeczywistych (odległość pomiędzy punktami, w centymetrach) oraz ich rzutów na płaszczyznę (w pikselach).

Wyróżnia się dwa modele kamery:

- *pinhole camera model* – podstawowy model kamery bez soczewki, szczegóły w dokumentacji OpenCV: [Pinhole model](#),
- *fisheye camera model* – wykorzystywany przede wszystkim w sytuacjach, gdzie zniekształcenia są ekstremalne, dobrze modeluje kamerę szerokokątne, szczegóły w dokumentacji OpenCV: [Fisheye model](#).

Przeprowadzenie prawidłowej korekcji zniekształceń – kalibracji kamery – jest niezbędne, jeśli chcemy dokonać pomiaru rozmiarów obiektów np. w centymetrach, zmierzyć

$$\begin{array}{c}
 \begin{array}{l} \text{Intrinsic Parameters (fundamental} \\ \text{characteristics of the camera)} \end{array} \\
 \begin{array}{c} \left[\begin{array}{ccc} \frac{1}{\rho_u} & 0 & u_0 \\ 0 & \frac{1}{\rho_v} & v_0 \\ 0 & 0 & 1 \end{array} \right] \left[\begin{array}{cccc} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{array} \right] \end{array} \\
 \text{Camera Matrix} \\
 \begin{array}{c} \text{Extrinsic Parameters (depend on} \\ \text{where camera is)} \\
 \left[\begin{array}{cc} \textcircled{R} & \textcircled{t}^{-1} \\ \text{0}_{1 \times 3} & 1 \end{array} \right] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \\
 \begin{array}{l} \text{Rotation Matrix} \\ \text{(orientation of camera)} \end{array} \\
 \text{Position of camera}
 \end{array}
 \end{array}
 \begin{array}{c}
 \left[\begin{array}{c} u' \\ v' \\ w' \end{array} \right] =
 \end{array}$$

Rysunek 11.3: Równanie modelu kamery bazujące na ogólnym wzorze 11.1. ρ_u, ρ_v to rozmiar obrazu x, y w pikselach, u_0, v_0 to środek optyczny, f to ogniskowa. Źródło: <https://towardsdatascience.com/understanding-transformations-in-computer-vision-b001f49a9e61>

odległość lub określić przemieszczenie kamery.

11.1 Kalibracja pojedynczej kamery

Procedura kalibracji kamery jest podzielona na etapy. Niemal identyczne kroki wykonuje się w przypadku kalibracji pojedynczej kamery lub kamer stereowizyjnych. Wykorzystany zostanie zbiór danych, zawierający pary obrazów pochodzących z kamery stereowizyjnej. Do kalibracji pojedynczej kamery należy wybrać zbiór z jednego obiektywu (odpowiednio lewe lub prawe obrazy). Zaproponowany zbiór został zarejestrowany przez kamerę, która wymaga zamodelowania przez model *fisheye* (tzw. rybie oko).

Ogólnie proces kalibracji kamery można podzielić na:

1. Wybranie odpowiedniego wzoru kalibracyjnego. Do najczęściej stosowanych zalicza się: szachownicę, chArUco (połączenie szachownicy oraz znaczników ArUco), macierz kółek, macierz asymetrycznych kółek.
2. Wykonanie zdjęć planszy kalibracyjnej pod różnymi kątami oraz z różnej odległości. Liczba prawidłowo wykonanych obrazów powinna być większa niż 10.
3. Przygotowanie parametrów planszy kalibracyjnej, np. zmierzenie rozmiaru pól szachownicy w centymetrach.
4. Przygotowanie oraz uruchomienie algorytmu do kalibracji.

Poniżej opisano etapy algorytmu do kalibracji pojedynczej kamery. Wykorzystano do tego wcześniej przygotowany zbiór obrazków (dostępny na platformie UPeL) oraz funkcje z biblioteki OpenCV.

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# termination criteria
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
calibration_flags = cv2.fisheye.CALIB_RECOMPUTE_EXTRINSIC+cv2.fisheye.CALIB_FIX_SKEW

# inner size of chessboard

```

```

width = 9
height = 6
square_size = 0.025 # 0.025 meters

# prepare object points, like (0,0,0), (1,0,0), (2,0,0) ..., (8,6,0)
objp = np.zeros((height * width, 1, 3), np.float64)
objp[:, 0, :2] = np.mgrid[0:width, 0:height].T.reshape(-1, 2)

objp = objp * square_size # Create real world coords. Use your metric.

# Arrays to store object points and image points from all the images.
objpoints = [] # 3d point in real world space
imgpoints = [] # 2d points in image plane.

img_width = 640
img_height = 480
image_size = (img_width, img_height)

path = ""
image_dir = path + "pairs/"

number_of_images = 50
for i in range(1, number_of_images):
    # read image
    img = cv2.imread(image_dir + "left_%02d.png" % i)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Find the chess board corners
    ret, corners = cv2.findChessboardCorners(gray, (width, height), cv2.
        CALIB_CB_ADAPTIVE_THRESH + cv2.CALIB_CB_FAST_CHECK + cv2.
        CALIB_CB_NORMALIZE_IMAGE)

    Y, X, channels = img.shape

    # skip images where the corners of the chessboard are too close to the edges of
    # the image
    if (ret == True):
        minRx = corners[:, :, 0].min()
        maxRx = corners[:, :, 0].max()
        minRy = corners[:, :, 1].min()
        maxRy = corners[:, :, 1].max()

        border_threshold_x = X/12
        border_threshold_y = Y/12

        x_thresh_bad = False
        if (minRx < border_threshold_x):
            x_thresh_bad = True

        y_thresh_bad = False
        if (minRy < border_threshold_y):
            y_thresh_bad = True

        if (y_thresh_bad==True) or (x_thresh_bad==True):
            continue

    # If found, add object points, image points (after refining them)
    if ret == True:
        objpoints.append(objp)

        # improving the location of points (sub-pixel)
        corners2 = cv2.cornerSubPix(gray, corners, (3, 3), (-1, -1), criteria)

        imgpoints.append(corners2)

        # Draw and display the corners
        # Show the image to see if pattern is found ! imshow function.
        cv2.drawChessboardCorners(img, (width, height), corners2, ret)
        cv2.imshow("Corners", img)
        cv2.waitKey(5)

```

```

else:
    print("Chessboard couldn't detected. Image pair: ", i)
    continue

```

Mając wyliczone współrzędne punktów można przystąpić do kalibracji:

```

N_OK = len(objpoints)
K = np.zeros((3, 3))
D = np.zeros((4, 1))
rvecs = [np.zeros((1, 1, 3), dtype=np.float64) for i in range(N_OK)]
tvecs = [np.zeros((1, 1, 3), dtype=np.float64) for i in range(N_OK)]

ret, K, D, _ = \
    cv2.fisheye.calibrate(
        objpoints,
        imgpoints,
        image_size,
        K,
        D,
        rvecs,
        tvecs,
        calibration_flags,
        (cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_MAX_ITER, 30, 1e-6)
    )
# Let's rectify our results
map1, map2 = cv2.fisheye.initUndistortRectifyMap(K, D, np.eye(3), K, image_size,
        cv2.CV_16SC2)

```

R Opis parametrów otrzymanych w procesie kalibracji:

- **ret** – wartość błędów średniokwadratowego projekcji wstecznej (opisuje jakość dopasowania),
- **K** – macierz parametrów wewnętrznych kamery w postaci:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (11.2)$$

gdzie: (f_x, f_y) – to długości ogniskowych, a (c_x, c_y) to współrzędne środka obrazu.

- **D** – współczynniki zniekształceń,
- **rvecs** – wektory rotacji,
- **tvecs** – wektory translacji (wraz z wektorami rotacji umożliwiają przekształcenie położenia wzorca kalibracyjnego z współrzędnych modelu do współrzędnych rzeczywistych)

Po otrzymaniu wszystkich potrzebnych parametrów w procesie kalibracji, należy teraz wybrać dowolny obraz z podanego zbioru oraz usunąć zniekształcenia. Do tego posłuży funkcja `cv2.remap()`:

```

undistorted_image = cv2.remap(image, map1, map2, interpolation=cv2.INTER_LINEAR,
        borderMode=cv2.BORDER_CONSTANT)

```

Ćwiczenie 11.1 Wykorzystując powyższy opis do kalibracji pojedynczej kamery, wykonaj zadanie.

- Wyznacz parametry kamery i wyświetl je.
- Sprawdź korekcję dla jednego z obrazów ze zbioru (test na linię prostą).
- Przeprowadź korekcję zniekształceń dla całego zbioru obrazów. Wyświetl oraz

porównaj obraz przed i po przeprowadzeniu kalibracji.

11.2 Kalibracja układu kamer

W największym uproszczeniu, kalibracja układu kamer (dla uproszczenia dwóch) ma umożliwić łatwe obliczenie map głębi. Zagadnienie to można rozumieć na kilka sposobów:

- ustawienie identycznych parametrów obu kamer,
- eliminację zniekształceń,
- sprowadzenie do wspólnego układu współrzędnych,
- rektyfikację.

Szczególnie interesujące jest ostatnie zagadnienie.

W ogólnym przypadku korespondujące ze sobą punkty na obu obrazach muszą leżeć na odpowiadających sobie liniach epipolarnych. **Rektyfikacja** polega na zastosowaniu odpowiednich przekształceń (zwykle projekcyjnych) obu obrazów tak, aby odpowiadające sobie linie epipolarne stały się współliniowe i równoległe do jednej z krawędzi obrazu (najczęściej poziomej). W praktyce oznacza to, że dla punktów odpowiadających sobie ich współrzędna pionowa jest taka sama, a różnica (dysparycja) występuje tylko w kierunku poziomym. Znakomicie ułatwia to wyznaczenie korespondencji (mapy głębi) – poszukiwania prowadzi się wtedy tylko w jednym kierunku (wzdłuż wiersza obrazu).

R Warto wiedzieć, że w OpenCV dostępna jest również funkcja rektyfikacji obrazów nieskalibrowanych `stereoRectifyUncalibrated`. Zachęcamy do jej uruchomienia.

Ćwiczenie 11.2 Przeprowadź proces kalibracji oraz rektyfikacji układu kamer stereo. Wyświetl obrazy po usunięciu zniekształceń oraz nanieś na nie poziome linie, które potwierdzą poprawność działania algorytmu.

Ogólny schemat postępowania:

1. Wykorzystaj kod do kalibracji pojedynczej kamery jako punkt wyjścia.
2. Wprowadź modyfikacje, aby uwzględnić analizę układu dwóch kamer.
3. Wyznacz parametry intrinsics (i dystorsję) osobno dla lewego i prawego obrazu.
4. Przeprowadź kalibrację stereo, wykorzystując otrzymane parametry kamer (do wykorzystania kod: 11.1).
5. Wykonaj rektyfikację i wyświetl zrektyfikowane (skalibrowane) obrazy.
6. Połącz obrazy i narysuj na nich poziome linie, aby sprawdzić, czy punkty odpowiadające leżą w tych samych wierszach.

Listing 11.1: Stereo calibration

```
imgpointsLeft = np.asarray(imgpointsLeft, dtype=np.float64)
imgpointsRight = np.asarray(imgpointsRight, dtype=np.float64)

(RMS, __, __, __, __, rotationMatrix, translationVector) = cv2.fisheye.stereoCalibrate(
    objpoints, imgpointsLeft, imgpointsRight,
    K_left, D_left,
    K_right, D_right,
    image_size, None, None,
    cv2.CALIB_FIX_INTRINSIC,
    (cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_MAX_ITER, 30, 0.01))
```

```

R2 = np.zeros([3,3])
P1 = np.zeros([3,4])
P2 = np.zeros([3,4])
Q = np.zeros([4,4])

# Rectify calibration results
(leftRectification, rightRectification, leftProjection, rightProjection,
 disparityToDepthMap) = cv2.fisheye.stereoRectify(
    K_left, D_left,
    K_right, D_right,
    image_size,
    rotationMatrix, translationVector,
    0, R2, P1, P2, Q,
    cv2.CALIB_ZERO_DISPARITY, (0,0) , 0, 0)

map1_left, map2_left = cv2.fisheye.initUndistortRectifyMap(
    K_left, D_left, leftRectification,
    leftProjection, image_size, cv2.CV_16SC2)

map1_right, map2_right = cv2.fisheye.initUndistortRectifyMap(
    K_right, D_right, rightRectification,
    rightProjection, image_size, cv2.CV_16SC2)

dst_L = cv2.remap(img_l, map1_left, map2_left, cv2.INTER_LINEAR)
dst_R = cv2.remap(img_r, map1_right, map2_right, cv2.INTER_LINEAR)

```

R Opis parametrów wykorzystanych w powyższych metodach:

- `imgpointsLeft`, `imgpointsRight` – lista wykrytych narożników szachownicy dla lewego oraz prawego obrazu, wartości pochodzą z funkcji `cv2.cornerSubPix`,
- `K_left`, `K_right` – macierz parametrów wewnętrznych kamery dla zbioru par obrazów, wartości pochodzą z funkcji `cv2.fisheye.calibrate`,
- `D_left`, `D_right` – współczynniki zniekształceń kamery, wartości pochodzą z funkcji `cv2.fisheye.calibrate`.

11.3 Obliczenia korespondencji stereo

Samo wyznaczenie korespondencji stereo (mapy dysparycji, a w konsekwencji mapy głębi) jest stosunkowo proste – przynajmniej w teorii i dla odpowiednich zdjęć. Celem jest znalezienie, o ile piksel $I_L(x,y)$ jest przesunięty na drugim obrazie, tj. odpowiada mu piksel $I_R(x+d,y)$, gdzie d oznacza dysparycję. Dla przypomnienia: szukamy tylko w liniach poziomych, bo zakładamy, że obrazy zostały poddane rektyfikacji. Można też zauważyć, że zadanie to jest w pewnym sensie podobne do przepływu optycznego. Tam mieliśmy dwie kolejne ramki z sekwencji zarejestrowane jedną kamerą, a tu dwa obrazy z dwóch kamer w tym samym czasie. Warto też wiedzieć, że na podstawie sekwencji z pojedynczej kamery można odtworzyć głębię, o ile kamera jest ruchoma – jest to zagadnienie *Structure from Motion*.

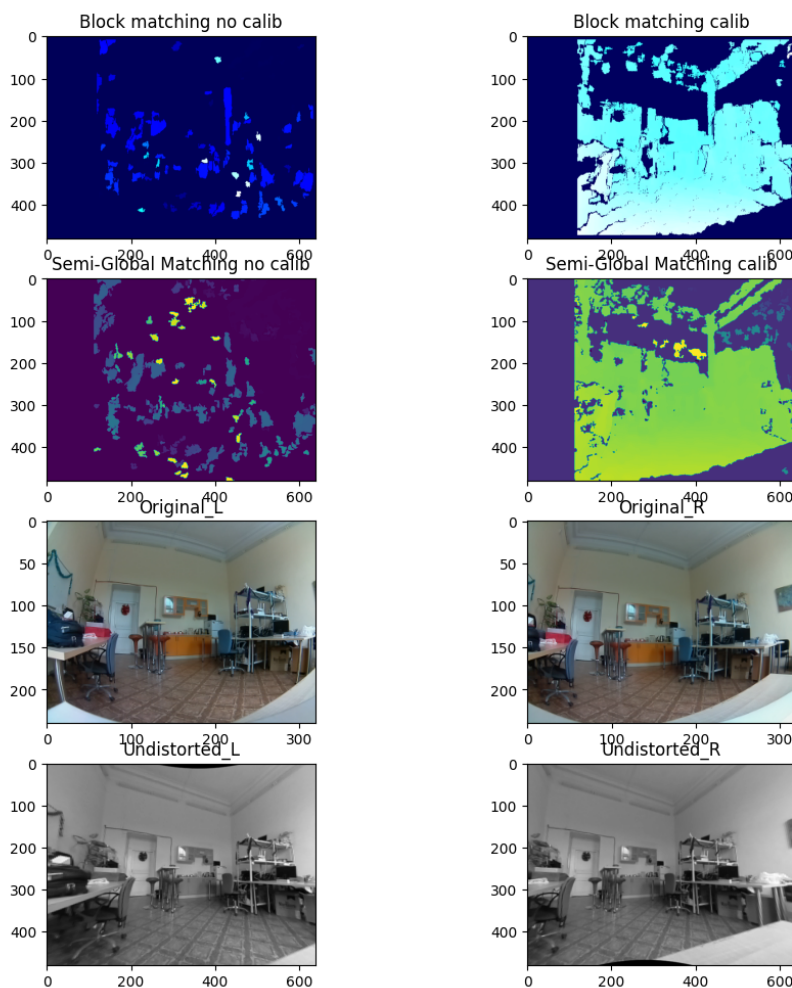
W bibliotece OpenCV dostępne są dwie popularne metody wyznaczania mapy dysparycji: dopasowanie bloków (*Block Matching*, `StereoBM`) oraz *Semi-Global Block Matching* (`StereoSGBM`). Działanie pierwszej jest dość intuicyjne; w przypadku drugiej stosuje się dodatkowo (w uproszczonej formie) optymalizację globalną, co zwykle poprawia ciągłość mapy.

Ćwiczenie 11.3 Proszę samodzielnie, na podstawie dokumentacji, wyznaczyć mapę dysparycji dla jednego z obrazów z folderu *example* przed i po procesie kalibracji. Para-

metry poszczególnych funkcji proszę dobrać empirycznie, stosując się do ograniczeń. Wyświetl obraz oryginalny przed i po kalibracji oraz odpowiednie mapy dysparycji (SGM i BM).

Przykładowe rozwiązanie zaprezentowano na rys. 11.4. ■

- R** Mapę dysparycji przed wyświetleniem należy znormalizować do przedziału 0 – 255. Można również zastosować funkcję do konwersji dysparycji do heatmapy: `cv2.applyColorMap(map, cv2.COLORMAP_HOT)`.



Rysunek 11.4: Przykładowe rozwiązanie.

11.4 Wykorzystanie sieci neuronowej do realizacji zadania analizy głębi obrazu

W ramach ćwiczenia proponujemy przeanalizować głęboką konwolucyjną sieć neuronową, która pozwala uzyskać mapy głębi na podstawie **pojedynczych** obrazków (metoda na *single-view depth*). Dodatkowo sieć uczona jest w sposób nienadzorowany (tj. bez nauczyciela – w tym przypadku bez referencyjnych map głębi). Ponadto sieć realizuje estymację pozycji kamery. Szczegóły dotyczące zastosowanej metody można przeczytać w artykule [ZHOU, Tinghui, et al., Unsupervised Learning of Depth and Ego-Motion from Video](#). Wykorzystamy udostępniony przez autorów model sieci. Kod jest dostępny na repozytorium GitHub – [SfmLearner-Pytorch](#).

Ćwiczenie 11.4 Zadanie nieobowiązkowe. Uruchom nienadzorowaną sieć neuronową SfmLearner.

Aby uruchomić wnioskowanie sieci neuronowej należy:

- Ustawić ścieżkę do nauczonego modelu sieci,
- Ustawić ścieżkę do obrazów testowych,
- Ustawić ścieżkę, gdzie mają być zapisane obrazy wynikowe,
- Ustawić flagi `output-disp`, `output-depth`, aby sieć zwróciła obrazy dysparycji oraz głębi.

```
python3 run_inference.py --pretrained 'dispnet_model_best.pth.tar' --dataset-dir  
'test_image/' --output-dir '/' --output-disp --output-depth
```

Porównaj wyniki uzyskane metodą DCNN, z klasycznymi BM i SGM (wizualnie).

■

12

Laboratorium 12

12	Termowizja	115
12.1	Prosta analiza obrazu termowizyjnego	
12.2	Wykorzystanie detektora bazującego na sieci neuronowej do analizy obrazu RGB + Termowizja (YOLOv3)	
12.3	Detekcja obiektów z wykorzystaniem wzorca probabilistycznego	

12. Termowizja

Obrazy termowizyjne mają tą zaletę, że obiekty, przykładowo ludzie czy zwierzęta, są na nich zwykle dość dobrze widoczne (tj. wyróżniają się od tła). Wyjątkiem jest sytuacja, gdy temperatura tła jest zbliżona do temperatury obiektu – przykładowo w bardzo ciepły dzień. Kolejna problematyczna sytuacja to ubiór, który dobrze izoluje od warunków zewnętrznych – przykładowo kurtka puchowa.

W ramach ćwiczenia zapoznamy się z trzema podejściami do zagadnienia:

- prosta detekcja obiektów z wykorzystaniem progowania i analizy obiektów,
- zastosowanie głębokiej konwolucyjnej sieci neuronowej i fuzji danych RGB i termowizyjnych,
- zastosowanie wzorca probabilistycznego (zadanie nieobowiązkowe).

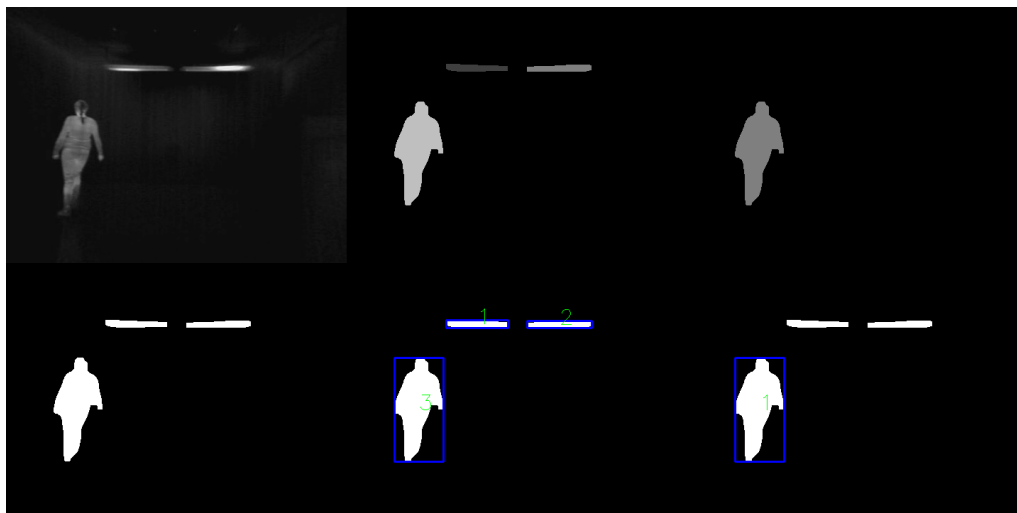
12.1 Prosta analiza obrazu termowizyjnego

Ćwiczenie 12.1 Przebieg ćwiczenia:

W tym ćwiczeniu należy przygotować prosty algorytm przetwarzania obrazu, którego celem jest wyodrębnienie interesujących obiektów oraz wyznaczenie dla nich prostokąta otaczającego. Zadanie realizuj krok po kroku zgodnie z poniższą listą.

1. Zastosuj binaryzację, filtrację oraz indeksację (etykietowanie) obiektów.
2. Pozostaw jedynie obiekty o odpowiednich proporcjach wysokości do szerokości.
3. Połącz wykryte części w jeden prostokąt otaczający (bounding box).
4. Przykładowy wynik algorytmu przedstawiono na rys. 12.1.

R Proszę założyć, że realizacja tego etapu ćwiczenia nie powinna trwać dłużej niż 30-35 min. Innymi słowy, proszę nie poświęcać zbyt dużo czasu na opracowanie algorytmu łączenia sylwetek (nawet kosztem jego skuteczności).



Rysunek 12.1: Przykładowy wynik algorytmu.

12.2 Wykorzystanie detektora bazującego na sieci neuronowej do analizy obrazu RGB + Termowizja (YOLOv3)

Pierwsze z podejść do analizy obrazu termowizyjnego pozwoliło zapoznać się z wyglądem oraz charakterystyką tego typu obrazów. Termowizja stanowi cenne źródło informacji, które może pełnić rolę dodatkowego kanału danych podczas analizy obrazów RGB. Fuzja danych z obrazów RGB i termowizyjnych stwarza możliwość poprawy skuteczności oraz dokładności detekcji, co można wykorzystać, stosując detektor bazujący na głębokiej konwolucyjnej sieci neuronowej.

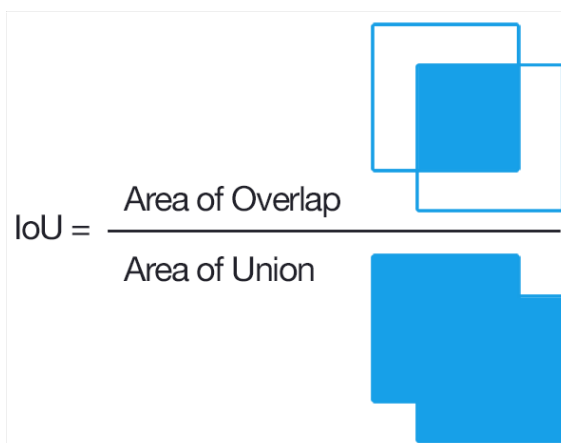
W przypadku wykorzystania sieci neuronowych fuzja danych może być realizowana na wiele sposobów:

1. wczesna (ang. **early fusion**) – połączenie danych na początku procesu detekcji. Obraz termowizyjny oraz RGB są łączone w jeden obraz, który następnie jest podawany do detektora wyznaczającego prostokąty otaczające obiekty oraz ich klasy.
2. pośrednia (ang. **middle fusion**) – detektor początkowo przetwarza obrazy osobno, jednak na pewnym etapie struktury sieci neuronowej mapy cech (ang. *feature maps*) reprezentujące oba obrazy są łączone. Finalnie na wyjściu sieci otrzymujemy prostokąty otaczające wraz z odpowiadającymi im klasami.
3. późna (ang. **late fusion**) – w tym przypadku dwa detektory otrzymują na wejściu po jednym obrazie. Na ich wyjściach uzyskuje się osobne wyniki, które następnie należy połączyć, np. poprzez znalezienie odpowiadających sobie prostokątów otaczających i uśrednienie ich parametrów, takich jak położenie czy rozmiar.

W ramach niniejszego ćwiczenia wykorzystane i porównane zostaną podejścia bazujące na wczesnej i późnej fuzji. Do wykonania tego zadania niezbędne jest pobranie zbioru zdjęć z bazy testowej, szablonu programu, a także wszystkich plików zawierających wagi i konfiguracje nauczonych detektorów YOLOv3, które są dostępne do pobrania na platformie UPeL.

Ćwiczenie 12.2 Uruchomienie sieci neuronowej.

1. W pliku `thermo_detection.py` ustaw ścieżki do `test_rgb` i `test_thermal` (*TODO0*).
2. W sekcji `METHOD` wybierz tryb: `FUSION = "EARLY"` lub `FUSION = "LATE"`.
3. **Wczesna fuzja:** OpenCV wczytuje kanały jako BGR, więc pozostaw B i G bez



Rysunek 12.2: *Intersection over Union*. Źródło: <https://en.wikipedia.org>

zmian, a kanał R ustaw jako wartość maksymalną z kanału R i termowizji w skali szarości (*TODO1*).

4. **Późna fuzja:** uruchom dwa detektory (RGB oraz termowizja w skali szarości), dopasuj do siebie ramki i uśrednij ich parametry, aby otrzymać jedną ramkę na jednego pieszego.
5. Dopasowanie ramek wykonaj z użyciem IoU (*Intersection over Union*, rys. 12.2); implementacja w *TODO2*.
6. Skrót algorytmu: policz IoU dla wszystkich par ramek *Rect1-Rect2* (dla $\text{IoU} > 0$), posortuj malejąco i zachłannie wybieraj pary tak, by każda ramka wystąpiła co najwyżej raz; dla wybranych par uśrednij współrzędne/rozmiary (rzutuj na *int*) i zapisz do *boxes*.
7. Uruchom program i porównaj działanie obu metod.

12.3 Detekcja obiektów z wykorzystaniem wzorca probabilistycznego

Inne podejście do zagadnienia detekcji pieszych polega na obserwacji, że sylwetki stojących osób mają dość charakterystyczny i powtarzalny wygląd. Pozwala to na stworzenie wzorca sylwetki, a następnie wyszukiwanie go na obrazie z zastosowaniem techniki okna przesuwanego.

Ćwiczenie 12.3 Zadanie nieobowiązkowe. Wzorzec probabilistyczny.

A. Zapisywanie sylwetek człowieka do pliku *png*.

W pierwszym etapie potrzebna będzie baza wycinków sylwetek o określonych rozmiarach – 192×64 px. W celu jej pozyskania wykorzystamy aplikację z pierwszej części ćwiczenia: dodajmy do niej możliwość wycinania obszaru zainteresowania (ROI) oraz zapisywania do plików *png* zweryfikowanych sylwetek.

1. Bazę wycinków realizujemy poprzez wycinanie ROI (Region of Interest) z obrazu.
2. Zapisujemy wyodrębnione ROI do osobnych plików *png*.
3. Wybieramy ręcznie ok. 30–50 zdjęć. Zasadniczo powinny one być frontalne, ale dopuszcza się również ujęcia z boku.

B. Tworzenie wzorca sylwetki.

1. W kolejnym kroku, w osobnym skrypcie należy wybrane zdjęcia wczytać, przeskalować i wyznaczyć wzorec. Skalowanie wykonujemy do rozmiaru 192×64 pikseli. Wzorec probabilistyczny powstaje na zasadzie $PDM = PDM + B$ (suma obrazów binarnych), przy czym B musi być binarny. Normalizujemy otrzymany wzorec, dzieląc przez liczbę obrazów.
2. Otrzymany wzorec zapisujemy do pliku. Wzorec powinien wyglądać podobnie do tego na rys. 12.3.

C. Detekcja obiektów.

1. Wczytaj sekwencję (jak w etapie 1). Najpierw przetestuj metodę na jednej ramce (np. rys. 12.4).
2. Wczytaj wzorec (ew. konwersja do skali szarości), skonwertuj do *float32* i podziel przez liczbę obrazów (skala 0–1). Oznacz jako PDM_1 oraz policz $PDM_0 = 1 - PDM_1$.
3. Zbinaryzuj ramkę testową i zastosuj okno przesuwne 192×64 (krok może być > 1). Dla maski binarnej (B) oblicz:

$$\sum_y \sum_x (B(y, x) * PDM_1(y, x) + (1 - B(y, x)) * PDM_0(y, x)) \quad (12.1)$$

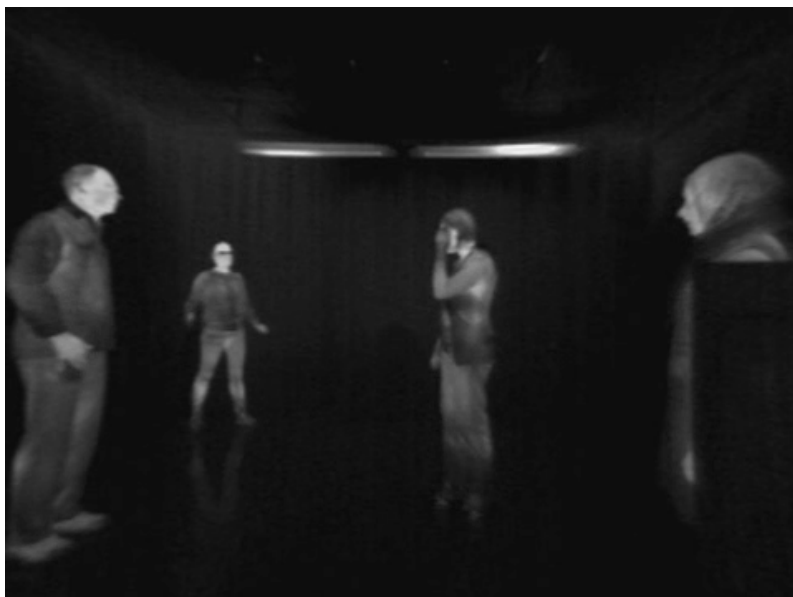
4. Do poprawnej wizualizacji znormalizuj wyniki:

```
result = result / np.max(np.max(result))
result_uint8 = np.uint8(result*255)
```

5. Znajdź maksima lokalne w `result`. Najprościej: wybierz maksimum w trakcie skanowania okien (da tylko jedno trafienie) i narysuj prostokąt otaczający.
6. Dla wielu detekcji: ustaw próg dolny i zastosuj eliminację obszarów już odwiedzonych (*Non-Maximum Suppression*).
7. Do przemyślenia: obecnie wykrywasz tylko jedną skalę (192×64). Jak dodać detekcję w wielu skalach? Zapisz komentarz w kodzie lub zaimplementuj.



Rysunek 12.3: Przykładowy wzorec.



Rysunek 12.4: Przykładowa ramka.

13

Extra laboratorium 3

13	Odometria wizyjna	123
13.1	Odometria wizyjna	
13.2	Śledzenie obiektów - zadanie	

13. Odometria wizyjna

13.1 Odometria wizyjna

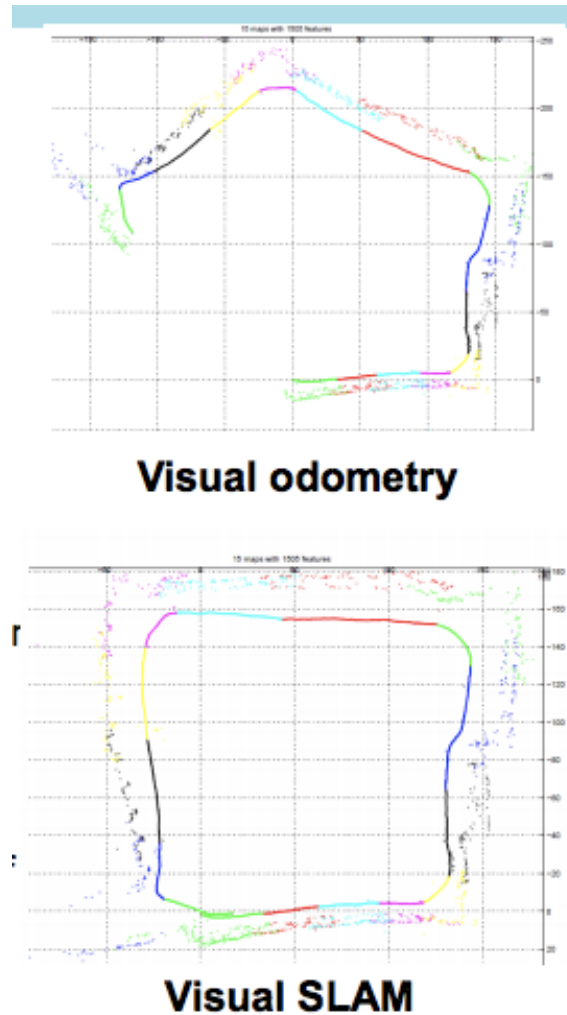
Odometrię wizualną (VO) definiuje się jako proces szacowania ruchu robota (translacji i rotacji względem ramki odniesienia) poprzez obserwację sekwencji obrazów jego otoczenia. VO jest szczególnym przypadkiem techniki znanej jako Structure From Motion (SFM), która zajmuje się problemem trójwymiarowej rekonstrukcji zarówno struktury otoczenia, jak i pozycji kamery z sekwencyjnie uporządkowanych lub nieuporządkowanych zestawów obrazów. Ostatni etap SFM, polegający na ostatecznym dopracowaniu i globalnej optymalizacji zarówno pozycji kamery, jak i struktury, jest kosztowny obliczeniowo i zwykle wykonywany w trybie off-line. Natomiast estymacja pozycji kamery w VO musi być przeprowadzana w czasie rzeczywistym. W ostatnich latach zaproponowano wiele metod VO, które można podzielić na metody z wykorzystaniem kamer monokularnych i stereokamerowych. Metody te są następnie dzielone na dopasowywanie cech (dopasowywanie cech na wielu klatkach), śledzenie cech (dopasowywanie cech w sąsiednich klatkach) oraz techniki przepływu optycznego (oparte na intensywności wszystkich pikseli lub określonych regionów w sekwencyjnych obrazach).

Jednoczesna lokalizacja i mapowanie (SLAM) jest rozszerzeniem metody VO, która radzi sobie z dryfem, ale także SLAM jest procesem, w którym robot ma za zadanie zlokalizować się w nieznanym środowisku i jednocześnie zbudować jego mapę bez żadnych wcześniejszych informacji za pomocą zewnętrznych czujników (lub pojedynczego czujnika).

Odometria wizyjna składa się z kilku kroków:

1. Detekcja punktów charakterystycznych (zwykle ORB - FAST+BRIEF)
2. Dopasowanie punktów charakterystycznych (Brute-force lub FLANN)
3. Generowanie chmury punktów 3D - triangulacja
4. Eliminacja błędnej reprojekcji, filtracja RANSAC
5. Estymacja ruchu - wyznaczenie rotacji oraz translacji

13.2 Śledzenie obiektów - zadanie



Rysunek 13.1: Porównanie działania VO i SLAM.

Ćwiczenie 13.1 Celem ćwiczenia jest realizacja prostej odometrii wizyjnej, czyli wyznaczenie trajektorii ruchu samochodu poruszającego się po ulicach na podstawie zbioru danych KITTI.

1. Pobierz materiały oraz szablon zadania z platformy UPeL.
2. Zadanie zostało przygotowane w notatniku Jupyter. Plik `.ipynb` można otworzyć i uruchomić za pomocą Google Colab (colab.research.google.com). Również możliwe jest uruchomienie pliku za pomocą edytora VS Code. Prawdopodobnie będzie wymagane zainstalowanie odpowiednich bibliotek. Jeśli wybieramy VS Code pomijamy wczytywanie biblioteki:

```
from google.colab import drive
drive.mount('/content/drive')
```

3. Wykonaj polecenia umieszczone w pliku.
4. Rozwiązaniem zadania jest uzupełniony oraz działający plik `.ipynb`, który należy wysłać na platformę UPeL.